



GRADO EN INGENIERÍA
MULTIMEDIA



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE GRADO

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA DE
REPRODUCCIÓN MUSICAL INMERSIVA 8D
MEDIANTE SEPARACIÓN DE STEMS,
INTELIGENCIA ARTIFICIAL Y VISUALIZACIÓN
TRIDIMENSIONAL INTERACTIVA.

AUTOR: CARLOS RUEDA MARTÍNEZ

TUTOR: MIGUEL LOZANO IBÁÑEZ

JUNIO 2026



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA DE
REPRODUCCIÓN MUSICAL INMERSIVA 8D
MEDIANTE SEPARACIÓN DE STEMS,
INTELIGENCIA ARTIFICIAL Y VISUALIZACIÓN
TRIDIMENSIONAL INTERACTIVA.

AUTOR: CARLOS RUEDA MARTÍNEZ

TUTOR: MIGUEL LOZANO IBÁÑEZ

Declaración de autoría:

Yo, Carlos Rueda Martínez, declaro la autoría del Trabajo Fin de Grado titulado “Diseño e implementación de un sistema de reproducción musical inmersiva 8D mediante separación de stems, inteligencia artificial y visualización tridimensional interactiva.” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores. El documento se licencia bajo los términos vinculantes de una *General Public License* (GPL).

Valencia, 26 de junio de 2026

**CARLOS|RUEDA|
MARTINEZ**



Firmado digitalmente por CARLOS|RUEDA|
MARTINEZ
Nombre de reconocimiento (DN): cn=CARLOS|
RUEDA|MARTINEZ, serialNumber=49429713E,
givenName=CARLOS, sn=RUEDA MARTINEZ,
ou=CIUDADANOS, o=ACCV, c=ES
Fecha: 2026.06.26 02:44:16 +02'00'

Fdo: Carlos Rueda Martínez

Resumen:

El presente trabajo de fin de grado (TFG) aborda el desarrollo de un reproductor 8D inmersivo utilizando separación mediante inteligencia artificial (IA) y Funciones de Transferencia Relativa a la Cabeza (HRTF). El objetivo principal es construir un sistema que, contenido en un espacio 3D, permita democratizar el acceso a la generación, modificación y creación de música espacial de calidad. Para lograrlo, se ha empleado una metodología de desarrollo incremental orientada a la creación de un producto funcional y de código abierto. El flujo de trabajo del sistema integra un primer módulo basado en Inteligencia Artificial para la separación automática de las fuentes musicales, seguido de un procesador de espacialización basado en Procesamiento Digital de Señales (DSP). Este módulo aplica los principios psicoacústicos de la audición binaural para simular el movimiento del sonido en un espacio tridimensional a través de impulsos sonoros (HRIR). Visualmente, la arquitectura del sistema ha sido integrada y sincronizada empleando la interfaz y API de Blender. Las pruebas de rendimiento y las evaluaciones de calidad de audio demuestran que el algoritmo desarrollado procesa y exporta la pista 8D con éxito, manteniendo un consumo de recursos viable y mitigando la aparición de posibles artefactos audibles como clicks en la pista final. A efectos prácticos, el presente TFG proporciona una herramienta práctica y escalable para la automatización del audio inmersivo en entornos 3D.

Palabras clave: Audio 8D, reproductor, binaural, Inmersivo, espacialización, Blender.

Abstract:

This Bachelor's Thesis (TFG) addresses the development of an immersive 8D audio player using artificial intelligence (AI) separation and Head-Related Transfer Functions (HRTF). The main objective is to build a system that, contained within a 3D space, allows democratizing access to the generation, modification, and creation of high-quality spatial music. To achieve this, an incremental development methodology has been employed, aimed at creating a functional and open-source product. The system's workflow integrates a first module based on Artificial Intelligence for the automatic separation of musical sources, followed by a spatialization processor based on Digital Signal Processing (DSP). This module applies the psychoacoustic principles of binaural hearing to simulate the movement of sound in a three-dimensional space through sound impulses (HRIR). Visually, the system's architecture has been integrated and synchronized using Blender's interface and API. Performance tests and audio quality evaluations demonstrate that the developed algorithm successfully processes and exports the 8D track, maintaining viable resource consumption and mitigating the appearance of possible audible artifacts such as clicks in the final track. For practical purposes, this thesis provides a practical and scalable tool for the automation of immersive audio in 3D environments.

Keywords: audio, reproduction, 8d, binaural, immersive, spacialization, blender.

Resum:

El present treball de fi de grau (TFG) aborda el desenvolupament d'un reproductor 8D immersiu utilitzant separació mitjançant intel·ligència artificial (IA) i Funcions de Transferència Relativa al cap (HRTF). L'objectiu principal és construir un sistema que, contingut en un espai 3D, permeti democratitzar l'accés a la generació, modificació i creació de música espacial de qualitat. Per aconseguir-ho, s'ha emprat una metodologia de desenrotllament incremental orientada a la creació d'un producte funcional i de codi obert. El flux de treball del sistema integra un primer mòdul basat en Intel·ligència Artificial per a la separació automàtica de les fonts musicals, seguit d'un processador d'espacialització basat en Processament Digital de Senyals (DSP). Este mòdul aplica els principis psicoacústics de l'audició binaural per a simular el moviment del so en un espai tridimensional a través d'impulsos sonors (HRIR). Visualment, l'arquitectura del sistema ha sigut integrada i sincronitzada emprant la interfície i API de Blender. Les proves de rendiment i les avaluacions de qualitat d'àudio demostren que l'algorisme desenvolupat processa i exporta la pista 8D amb èxit, mantenint un consum de recursos viable i mitigant l'aparició de possibles artefactes audibles com a clics en la pista final. A efectes pràctics, el present TFG proporciona una ferramenta pràctica i escalable per a l'automatització de l'àudio immersiu en entorns 3D.

Paraules clau: Àudio 8D, reproductor, binaural, Immersiu, espacialització, Blender.

Agradecimientos:

Doy gracias a mi familia, soy increíblemente afortunado. En especial a mi padre, por ponernos música para desayunar, almorzar, comer, merendar y cenar. Gracias por dejarme golismear tus discos. También a mi madre, por su sacrificio diario y por, contra todo pronóstico, estar siempre ahí.

A Javier, Gema, Rubén, Diana, mi compai Maurizio, a Inés, Pedro, al mejor compañero de piso, Carlos Heras, a Miguel, Pascu, Edu, y *Χριστίνα*, mi segunda madre durante seis maravillosos meses. Personas a las que les profeso un gran cariño. Ama, ama, ama y ensancha el alma.

A la marquesa, una luchadora. Por tener la suerte de seguir escuchando tus historias cuando estás al otro lado del teléfono todos los días al salir de la biblioteca. Te quiero mucho, ¡y que no se te olvide!

Quedan immortalizados todos los chicos *Road To Gold* y el *Consejo de Sabios*. Estamos ahí, believing in it.

Por último, a mi tutor, Miguel Lozano, por compartir mi visión sobre la música y echarme una mano cuando se ha necesitado durante el desarrollo de este trabajo.

Estudiar es un privilegio, pero más lo es tu salud. Prioriza siempre tu salud mental, no juegues con ella, pues es difícil no dejarse llevar. Somos lo que pensamos, y con ello, construimos el mundo que nos rodea.

Ah, y a quienes no creyeron en mí, ¡A ellos también se lo agradezco! <3.

A Francisco Martínez García, de quien no pude despedirme a tiempo. Quienes tuvimos la suerte de coincidir contigo y disfrutar de tu felicidad atesoramos los mejores recuerdos. Por aquí seguimos bailando, pero no tanto como tú. Por los siglos de los siglos. Q.E.P.D

Après un rêve.

Índice general

Resumen	7
Índice de figuras	21
Índice de tablas	23
Índice de códigos	25
Lista de acrónimos	27
1. Introducción	29
1.1. Introducción	29
1.2. Motivación	30
1.3. Objetivos	30
1.4. Estructura de la memoria	31
2. Estado del arte	33
2.1. Fundamentos teóricos	33
2.1.1. Audición espacial	33
2.1.2. Audición binaural	34
2.1.2.1. Arquitectura del espacio sonoro	34
2.1.2.2. Teoría Dúplex y lateralidad	35
2.1.2.3. El cono de confusión	38
2.1.2.4. Pistas espectrales	40
2.1.2.5. Intensidad relativa	42
2.1.2.6. Externalización	44
2.1.3. Representación de una señal de onda digital	46
2.1.4. HRTF	46
2.1.4.1. Convolución de HRTF en cadenas DSP	49
2.2. Tecnologías habilitadoras	51
2.2.1. Python	51

2.2.2.	Blender	52
2.2.3.	Demucs	53
2.3.	Análisis del mercado	54
2.3.1.	Tipos de herramientas existentes y limitaciones	54
2.3.2.	Aspectos diferenciadores de la propuesta	55
3.	Planificación	57
3.1.	Metodología de desarrollo	57
3.2.	Estructura del desglose de trabajo	58
3.3.	Asignación de perfiles profesionales	59
3.4.	Estimación temporal	59
3.4.1.	Diagrama de Gantt	61
3.5.	Estimación de costes	63
3.5.1.	Costes personales	63
3.5.2.	Costes de software	63
3.6.	Análisis de viabilidad del proyecto	64
3.6.1.	Viabilidad técnica	64
3.6.2.	Viabilidad económica	64
3.6.3.	Viabilidad legal	65
3.7.	Análisis de riesgos	65
4.	Análisis	69
4.1.	Requisitos funcionales y no funcionales	69
4.1.1.	Requisitos funcionales	69
4.1.2.	Requisitos no funcionales	70
4.2.	Diagrama de casos de uso	71
4.2.1.	Descripción de los casos de uso	72
4.3.	Diagrama de clases conceptual	75
4.3.1.	Descripción de atributos del DCC	77
4.4.	Diagrama de secuencia conceptual	77
5.	Diseño	79
5.1.	Arquitectura	79
5.2.	Diseño lógico	81
5.2.1.	Diagrama de paquetes	82
5.2.2.	Diagrama de clases	82
5.3.	Diseño de procesos	85
5.3.1.	Mockup de la UI	85

5.3.2.	Diagrama de actividad del CU-02	86
5.3.3.	Diagrama de secuencia general	86
5.3.4.	Diseño de la DSP	88
5.4.	Diseño de datos	91
5.5.	Documentación para addons en Blender	91
6.	Implementación	95
6.1.	Incremento 1: Arquitectura base y módulo de IA	95
6.1.1.	Módulo de separación de pistas por AI	98
6.2.	Incremento 2: Escena 3D y procesador 8D	102
6.2.1.	Implementación de los aspectos visuales de la escena	102
6.2.2.	Implementación del procesador 8D	107
6.3.	Incremento 3: Módulo de exportación	113
7.	Pruebas	117
7.1.	Pruebas de rendimiento	117
7.2.	Pruebas de calidad de audio	118
7.2.1.	Escenario 1: Posicionamiento estático	119
7.2.2.	Escenario 2: Posicionamiento dinámico	122
7.2.3.	Escenario 3: Posicionamiento mixto	126
8.	Conclusiones	129
8.1.	Contraste empírico del proyecto temporal y económico	129
8.2.	Evaluación de objetivos, requisitos y riesgos	132
8.3.	Líneas de trabajo futuro	135
A.	Resolución de diagramas, tablas y declaración de uso de IA	137
A.1.	Descripción de atributos del DCC	138
A.2.	DA del CU-02	139
A.3.	Bloque de código de la inicialización de paneles	140
A.4.	Bloque de código de la inicialización de operadores	141
A.5.	Bloque de código de una trayectoria dinámica	142
A.6.	Bloque de código de la convolución dinámica	143
A.7.	DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL	144
	Bibliografía	145

Índice de figuras

1.1. Esquema general de la propuesta	30
2.1. Sistema de coordenadas relativo a la cabeza.	34
2.2. Cognición de una onda por ITD.	35
2.3. Cognición de una onda por ILD.	35
2.4. Valor de ITD con respecto al ángulo azimutal de una fuente.	36
2.5. Valor de ILD con respecto a un rango azimutal y frecuencial.	37
2.6. Ejemplificación de un cono de confusión cualquiera del oído izquierdo.	38
2.7. Cono de confusión a ambos lados de la cabeza.	39
2.8. Ejemplo de pistas espectrales resultantes para un bajo.	40
2.9. Función DTF del oído izquierdo y sus muescas espectrales.	41
2.10. Representación de la Ley Inversa del cuadrado.	42
2.11. Curvas isofónicas.	44
2.12. Representación de una señal de onda digital genérica tras aplicar un ADC y escalado específicos.	46
2.13. Puntos de medición para HRTF.	47
2.14. Medición $HRTF_L$ para $\theta = 90^\circ$	48
2.15. Medición $HRTF_R$ para $\theta = 90^\circ$	48
2.16. Utilización de FFT para convertir las distintas señales en el dominio fre- cuencial.	50
2.17. Logotipo de Python	51
2.18. Logotipo de Blender	52
2.19. Logotipo de Meta, desarrolladores de Demucs.	53
2.20. Interfaz principal de Sennheiser AMBEO Orbit.	55
3.1. Diagrama de Gantt estimado del proyecto.	62
4.1. DCDU del sistema de reproducción 8D.	71
4.2. DCC del sistema de reproducción 8D	76
4.3. DSC del sistema de reproducción 8D	78
5.1. DP del sistema de reproducción 8D	82

5.2.	Diagrama de clases del sistema.	83
5.3.	Mockup UI del sistema de reproducción 8D	85
5.4.	Diagrama de secuencia general del sistema.	87
5.5.	Diagrama de actividad del procesador DSP.	89
6.1.	Mockup UI del sistema de reproducción 8D	98
6.2.	Tiempo de separación para la pista <i>AI James - Schoolboy Facination</i> . . .	101
6.3.	Verificación de las subpistas de <i>AI James - Schoolboy Facination</i>	101
6.4.	Representación de la cabeza en Blender	103
6.5.	Representación de las subpistas estáticas en blender	104
6.6.	Representación de las subpistas dinámicas en blender	107
6.7.	Vista del addon con la pista 8D cargada	114
6.8.	Ejecución del operador de exportación	115
6.9.	Vista del explorador de archivos al exportar una pista 8D	115
6.10.	Vista del reproductor al iniciar el addon por primera vez	115
7.1.	Unidades de los espectrogramas analizados en los tres escenarios	119
7.2.	Espectrogramas resultantes de la convolución estática para cada subpista del Escenario 1.	120
7.3.	Espectrogramas resultantes de la separación de las subpistas vocals y drums del Escenario 1.	121
7.4.	Vista del escenario 2	123
7.5.	Espectrogramas L y R correspondientes a distintos puntos de elevación de la pista drums del Escenario 2.	124
7.6.	Espectrograma de la pista Other L y R sin procesar del Escenario 2.	125
7.7.	Espectrograma de la pista Other 8D L y R del Escenario 2. Eje OY - Hz .	126
7.8.	Espectrograma de la pista final 8D L y R del Escenario 3. Eje OX - Tiempo, Eje OY - Hz	127
7.9.	Espectrograma de la pista final sin procesar L y R del Escenario 3. Eje OX - Tiempo, Eje OY - Hz	127
8.1.	Diagrama de Gantt real del proyecto.	130
A.1.	Diagrama de actividad del CU-02	139

Índice de tablas

2.1. Comparativa de características entre los principales motores 3D.	53
2.2. Comparativa entre los ejemplos mencionados y la propuesta	56
3.1. Estimación temporal de la Fase 1	60
3.2. Estimación temporal de la Fase 2	60
3.3. Estimación temporal de la Fase 3	60
3.4. Estimación temporal de la Fase 4	61
3.5. Estimación temporal de la Fase 5	61
3.6. Resumen de la estimación temporal total del proyecto	61
3.7. Desglose de los costes directos personales para cada perfil profesional . . .	63
3.8. Desglose de los costes personales	63
3.9. Costes indirectos de software y licencias del proyecto.	64
3.10. Evaluación de riesgos del proyecto.	66
4.1. Especificación del CU-01: <i>Cargar pista de audio</i>	72
4.2. Especificación del CU-02: <i>Generar pista 8D</i>	73
4.3. Especificación del CU-03: <i>Modificar trayectorias manualmente</i>	74
4.4. Especificación del CU-04: <i>Exportar la pista de audio resultante</i>	75
5.1. Comparativa entre las distintas arquitecturas	80
7.1. Resultados empíricos de las pruebas de rendimiento del motor DSP.	118
7.2. Promedio global de las métricas de rendimiento evaluadas en las cuatro pruebas.	118
7.3. Parámetros de la escena y de las subpistas del Escenario 1.	119
7.4. Parámetros de la escena y de las subpistas del Escenario 2.	122
7.5. Parámetros de la escena y de las subpistas del Escenario 3.	127
8.1. Desvío de la planificación temporal frente a la real.	131
8.2. Desglose de los costes personales reales post-desarrollo.	132
8.3. Costes indirectos de software y licencias reales reales.	132

8.4. Comparativa de costes personales brutos estimados frente a al real y su desvío equivalente.	132
8.5. Resumen porcentual del grado de consecución de los objetivos, requisitos y la mitigación de los riesgos del proyecto.	134
A.1. Identificación y descripción de los atributos del DCC.	138

Índice de códigos

5.1. Estructura de inicialización y metadatos de un addon en Blender.	92
5.2. Creación de un panel básico utilizando la API de Blender.	93
5.3. Creación de un operador básico utilizando la API de Blender.	93
6.1. Instalación base del addon según dependencias.	96
6.2. Modelo de datos del sistema.	97
6.3. Conexión operador IA, kernel y plugin.	99
6.4. Plugin de separacion con IA.	100
6.5. Generación del modelo esférico en Blender.	102
6.6. Inicialización de subpistas en Blender.	104
6.7. Generación de la trayectoria estática en Blender.	106
6.8. JSON con las propiedades de la escena y subpistas.	108
6.9. Bucle de carga de las HRIR.	109
6.10. Búsqueda de una HRIR en diffuse.zip	110
6.11. Proceso de convolución estática	111
6.12. Proceso de masterización de la pista 8D	112
6.13. Proceso carga de la pista 8D al timeline de Blender	113
6.14. Proceso de exportación de la pista 8D del Video secuenciador	114
A.1. Inicialización de los paneles del reproductor.	140
A.2. Inicialización de los operadores del reproductor.	141
A.3. Sistema de generación de trayectorias dinámicas.	142
A.4. Proceso de convolución dinámica.	143

Lista de acrónimos

ADC *Analog-to-digital converter*. 46

AI *Artificial Intelligence*. 29, 51, 53, 55, 56, 58, 60, 65, 66, 69, 72, 74, 76, 77, 80, 81, 84, 98, 99, 107, 125, 131–133, 135, 138

API *Application Programming Interface*. 53, 58, 60, 65, 66, 91, 105, 133

CCO *Creative Commons*. 65

CU *Caso de uso*. 72–75, 78, 80, 86

D-CDU *Diagrama de Casos de Uso*. 70, 71, 78

DA *Diagrama de Actividad*. 86, 88

DAW *Digital Audio Workstation*. 54, 56

DC *Diagrama de Clases*. 82, 84, 88

DCC *Diagrama de Clases Conceptual*. 75–77, 82, 84, 88

DG *Diagrama de Gantt*. 61, 63, 129

DP *Diagrama de Paquetes*. 82, 91

DSC *Diagrama de Secuencia Conceptual*. 77, 78, 85, 86, 88

DSG *Diagrama de Secuencia General*. 86, 88

DSP *Digital Signal Processing*. 46, 48–51, 56–58, 60, 70, 73, 76, 84–86, 88, 90, 98, 102, 110, 112, 113, 131–133, 138

DTF *Directional Transfer Function*. 41

EDT *Estructura de desglose del trabajo*. 58, 59, 61

FFT *Fast Fourier Transform*. 50, 90, 111, 118

FIR *Finite Impulse Response*. 48

GPL *General Public License*. 5, 65

GUI *Graphical User Interface*. 51–53

HRIR *Head-Related Impulse Response*. 48–50, 67, 73, 76, 80, 84, 88, 90, 91, 108–112, 118, 119, 121, 122, 125, 133, 138

HRTF *Head-Related Transfer Function*. 29, 46–50, 70, 84, 105, 121

IFFT *Inverse Fast Fourier Transform*. 50

ILD *Interaural Level Difference*. 35, 37, 38, 40, 43, 45, 119, 121

ITD *Interaural Time Difference*. 35–38, 40, 45, 119, 121

LPI *Ley de Propiedad Intelectual*. 65

MIT *Massachusetts Institute of Technology*. 49, 109, 138

MVC *Modelo-Vista-Controlador*. 81, 82, 84, 99

NR *Nivel de riesgo*. 66

RF *Requisito Funcional*. 69, 70, 72–75, 81, 86, 129, 132, 133

RGPD *Reglamento General de protección de Datos*. 65

RNF *Requisito No Funcional*. 69, 70, 72–75, 81, 86, 90, 117, 118, 129, 132–134

SDR *Signal-To-Distorsion ratio*. 54, 65

TFG *Trabajo de Fin de Grado*. 29, 30, 134, 135, 144

UI *User Interface*. 57, 58, 60, 82, 85, 86, 91, 98, 103, 109, 131

UX *User Experience*. 57

Capítulo 1

Introducción

El siguiente capítulo tiene como objetivo presentar e introducir el tema del *Trabajo de Fin de Grado* (TFG), al igual que la motivación que deriva de esta. El capítulo también incluye los objetivos a conseguir y la estructura de la memoria técnica.

1.1. Introducción

En los últimos años, la industria musical ha experimentado una evolución tecnológica de las técnicas de reproducción sonora, buscando ofrecer al usuario experiencias auditivas cada vez más inmersivas. Frente a los sistemas estéreo convencionales, ha tenido una enorme popularidad el concepto de audio espacial o más conocido comercialmente como "música 8D". Siendo el término 8D una estrategia de marketing para atraer interés. Estas tecnologías se fundamentan en recrear los procesos biológicos de la audición binaural para simular que las fuentes de sonido provienen de múltiples direcciones dentro de un espacio tridimensional, dotándolas de propiedades espaciales para generar un efecto inmersivo o de externalización.

A pesar de tener un formato atractivo para muchos consumidores y profesionales, en la actualidad se enfrenta a importantes barreras de entrada. Las herramientas existentes en el mercado exigen licencias de pago y requieren profundos conocimientos técnicos en ingeniería acústica para poder obtener un buen rendimiento de ellas. Además, estos sistemas asumen la necesidad de tener en posesión las pistas de audio individuales para poder espacializarlas, lo que limita severamente al usuario que solo dispone de archivos musicales estéreo ya masterizados.

Para dar respuesta a esta problemática del estado actual, este TFG propone el diseño y la implementación de un sistema de reproducción inmersiva 8D de código abierto. Integrando un módulo de *Artificial Intelligence* (AI) capaz de procesar y separar una pista estéreo en sus instrumentos básicos, solventando la problemática sobre la posesión de los masters.

Continuando, el sistema incorpora un flujo algorítmico que aplica *Head-Related Transfer Function* (HRTF) a una pista de audio, realizando una convolución con el objetivo de otorgar propiedades espaciales a la pista. Todo ello estará encapsulado como un add-on interactivo dentro del motor de desarrollo de Blender, lo que proporciona al usuario un espacio de visualización donde configurar las propiedades binaurales del sonido antes de exportar la pista 8D final. La figura 1.1 presenta un esquema general de la solución propuesta. Finalmente, en base al esquema y la motivación general del proyecto, se desa-

rollarán los objetivos del TFG.

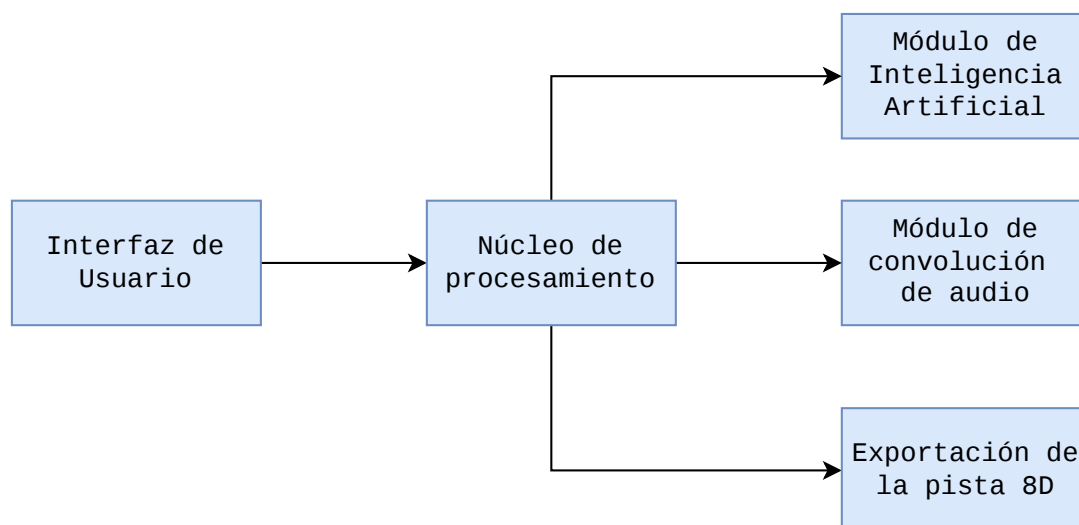


Figura 1.1: Esquema general de la propuesta

1.2. Motivación

La motivación del proyecto surge con fines de tener una herramienta que permita reproducir, crear y exportar música 8D de forma sencilla y transparente. Democratizando el acceso a este tipo de música permitiendo interactuar con un formato de audio más inmersivo que el estéreo. El proyecto quiere acercar tanto a personas que no tienen conocimientos técnicos, como profesionales del mundo audiovisual, reduciendo las barreras de interacción al mínimo. Finalmente, la iniciativa está orientada a ofrecer una alternativa tecnológica de código abierto y totalmente gratuita, eliminando las barreras económicas y reduciendo la dependencia frente a las plataformas comerciales cerradas y servicios de terceros actuales.

1.3. Objetivos

A continuación, se detallan los objetivos del presente TFG, derivados de la motivación y problemática previamente expuestas. Dicho esto, la descripción de los objetivos es la siguiente, empezando por el principal y más importante.

El **objetivo principal** del TFG es **diseñar e implementar un sistema de reproducción musical inmersiva 8D** que integre un módulo de separación de pistas mediante inteligencia artificial y otro de espacialización 8D dentro de un entorno 3D interactivo.

Después, la descripción de los subobjetivos representan la implementación de los bloques del esquema de la figura 1.1, los cuales a su vez determinarán la consecución del objetivo principal:

- **O1-Desarrollar una interfaz gráfica tridimensional interactiva:** construir un

entorno visual utilizando la API de Blender y así permitir al usuario parametrizar y visualizar las trayectorias espaciales de cada subpista en tiempo real.

- **O2-Implementar un módulo de separación mediante IA:** integrar un modelo de inteligencia artificial capaz de separar una pista de audio estéreo en sus instrumentos fundamentales.
- **O3-Implementar un motor algorítmico de convolución de audio:** implementar el procesamiento digital de señales necesario para aplicar la convolución utilizando respuestas al impulso binaurales que correspondan a las coordenadas dictadas por la interfaz 3D.
- **O4-Garantizar la exportación y validación del audio generado:** unificar y masterizar las subpistas procesadas en un único archivo 8D exportable, verificando que el rendimiento algorítmico y la fidelidad acústica cumplen con estándares donde el audio quede libre de artefactos.

1.4. Estructura de la memoria

La memoria se articula secuencialmente para facilitar la comprensión lógica del proyecto, si bien el desarrollo del mismo se ha ejecutado rigiéndose por el ciclo de vida del desarrollo de software incremental propuesto en la planificación metodológica. Aclarado el orden de lectura e implementación, la memoria está separada en ocho capítulos.

Capítulo 1, la introducción da a conocer el proyecto con el fin de entender la motivación y los objetivos propuestos que llevan a cabo la realización del mismo.

Capítulo 2, el estado del arte presenta los aspectos teóricos, técnicos, conjunto de herramientas y análisis del mercado que moldea el proyecto. Este contiene todo el material sobre el que están fundamentadas las decisiones de diseño e implementación junto con las afirmaciones expuestas en el capítulo 7 de pruebas.

Capítulo 3, la planificación justifica y desarrolla la metodología incremental utilizada para la organización de las tareas a beneficio del tipo de proyecto que se quiere llevar a cabo. En él se desglosan los costes temporales y económicos, desarrollando también la viabilidad técnica, económica y legal junto con los posibles riesgos que pudieran surgir durante el desarrollo.

Capítulo 4, el análisis aborda *qué* hace el proyecto en detalle presentando el conjunto de requisitos y diagramas conceptuales que permitan comprenderlo a gran escala, sin necesidad de conocimientos técnicos.

Capítulo 5, el diseño aborda *cómo* se llevará a cabo el proyecto desarrollando los diagramas descritos en el capítulo anterior integrando la arquitectura, patrón de desarrollo y especificaciones de programación del proyecto.

Capítulo 6, la implementación desarrolla cómo se han integrado todos los distintos incrementos y el código fuente en general. En él se justifican las decisiones tomadas durante la implementación.

Capítulo 7, las pruebas abordan un análisis y comprobación de que el rendimiento y calidad de audio responden a las características esperadas del tipo de reproductor 8D.

Capítulo 8, las conclusiones presentan una comparación entre la planificación estimada

y real junto con una evaluación de objetivos, requisitos y riesgos. Al final, se proponen las líneas de trabajo futuro a terceros con fines de seguir desarrollando el proyecto.

Capítulo 2

Estado del arte

A continuación, se presentan los fundamentos teóricos, tecnologías habilitadoras para la construcción del proyecto y un análisis del mercado con las herramientas existentes actualmente en comparación con el proyecto. El fin de este capítulo es introducir los conceptos suficientes a fines de entender la cometido del proyecto.

2.1. Fundamentos teóricos

El propósito de este apartado es conocer los conceptos teóricos y técnicos necesarios para la creación de un sistema de reproducción inmersiva en tres dimensiones. Los conceptos aquí presentados explican cómo el ser humano es capaz de localizar y procesar el sonido a través de los oídos como receptores para la percepción espacial con los que construir la escena auditiva que lo rodea.

2.1.1. Audición espacial

En el contexto físico, el ser humano convive en un espacio tridimensional donde todos los objetos emiten un movimiento que modifica la vibración de las partículas del aire. A este fenómeno físico se le conoce como "sonido". La capacidad de percepción de un sonido en el entorno hace posible construir un espacio sonoro. Sin embargo, para poder entender cómo percibimos este mismo es preciso aclarar dos conceptos definidos por Blauert (1997): el suceso sonoro (*sound event*) y el suceso auditivo (*auditory event*).

Un suceso sonoro se define dada la manifestación física de cualquier ruido en un espacio y tiempo específicos, como por ejemplo, la alarma de un coche situada al final de un garaje. Un suceso auditivo se define como la percepción consciente y subjetiva en la mente de un oyente sobre un sonido que proviene del medio, poniendo como ejemplo otra vez, el sonido lejano de esta misma alarma. La correlación de estos conceptos es lo que se conoce como audición espacial, el suceso auditivo donde se reconocen las propiedades de dirección, tiempo y espacio de un suceso sonoro.

De este modo se aclara la capacidad innata del ser humano de percibir fuentes sonoras a través de la audición espacial. La integración de estos sucesos sonoros define un espacio 3D que hace al oyente tener una experiencia inmersiva constante a través de la asignación de unas propiedades posicionales a cada una de las fuentes que recibe.

2.1.2. Audición binaural

Si la audición espacial es el fenómeno cognoscible de un suceso sonoro, la audición binaural es el proceso biológico con el que se construye el espacio sonoro con todas sus propiedades. La audición binaural define cómo escuchamos y entendemos el medio. A continuación se presentan los procedimientos internos de la audición binaural extrapolados al ámbito de la ingeniería.

2.1.2.1. Arquitectura del espacio sonoro

En primer lugar, que la audición se defina como binaural no es aleatoria. Su arquitectura es cimentada a través de los dos receptores auditivos (de contar con uno, se hablaría de monoaural) y que están representados por los ejes cartesianos de la figura 2.1 representada en la obra de Blauert (1997).

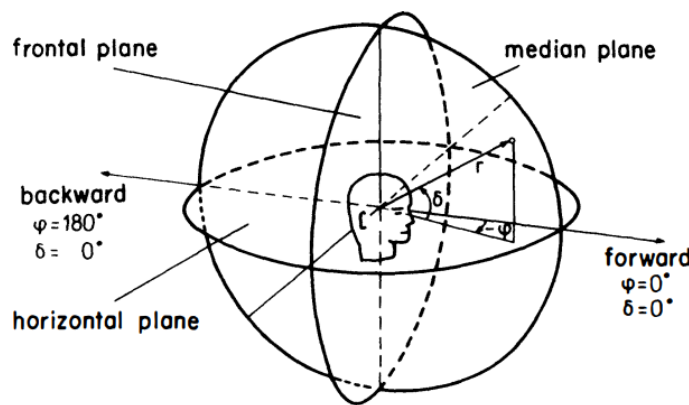


Figura 2.1: Sistema de coordenadas relativo a la cabeza.

Nota. Adaptado de *Spatial hearing: The psychophysics of human sound localization*, por Blauert (1997).

Este sistema de coordenadas tiene tres ejes que en analogía con el cartesiano OX , OY y OZ , se denominan *profundidad*, *horizontal* y *mediano*¹ respectivamente. El origen $(0,0,0)$ de este sistema de coordenadas es el centroide, situado en el punto equidistante a ambos oídos sobre la línea imaginaria que pasa por el eje horizontal de la cabeza que los atraviesa, más conocido como punto medio del eje interaural. Este espacio tridimensional es lo que se conoce geométricamente como espacio sonoro. Su espacio de representación es de naturaleza esférica y por eso es obligatorio entender que, aún ya mencionados los planos cartesianos, los cálculos matemáticos que se implementan físicamente son descritos por el trío de valores (r, θ, δ) . Para cada uno de ellos, r es la distancia desde el centroide hasta la fuente, θ (azimut) el ángulo de desviación lateral respecto al plano horizontal tomando como referencia el eje frontal del oyente (la nariz); y δ (elevación) el ángulo de desviación vertical respecto al plano horizontal. Estas tres variables son los parámetros de entrada necesarios para modelar las fuentes y posicionarlas dentro del espacio sonoro.

¹Representados en la figura 2.1 como *forward-backward*, *horizontal* y *median*.

2.1.2.2. Teoría Dúplex y lateralidad

La base teórica del mecanismo para la audición binaural yace en la Teoría Dúplex postulada por Rayleigh (1907) quien propuso cómo las diferencias de fase e intensidades entre ambos oídos permiten saber la posición de una fuente lateralmente. Estas fueron descritas matemáticamente más tarde con los conceptos de *Interaural Time Difference* (ITD)² e *Interaural Level Difference* (ILD)³. Son conceptos inseparables que se complementan entre sí. Para ambos se tiene en cuenta el rango frecuencial del espectro humano $[20Hz - 20kHz]$.

ITD define el desfase temporal entre ambos oídos medido en μs que experimenta el cerebro para cada periodo en una onda. Esto es causado por la separación física entre ellos por el eje interaural, lo que provoca que la onda tenga que recorrer una mayor distancia para el oído más alejado de la fuente. Por el contrario, ILD cuantifica la diferencia en dB entre las intensidades que le llegan a cada oído. Las representaciones de cada una de ellas, dadas por 2.2 y 2.3, son las siguientes:

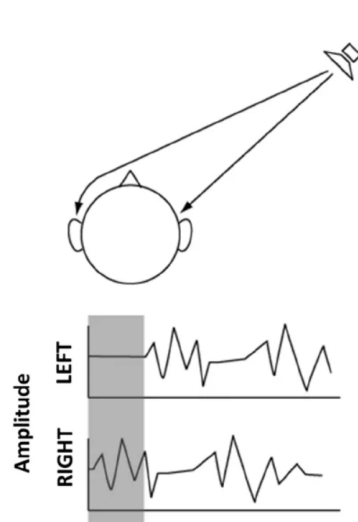


Figura 2.2: *Cognición de una onda por ITD.*

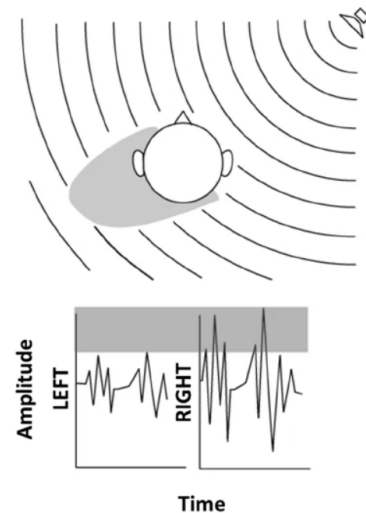


Figura 2.3: *Cognición de una onda por ILD.*

Nota. Ambas figuras fueron adaptadas de *Spatial audio through head-related transfer functions*, por Agrawal y Reiss (2015).

Para estos mecanismos de localización lateral existen limitantes frecuenciales y angulares. Por ahora, solo se tendrán en cuenta valores de elevación $\delta = 0^\circ$, representando únicamente la ubicación azimutal. ITD solo es útil para ondas graves menores a $1500Hz$, su frecuencia límite. Para valores mayores, su capacidad de medición es despreciable. Este fenómeno se conoce como ambigüedad de fase. Si el valor de ITD es mayor que el tiempo que tarda la onda de frecuencia f_{lmite} en completar un periodo, el cerebro es incapaz de distinguir si dicho periodo pertenece a un valor de ITD individual o al comienzo de un nuevo periodo. Este límite teórico viene dado por la relación entre el tiempo máximo recorrido ITD_{max} de un oído a otro y la velocidad de propagación de la propia onda $c = \lambda \cdot f_{lmite}$. Los

²En castellano, *Diferencia de tiempo interaural*.

³En castellano, *Diferencia de intensidad interaural*.

valores angulares para ITD_{max} se sitúan en el punto de lateralidad estricta, alineados con el eje interaural en $\delta = 90^\circ$ y 270° . Se tiene en cuenta el radio craneal de una persona promedio $a = 0,0875m$ y la velocidad del sonido $c = 343m/s$. Estos valores determinan la distancia extra máxima que deberá recorrer la onda cuando llega a la cabeza para llegar a cualquiera que sea el oído más alejado. El resultado de esta medición se obtiene a través del modelo esférico sólido propuesto por Woodworth el cual ya es mencionado en Blauert (1997). La justificación se demuestra con los siguientes cálculos:

$$ITD = \frac{a}{c}(\theta + \sin \theta) \rightarrow ITD_{max} = \frac{0,0875}{343}(\frac{\pi}{2} + \sin \frac{\pi}{2}) \approx 665\mu s \quad (2.1)$$

El valor obtenido corresponde al retraso de tiempo máximo teórico para un radio craneal promedio. Para este valor se desea la distancia extra máxima d_{extra} que tendría que recorrer la onda alrededor de la cabeza sin sobrepasar el límite:

$$d_{extraMax} = a(\theta + \sin \theta) \quad (2.2)$$

$$d_{extraMax} = 0,0875(\frac{\pi}{2} + 1) \approx 0,225m \quad (2.3)$$

$$f_{limiteITD} = \frac{c}{\lambda} = \frac{c}{d_{extraMax}} = \frac{343}{0,225} \approx 1524Hz \quad (2.4)$$

Entonces, por una limitación anatómica, para cualquier frecuencia aproximada a $1500Hz$ el cerebro rechazará la medición de ITD como válida pues todos los periodos serán más grandes que $665\mu s$. La demostración fue explicada a través del gráfico presentado en la obra de Moore (2013) incluida aquí en la figura 2.4:

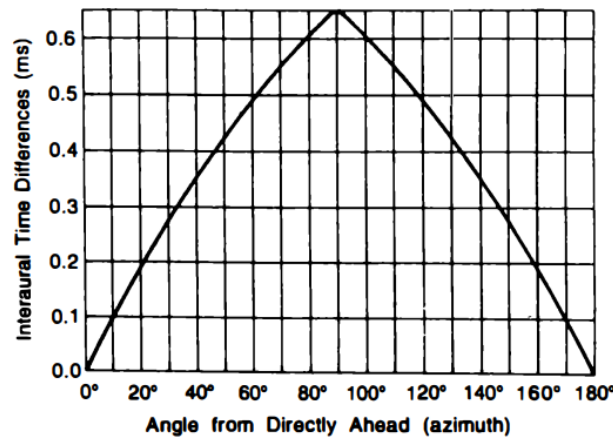


Figura 2.4: Valor de ITD con respecto al ángulo azimuthal de una fuente.

Nota. Adaptado de *An introduction to the psychology of hearing*, por Moore (2013).

La figura 2.4 también arroja información sobre los puntos azimutales donde ITD es inexistente ($ms = 0$). Estos son $\theta = 0^\circ$ y 180° . Para estos valores, independientemente de la distancia r , la fuente sonora es perpendicular al plano horizontal seccionando el centroide. Resulta imposible para el cerebro distinguir si el sonido está situado detrás o delante de él por ITD nulo.

De igual importancia, ILD es útil para las ondas agudas superiores aproximadamente a 1500Hz , a causa de la difracción acústica. Este es definido como el fenómeno donde una onda se curva y propaga alrededor de un obstáculo, siendo este la cabeza. La densidad craneal actúa como una barrera natural que limita la intensidad en dB que llega a cada oído. Partiendo del modelo esférico mencionado anteriormente se justificarán dos umbrales de difracción frecuencial críticos. Uno para cuando ILD empieza a debilitarse como mecanismo $f_{\text{limiteILD}}$ y por tanto la longitud de la onda λ difracta alrededor de la cabeza y otro para cuando ILD es inservible $f_{\text{inservible}}$. Se mantienen los mismos datos utilizados previamente para la justificación de la limitación de ITD añadiendo D como el diámetro craneal y $C = 2\pi a$, como la circunferencia esférica de la cabeza. El primero se obtiene con:

$$D = 2a = 2 \cdot 0,0875 = 0,175 \text{ m}$$

$$f_{\text{limiteILD}} = \frac{c}{\lambda} = \frac{c}{D} = \frac{343}{0,175} = 1960 \text{ Hz} \quad (2.5)$$

El segundo umbral se obtiene de la misma forma, solo que teniendo en cuenta toda la circunferencia esférica C :

$$f_{\text{inservible}} = \frac{c}{C} = \frac{c}{2\pi a} = \frac{343}{2 \cdot \pi \cdot 0,0875} \approx 624 \text{ Hz} \quad (2.6)$$

Previamente se puntualiza la limitación psicoacústica para distinguir frecuencias donde la diferencia de intensidad entre cada oído es inferior a 1dB . Traspasado el primer umbral, estas comienzan a difractar alrededor de la cabeza y, en consecuencia, los niveles de intensidad se igualan hasta 0dB , donde ITD ya es inservible. En la obra de Moore (2013) se refleja el incremento de ILD, representado aquí en la figura 2.5, para el espectro frecuencial deseado calculado anteriormente. También se aprecia cómo los valores a partir de 500Hz empiezan a ser despreciables.

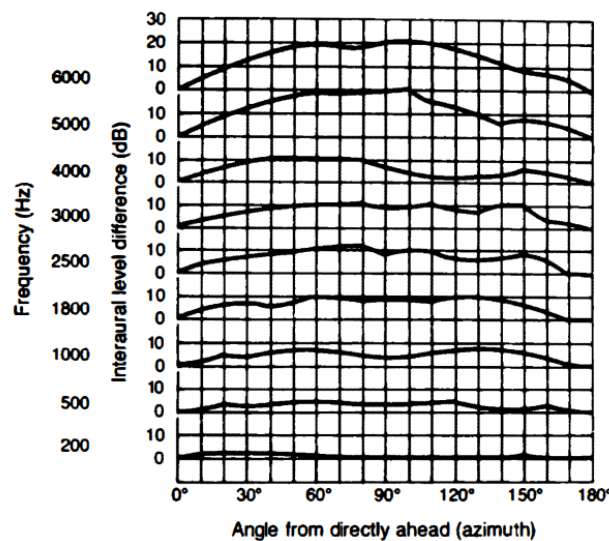


Figura 2.5: Valor de ILD con respecto a un rango azimutal y frecuencial.

Nota. Adaptado de *An introduction to the psychology of hearing*, por Moore (2013).

En la figura superior también se recogen los puntos azimutales $\theta = 0^\circ$ y 180° donde ILD es inexistente ($dB = 0$). De este modo, se enmarca la utilidad de la teoría Dúplex con respecto a la audición binaural en cuanto a su capacidad para definir al grado de lateralidad (izquierda o derecha) de una fuente de sonido. Sin embargo, los mecanismos descritos en dicha teoría no abarcan los casos en donde la fuente esté situada justo enfrente y detrás, ($\theta = 0^\circ$ y 180°), ni arriba y abajo ($\delta = 90^\circ$ y 270°). Para comprender sus limitaciones, se necesita conocer el cono de confusión.

2.1.2.3. El cono de confusión

Como la teoría Dúplex solamente recoge mecanismos laterales computados con ITD e ILD, la audición binaural quedaría reducida al campo panorámico. Condiciona entonces a efectos posicionales que el cerebro no sepa de donde proviene la fuente para estas cuatro posibles direcciones a efectos generales:

- D) Enfrente o B) Detrás (Separados por el plano frontal).
- A) Arriba o C) Abajo (Separados por el plano horizontal).

Geométricamente el cono de confusión es la superficie de revolución donde el eje de simetría es el eje interaural. La base se comienza a formar cuando $\theta \neq 90^\circ$ y 270° , o $\delta \neq 0^\circ$, donde la fuente deja de coincidir con el eje interaural. El cono se genera mediante la operación de revolución donde θ actúa como el grado de apertura, pudiendo ser así más grande o pequeño. La figura 2.6 representa las cuatro posibles direcciones para un mismo grado de lateralidad:

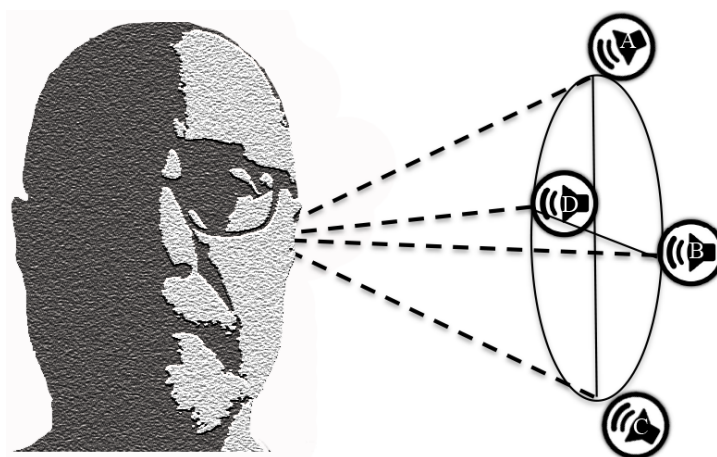


Figura 2.6: Ejemplificación de un cono de confusión cualquiera del oído izquierdo.

Nota. Adaptado de *Psychoacoustics in music*, por Gumm (2012).

Por consiguiente, la confusión se establece porque asumiendo un mismo valor r , todos los puntos situados sobre la circunferencia de la base del cono comparten los mismos valores de ITD e ILD. Técnicamente, el cono siempre existe solo que empieza a degenerarse cuando alcanza sus límites que son los ya mencionados $\theta = 0^\circ$ y 180° , formando el plano mediano; y $\theta = 90^\circ$ formando una recta alineándose con el eje interaural. Conocido el cono se introducirá el concepto de ángulo lateral α el cual la teoría Dúplex llegó a comprender pero no acotó. α representa el ángulo que tiene una fuente con respecto al eje interaural,

es decir, cuánto está situada a la izquierda y derecha sin tener en cuenta la profundidad. Se seleccionan entonces dos puntos que se encuentran en la misma base de su cono de confusión para entender cómo se proyectan con respecto al eje interaural y producen el mismo α , causando que Dúplex la sitúe en el mismo sitio. La relación trigonométrica entre azimut, elevación y ángulo lateral es descrita en Xie (2013), es:

$$\sin(\alpha) = \cos(\delta) \cdot \sin(\theta) \quad (2.7)$$

Con ello se demuestra matemáticamente cómo dos fuentes sonoras ubicadas en posiciones tridimensionales completamente distintas proyectan el mismo ángulo lateral sobre el eje interaural:

Caso A: Posición: $\theta = 30^\circ$ (frente-derecha), $\delta = 0^\circ$ (sin elevación).

Caso B Posición: $\theta = 90^\circ$ (derecha), $\delta = 60^\circ$ (elevado).

$$\sin(\alpha_A) = \cos(0^\circ) \cdot \sin(30^\circ)$$

$$\sin(\alpha_A) = 1 \cdot 0,5$$

$$\sin(\alpha_A) = 0,5$$

$$\alpha_A = \arcsin(0,5) = 30^\circ$$

$$\sin(\alpha_B) = \cos(60^\circ) \cdot \sin(90^\circ)$$

$$\sin(\alpha_B) = 0,5 \cdot 1$$

$$\sin(\alpha_B) = 0,5$$

$$\alpha_B = \arcsin(0,5) = 30^\circ$$

Como queda demostrado, tanto el *Caso A* como el *Caso B* resultan en $\alpha = 30^\circ$. Puesto que la teoría Dúplex calcula la ITD y la ILD basándose exclusivamente en este ángulo de proyección interaural, el sistema auditivo recibirá exactamente la misma información lateral temporal y de nivel para ambas posiciones, confirmando así la existencia geométrica del cono de confusión.

Por último, se comprenden las posibles direcciones de manera generalista debido a que dentro de toda la circunferencia existen infinitos puntos A, B, C y D. Los casos expuestos en el subapartado 2.1.2.2 pertenecen al caso más extremo. Y, para cada oído, existe un único cono de confusión representado en la figura 2.7. Por consiguiente, se ha de conocer que en la resolución sobre la ubicación de una fuente en el eje azimutal y su elevación, el propio oído realiza un filtrado de frecuencias específico.

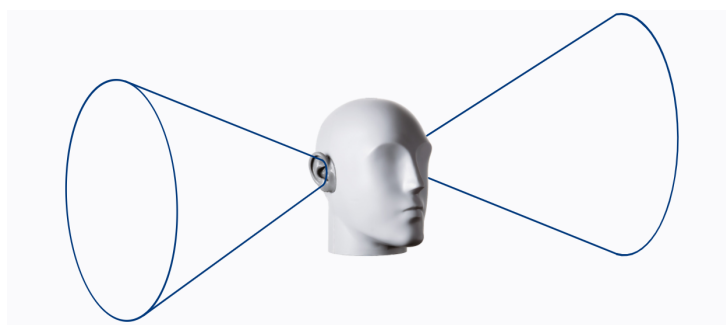


Figura 2.7: *Cono de confusión a ambos lados de la cabeza.*

Nota. Adaptado de *Psicoacústica: Conos de confusión*, por eMe - Escuela Universitaria de Música (2020).

2.1.2.4. Pistas espectrales

Una pista espectral o *spectral cue* es la modificación del espectro frecuencial proveniente de una fuente a causa de la anatomía natural del cuerpo, los hombros y el pabellón auricular. Siendo una modificación pasiva al provenir de forma natural. Una pista espectral permite al cerebro, por diversos métodos, reconocer las componentes θ y δ y así resolver el punto dentro del cono de confusión. La contribución de estas pistas espectrales para la localización de una fuente Langendijk y Bronkhorst (2002) tiene lugar en dos procesos: generación e interpretación. En primer lugar, los mecanismos de generación son:

- **Filtrado de *Pinna*:** a raíz de la forma del pabellón auricular se establecen diferentes filtrados. El primero, los pliegues externos e internos de la cavidad hacen que existan difracciones y reflexiones en la onda alterando su valor de ITD e ILD, aportando así un rasgo diferenciador para obtener δ y θ . El segundo, dependiendo de las fases de estas reflexiones se generarán alineaciones o cancelaciones de fase generando picos o valles dentro del espectro. A estas muescas espectrales se les llama *spectral notches*. El tercero, cuando ciertas frecuencias resuenan en el conducto auditivo ciertas bandas frecuenciales también se ven acentuadas generando picos.
- **Dependencia de la elevación:** según el incremento o decremento de δ y θ , la geometría de los rebotes en el pabellón cambia. Esto demuestra que la dependencia entre el grado de elevación y las muescas es dinámica. Las muescas se desplazan en distintas regiones frecuenciales que el cerebro utiliza para decodificar su elevación. Siendo $[6kHz - 12kHz]$ para encontrar su ubicación arriba o abajo δ y $[8kHz - 16kHz]$ para discriminar entre enfrente o detrás θ .

En la figura 2.8 se puede ver la pista espectral para una fuente sonora (en este caso un bajo) siguiendo una dirección específica. La figura también ejemplifica como influye la composición de la pista en base a los distintos tonos producidos por el instrumento.

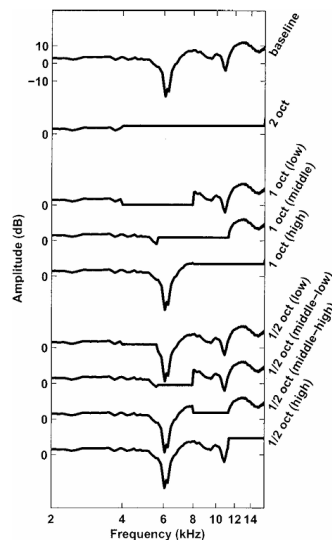


Figura 2.8: Ejemplo de las pistas espectrales resultantes para un bajo posicionado en $\theta = 0^\circ$, $\delta = -56^\circ$ y distintos tonos.

Nota. Adaptado de *Contribution of spectral cues to human sound localization*, por Langendijk y Bronkhorst (2002).

El segundo proceso recae en la interpretación de dichas pistas espectrales mediante un proceso de aprendizaje y comparación. Por la propia neuroplasticidad del cerebro, los seres humanos poseen una serie de plantillas de pistas espectrales particulares con las que interpretan de dónde proviene una fuente. Lo que se realiza entonces es un proceso de comparación entre el espectro de la señal que llega al tímpano y estas plantillas, eligiendo la de mayor similitud. A este procedimiento se le llama *template matching*⁴. En la figura 2.9 se puede ver cómo el resultado del proceso de aprendizaje por parte de las pistas espectrales relaciona las muescas espectrales con la posición de una fuente, en este caso para valores del plano medio.

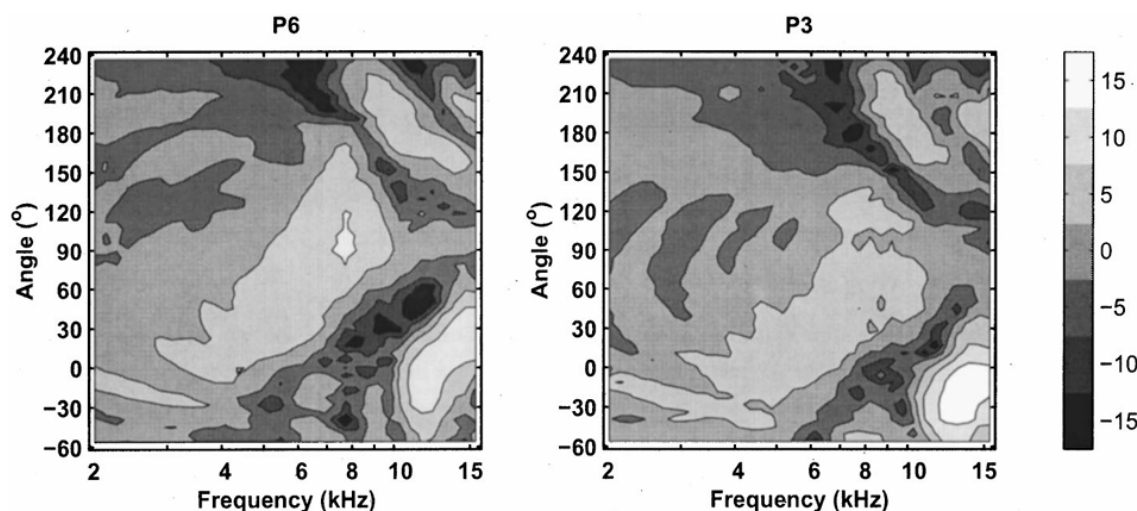


Figura 2.9: Función DTF del oído izquierdo y sus muescas espectrales según el ángulo de elevación para dos oyentes (P6 y P3). La amplitud en dB percibida para cada frecuencia es representada por la barra vertical lateral derecha.

Nota. Adaptado de *Contribution of spectral cues to human sound localization*, por Langendijk y Bronkhorst (2002).

Ambos gráficos resaltan varios de los puntos explicados anteriormente. Y son, que todas las regiones para un mismo valor de intensidad están encerradas diagonalmente, con lo que el cambio de los ángulos δ y θ son dependientes al aumento frecuencial, y por tanto dinámico; que a pesar de las diferencias entre los mapas de cada oyente los valores y regiones son similares y que las regiones frecuenciales acotadas por las muescas con respecto a la dependencia de la elevación coinciden con el mapa. Se ha de mencionar el uso de DTF para esta medición, pese a ser importante, por ahora se mencionará que únicamente recoge exclusivamente las variaciones frecuenciales por el cambio de posición de la fuente.

Ahora que ya se tienen las herramientas para comprender la posición angular de una fuente sonora θ y δ , necesitamos entender cómo la audición binaural interpreta la distancia r de una fuente.

⁴En castellano, *Coincidencia de plantillas*. Es una técnica clásica de interpretación visual artificial utilizada para encontrar áreas dentro de una imagen que coincidan con una más pequeña.

2.1.2.5. Intensidad relativa

La relación de intensidad relativa o nivel de intensidad sonora de acuerdo con Serway et al. (2000), se define como la relación logarítmica entre la intensidad I del sonido proveniente de una fuente y una de referencia, que para el ser humano es el umbral de audición mínimo $I_0 = 10^{-12} \text{W/m}^2$. Matemáticamente, la relación de intensidad relativa expresada y medida en decibelios dB es la siguiente:

$$dB = (10 \text{ dB}) \log \frac{I}{I_0} \quad (2.8)$$

El único dato de esta expresión que no se conoce es la intensidad I de la fuente. Entendiendo que la propagación de una onda tridimensional se distribuye uniformemente en condiciones ideales (ausencia de absorción atmosférica) y con una potencia W , también constante, generada por la fuente P_{av} , la energía de la fuente se distribuye sobre una superficie esférica y uniforme dado un valor r . Entonces, para cualquier punto situado a esta distancia de una fuente su intensidad es determinada por:

$$I = \frac{P_{av}}{A} = \frac{P_{av}}{4\pi r^2} \quad (2.9)$$

Por lo tanto, al conocer la relación matemática entre la distancia r y el nivel de intensidad sonora medido en dB, se debe especificar como sus propiedades afectan a la audición binaural. Primero, la ecuación 2.9 evidencia que la intensidad percibida es inversamente proporcional al cuadrado de la distancia ($1/r^2$). A este fenómeno se le conoce como la ley de la inversa del cuadrado y es válido para cualquier tipo de onda tridimensional (ya sea de naturaleza acústica, electromagnética o elástica). La figura 2.10 define cómo la intensidad de una onda (representada aquí con E pues la figura se utilizó para una onda electromagnética) decrece a medida que aumenta r por la expansión del frente de onda esférico:

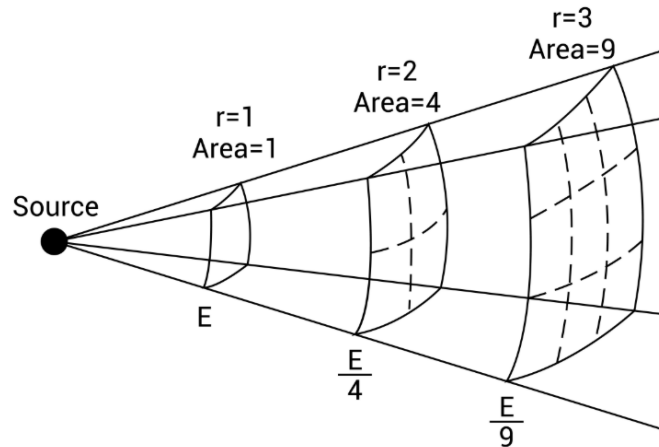


Figura 2.10: Representación de la Ley Inversa del cuadrado.

Nota. Adaptado de *Electric fields*, por Monash University (2024).

Segundo, la ecuación 2.8 refleja una escala no lineal sino logarítmica. Estos cambios sugieren que los cambios de intensidad sonora son bastante más perceptibles en distancias pequeñas que en distancias mayores. La referencia se establece en una reducción de -6dB

cada vez que separamos la fuente el doble de distancia con respecto a su posición original, es decir, pasamos de r a $2r$. Este concepto es fácilmente demostrable al unir la ecuación de relación de intensidad relativa y la ley inversa del cuadrado. Si queremos comparar empíricamente cuánto varían en intensidad dos fuentes idénticas ΔdB donde únicamente cambia su distancia en las condiciones expuestas previamente, se puede ejemplificar una fuente I_1 con una r cualquiera y la misma fuente I'_1 pero separada el doble $2r$:

$$\Delta dB = 10 \log_{10} \left(\frac{I'_1}{I_1} \right) \quad (2.10)$$

Donde la relación entre cada una de sus intensidades viene dada con la ley inversa del cuadrado y que I'_1 se ve reducida en una cuarta parte con respecto a I_1 . Dando finalmente la variación resultante a través de la siguiente expresión:

$$I_1 = \frac{P_{av}}{4\pi r^2} \quad \text{y} \quad I'_1 = \frac{P_{av}}{4\pi (2r)^2} = \frac{P_{av}}{16\pi r^2} \quad (2.11)$$

$$\frac{I'_1}{I_1} = \frac{\frac{P_{av}}{16\pi r^2}}{\frac{P_{av}}{4\pi r^2}} = \frac{4\pi r^2}{16\pi r^2} = \frac{4}{16} = \frac{1}{4} \quad (2.12)$$

$$\Delta dB = 10 \log_{10} \left(\frac{I'_1}{I_1} \right) = 10 \log_{10} \left(\frac{1}{4} \right) = -6,02 \text{dB} \quad (2.13)$$

Este resultado confirma que la percepción del sistema auditivo para una misma fuente aporta el mismo cambio de intensidad, independientemente de que estén alejadas entre $[1m - 2m]$ y/o $[100m - 200m]$. En resumen, variaciones de intensidad en distancias cortas (ej. $1m \rightarrow 1.5m$) serán mucho más intensas que en distancias largas (ej. $100m \rightarrow 125m$) afectando notablemente a la percepción de la intensidad r y con ello a los valores de ILD y la precisión con la que una persona puede percibir la distancia de los sucesos sonoros dentro del espacio.

Por último, el oído humano cuenta con otro fenómeno psicoacústico vinculado con la intensidad relativa que no afecta directamente a la construcción de las pistas binaurales para posicionar una fuente, pero sí a la percepción auditiva y es merecedor de mención. Este concepto es la sonoridad. A través de las curvas isofónicas, podemos contemplar la cantidad en decibelios necesarios para que la intensidad de diferentes frecuencias se perciba subjetivamente igual. Esta sonoridad se mide en fonios y cambia a medida que aumenta la intensidad relativa en dB . A pesar de ser una característica interesante en el proyecto, las curvas isofónicas se mantendrán planas para el oyente, priorizando así la fidelidad en la identificación espacial de las pistas binaurales generadas frente a un enfoque más psicoacústico.

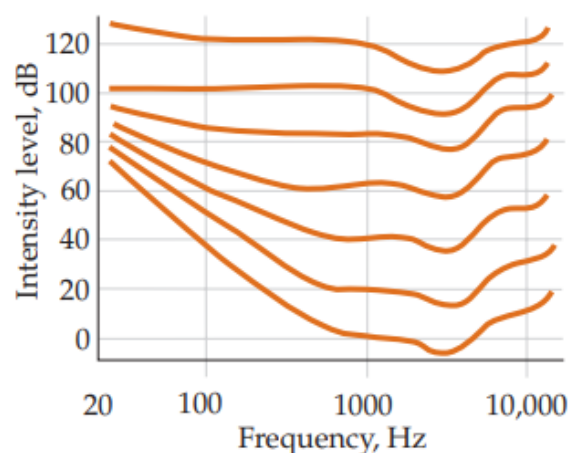


Figura 2.11: *Curvas isofónicas.*

Nota. Adaptado de *Physics for scientists and engineers: With modern physics*, por Tipler y Mosca (2007).

Por último, la audición binaural no queda resuelta sin el proceso de externalización, un paso clave con el que se comprende como todos estos parámetros pasan de ser interpretados internamente (dentro del cráneo) hacia afuera (posicionamiento espacial).

2.1.2.6. Externalización

A grandes rasgos la externalización es la capacidad de proyectar la imagen sonora que se recibe en un punto tridimensional específico fuera de la cabeza del oyente. Este punto lo conformaría el conjunto de valores de la arquitectura (r, θ, δ) presentada en el apartado 2.1.2.1. No obstante, antes de entender este concepto se necesita diferenciar el medio natural en donde la audición binaural se desarrolla por sí misma, del medio virtual en donde se va a construir el sistema de reproducción. Ambos son descritos por Begault (2000) a través del modelo Fuente-Medio-Receptor.

En el medio natural, todos los sucesos sonoros (fuentes), ocurren en un momento y espacio determinados (medio) donde la onda viaja y se transforma de forma natural a través de los fenómenos físicos, anatómicos y psicoacústicos referidos anteriormente, llegando hasta nosotros (receptor). Por el contrario, un sistema de reproducción virtual parte de la base donde la única información que se tiene del suceso sonoro es el conjunto de datos eléctricos o digitales (fuente) que representan la onda de sonido hasta llegar al oído (receptor). En un modelo físico, y por ello externo, el medio es inherente a la fuente; para un sistema de reproducción virtual el medio ha de ser simulado.

La falta de un medio físico, y por tanto, la ausencia de información espectral, donde una onda pueda propagarse dentro de un entorno virtual, se produce la internalización, lo contrario a la externalización. Esta ausencia confina la imagen sonora dentro del eje interaural, aportando únicamente información del campo estéreo (sea este el factor de lateralidad α descrito anteriormente). Este fenómeno hace que el sonido nazca dentro de la cabeza sin información espacial y quede dentro haciéndose inidentificable espacialmente. Por ello la externalización también se puede comprender como el mecanismo de reversión de la internalización.

Para revertirla, el sistema de reproducción ha de procesar las propiedades de la audición

binaural partiendo de la fuente de su señal analógica o digital. Resumidamente, las analógicas son las capturadas en tiempo real y las digitales son almacenadas como datos en soportes externos o internos. Pero independientemente de si la fuente es de un tipo u otro, la clave reside en el método de entrega al receptor. Mientras que los sistemas de altavoces y equipos PA⁵ proyectan la onda en un entorno físico, introduciendo componentes posicionales (r, θ, δ) , los sistemas de reproducción personal controlan estas variables. A continuación se exponen las principales razones, descritas por Begault (2000), sobre por qué es imperativo utilizar sistemas personales (auriculares) en vez de altavoces para la construcción de un sistema de reproducción virtual que invierta la externalización:

- **Eliminación de la diafonía (Cross-talk):** la diafonía consiste en la perturbación de un canal en otro distinto. Para que exista se necesitan dos altavoces y dos receptores que corresponden a cada oído, L y R. Al estar expuestos para los oídos, los altavoces dan información que después será captada inevitablemente por el oído opuesto con un ligero retardo y filtrado físico. Este cruce de señales anula las diferencias de ITD e ILD, provocando que el cerebro reciba información contradictoria y colapse la imagen tridimensional diseñada antes de su transmisión. Viéndose alterada la acústica espacial la imagen se internaliza, resultando en una imagen estéreo. Por el contrario los auriculares, al aislar acústicamente cada canal, garantizan que el receptor perciba únicamente la señal procesada para cada oído, obteniendo así datos posicionales. Simplificando el problema, véase el caso en el que únicamente se tiene un altavoz, ambos oídos percibirían la misma información. Es la independencia entre las ondas que percibe cada oído la que externaliza el sonido, cualquier alteración de la misma rompe la capacidad para la audición binaural.
- **Independencia del punto ideal:** tener que emular los mecanismos de audición binaural implica tener en cuenta el punto donde está situado el oyente. Por ello la mezcla resultante para externalizar espacialmente el sonido se diseña en condiciones donde el sujeto es inmóvil, cualquier movimiento por parte del oyente truncaría la mezcla generando desfases entre oídos perdiendo la posibilidad de ubicar una fuente. Con respecto al oyente los altavoces están quietos, los auriculares se mueven con el oyente. Liberando así la posición desde donde se debe hacer la escucha.
- **Compatibilidad con el movimiento de la cabeza:** al igual que la independencia del punto ideal, los movimientos de cabeza también influyen en la construcción del espacio sonoro. Los auriculares ofrecen una transducción donde ningún factor externo influye hasta que el sonido es recibido por cada receptor.
- **Componentes reverberantes:** la naturaleza de los altavoces hace que la onda se propague en todas las direcciones. Al no independizar la fuente a cada uno de los receptores auditivos se producen reflexiones tempranas y reverberaciones alterando drásticamente la percepción del sonido y en consecuencia la posición de la fuente.

Concluyendo todo el apartado sobre la audición binaural. A base de distintos mecanismos, factores físicos, psicoacústicos y anatómicos, la audición binaural permite obtener las componentes posicionales de una fuente. Estos datos tienen que externalizarse, siendo este el proceso de proyectar la imagen sonora en el punto conocido a través de todo este mecanismo, sacándolo de la cabeza. Por ello durante el desarrollo del último punto se

⁵Equipos de sonorización diseñados para llevar el sonido al oyente, incluyendo altavoces, preamplificadores y mesas de mezcla.

ha discutido sobre el método de transducción adecuado para el sistema de reproducción virtual de interés, los auriculares. A efectos para la construcción de un espacio virtual se necesitan herramientas matemáticas que sean capaces de modelar la audición binaural virtualmente, siendo la principal la HRTF. No sin antes entender cómo se representa una señal de onda en el ámbito digital.

2.1.3. Representación de una señal de onda digital

Dentro del ámbito digital Verfaille et al. (2011) concuerdan que la representación de una onda analógica $x(t)$ que emite una fuente cualquiera no es más que la concatenación de valores de amplitud tomados en instantes de tiempo equidistantes. Esta conversión es realizada a través de un *Analog-to-digital converter* (ADC) que utiliza técnicas de recogida de muestreo o *sampling* que después se cuantizan asignándoles valores numéricos. Puede comprenderse como un vector de puntos cualquiera $x(n)$ donde n sitúa el índice del tiempo analógico t discreto o en instantes específicos. El intervalo de recogida entre n y $n + 1$ es denominado como intervalo de muestreo. Dependiendo de las necesidades gráficas estos valores n se podrán representar de diversos motivos en los que no se entrará en detalle pues el formato de audio que se quiere utilizar para la construcción del sistema .WAV no comprime los datos de la onda $x(t)$. No existe pérdida de información. En la figura 2.12 se puede ver la representación de una señal de onda digital:

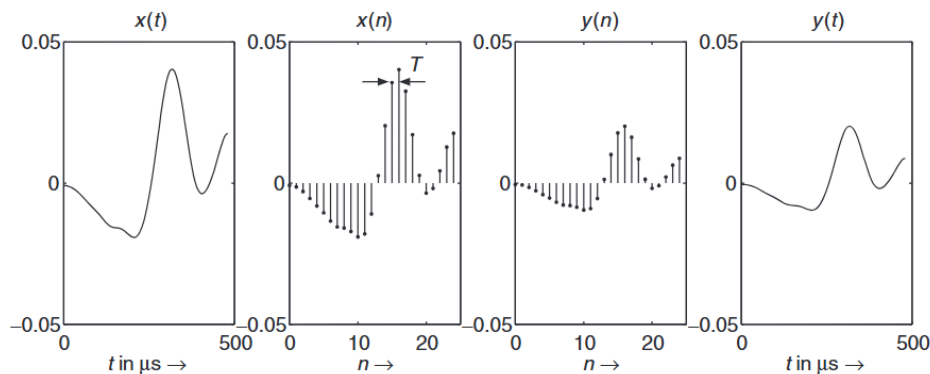


Figura 2.12: Representación de una señal de onda digital genérica tras aplicar un ADC y escalado específicos.

Nota. Adaptado de *Adaptive digital audio effects*, por Verfaille et al. (2011).

Este apartado cimienta la base para entender la representación de una onda digital y así aplicar el modelo matemático que aporte las características de posicionamiento en un sistema de reproducción virtual. La herramienta principal utilizada en *Digital Signal Processing* (DSP) para dotar de dichas propiedades a una onda es la HRTF.

2.1.4. HRTF

La definición aportada por el material bibliográfico Blauert (1997) y Begault (2000) concuerdan en que una HRTF se define como la función que representa la relación entre la presión sonora para una frecuencia medida en el tímpano cuando la persona está presente y cuando está ausente. En pocas palabras una HRTF recoge la influencia frecuencial causada por la anatomía del oyente (cabeza, torso, pabellón y pabellón auricular), su

impronta frecuencial. Como el sistema auditivo es diferencial entre el oído izquierdo y derecho (L y R), se ha de obtener una HRTF diferente para cada uno pues con ellas el cerebro triangula estas diferencias frecuenciales al localizar una fuente. Ambas funciones es lo que Blauert (1997) denomina como HRTF binaural. El desglose matemático para una HRTF es la siguiente:

$$HRTF(f, \theta, \delta, r) = \frac{P_{timpano}(f, \theta, \delta, r)}{P_{libre}(f, r)} \quad (2.14)$$

$$P_{timpano} \rightarrow f(f, \theta, \delta, r) \quad (2.15)$$

$$P_{libre} \rightarrow f(f, r) \quad (2.16)$$

Independientemente de la HRTF a computar se deja claro que la aparición de la anatomía diferencia automáticamente la presión percibida pues la onda viaja en condiciones que rompen la simetría del espacio, interactuando con obstáculos físicos $P_{timpano}$, clasificándose como anisótropa. Al contrario, una onda que viaja libremente sin restricciones anatómicas desconoce una posición en su cómputo de presión P_{libre} , dentro de un modelo esférico sus valores de presión son siempre los mismos. Únicamente dependerá del tiempo que viaje la onda y en consecuencia de la distancia entre la fuente y el punto de referencia, clasificándose como isotropa. Queda entonces descrito que digitalmente HRTF normaliza la posición de una fuente y que, también, para cada oído su función HRTF será indicada como $HRTF_L$ y $HRTF_R$. Gráficamente tanto los puntos donde se realiza la medición de lo que se ha llamado como $P_{timpano}$ y P_{libre} , como el espectro frecuencial HRTF para cada oído en un posición de ejemplo corresponden a las figuras 2.13, 2.14 y 2.15 respectivamente:

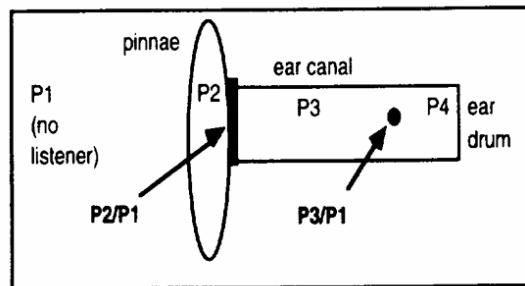


Figura 2.13: *Puntos de medición para HRTF.*

Nota. En correlación a la figura, P_4 es el punto utilizado para obtener $P_{timpano}$ y P_1 para P_{libre} . Adaptado de *3-D sound for virtual reality and multimedia*, por Begault (2000).

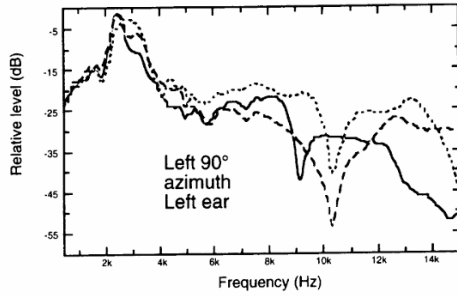


Figura 2.14: Medición $HRTF_L$ para $\theta = 90^\circ$.

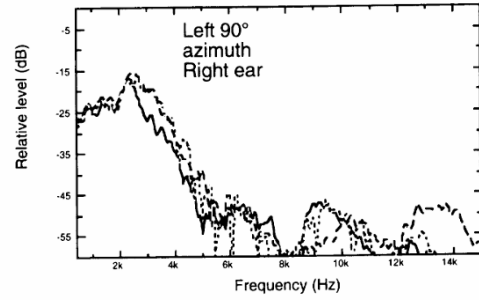


Figura 2.15: Medición $HRTF_R$ para $\theta = 90^\circ$.

Nota. Ambas figuras fueron adaptadas de *3-D sound for virtual reality and multimedia*, por Begault (2000).

Las distintas huellas frecuenciales dejadas por las HRTF en cada receptor auditivo son las que permiten al cerebro diferenciar y externalizar la fuente. Esta función de respuesta no solo se puede representar en el dominio frecuencial sino que este impulso también existe en el dominio del tiempo. Su equivalente es la *Head-Related Impulse Response* (HRIR). HRIR es la secuencia digital que representa las alteraciones de amplitud exactas que sufre el impulso sonoro de la onda al interactuar con la anatomía del oyente antes de llegar al tímpano. En un sistema de reproducción digital HRIR actúa como un filtro continuo finito o *Finite Impulse Response* (FIR) pues, tanto la pista digital a la que se va a aplicar como la HRIR proveniente de una HRTF no son infinitas en un dominio del tiempo $f(t)$. HRIR es la base para la construcción de cualquier sistema de reproducción digital que quiera otorgar a una fuente una posición (r, θ, δ) . Para obtener la HRIR proveniente de su HRTF es la siguiente se realiza el siguiente proceso:

$$HRIR(t, \theta, \delta, r) = \mathcal{F}^{-1}\{HRTF(f, \theta, \delta, r)\} \quad (2.17)$$

La operación realizada es la transformada inversa de Fourier o $\mathcal{F}^{-1}$, la cual convierte una función de frecuencia $F(z)$ en una función de tiempo $f(t)$. Ambos recursos matemáticos se implementan dentro de la cadena DSP del sistema de reproducción, introduciendo las huellas posicionales para las fuentes. Las HRTF no son obtenibles de forma natural pues son un atributo que ya posee el ser humano de forma innata, descrito anteriormente en la definición de las pistas espectrales 2.1.2.4. Para poder implementar dichas componentes digitalmente han de obtenerse desde el medio físico.

El proceso de captación descrito por Begault (2000) describe el método más común para ello. Primero, estas se simulan en entornos favorables, donde se tienen en cuenta efectos adversos a la externalización, como la anulación de componentes reverberantes. Siendo los espacios idóneos las cámaras anecoicas. El objetivo es obtener los valores de $P_{timpano}$ y P_{libre} de la manera más fidedigna posible que esta vez se traducen directamente en señales al captarse digitalmente. Sean pues los datos utilizados para la captación: un micrófono situado en la posición (r, θ, δ) deseada, un maniquí de prueba simulando las propiedades anatómicas del oyente promedio, dos micrófonos situados cada uno en las cavidades auriculares L y R, y el sistema de captación para tomar las señales recogidas. El proceso general a seguir es el siguiente:

1. **Emisión de la señal acústica:** se emite una onda analítica (una onda sin componentes de amplitud negativas) con la que poder tratarse mejor en el post-procesado desde la posición deseada. La onda emitida es un tono puro que cubre todo el espectro audible. Todo ello desde un altavoz.
2. **Captura y grabación:** la captura de la respuesta a la presión es diferente dependiendo de si se pretende obtener la señal del tímpano o la libre. La señal de los tímpanos recoge la captura del tono en ambos oídos con el maniquí expuesto, por otro lado, la captura libre sitúa un micrófono en el centro del eje interaural sin el maniquí, imitando así la captura sin la obstaculización del cuerpo. La grabación se acaba transmitiendo a los sistemas de captura.
3. **Ecualización compensatoria:** tras la grabación se realiza un post-procesado que dependerá de las características de la sala, el altavoz utilizado para la emisión y el sistema de microfonía. El objetivo es corregir las atenuaciones y/o aumentos introducidos por las irregularidades de estos equipos. Para ello se realiza una segunda medición, reproduciendo el mismo tono pero con unos auriculares puestos en el maniquí. Finalmente, la aplicación de un filtrado inverso, que no es más que un método de procesamiento para anular la distorsión introducida por los equipos obtiene la respuesta plana y limpia de una HRTF.
4. **Formateo de salida:** la forma frecuencial de las mediciones son muy parecidas a las *spectral cues* discutidas en el punto 2.1.2.4. El formato final dependerá de las características del objeto de estudio. La eliminación del ruido al comienzo de la grabación, la normalización en la amplitud de los datos y la traducción de estos a formatos más cercanos al procesador son algunas de las técnicas utilizadas.

Acorde a lo mencionado, el modelo HRTF no es universal en su captura e irremediablemente tampoco lo es en la obtención de los datos. Otros procedimientos en la captura de HRTF distinguimos las realizadas con múltiples personas por el CIPIC Algazi et al. (2001) o la IRCAM Warusfel (2002), ambas instituciones investigadoras. La más conocida es la obtenida por Gardner y Martin (1994) en el *Massachusetts Institute of Technology* (MIT). En ella se utilizó un modelo de maniquí KEMAR que representa la anatomía normalizada del ser humano y, entre los datos crudos se encuentra un set HRIR ecualizado para el campo difuso orientado a entregar una respuesta plana frecuencial que poder aplicar a cualquier tipo de auriculares. Este tipo de ecualización busca que la respuesta del timbre de todas las frecuencias sea lo más neutra posible, o mejor dicho, que no existan realces y coloraciones frecuenciales. Por estas características y el aspecto diferenciador en el uso de un maniquí KEMAR frente al resto de sets que utilizan un modelo individualizado en sus mediciones; la base de datos HRTF del MIT será la utilizada en la construcción de la cadena DSP del sistema. Es su introducción dentro de la cadena lo que acontece a desarrollar.

2.1.4.1. Convolución de HRTF en cadenas DSP

La convolución * desarrollada en la documentación Verfaille et al. (2011) es una operación matemática básica que permite filtrar y/o imponer una respuesta conforme a un impulso. Esto implica imponer unas condiciones de señal que son aplicables tanto en el dominio temporal como el frecuencial. Sea ese caso, la señal original a la que se le quiere aplicar un filtrado $x(n)$ y la respuesta al impulso HRIR $h(n)$, su proceso de convolución

generaría la salida binaural deseada $y(n) = x(n) * h(n)$, pues ahora la onda original contiene la información temporal al impulso espacial generado con HRIR dada por (r, θ, δ) . No obstante, la convolución tiene un coste algorítmico muy alto $O(n^2)$ al tener que sumar y multiplicar la respuesta al impulso $h(n)$ por todas las componentes de $x(n)$, para todos los valores de n . Entonces, para reducir este coste computacional se utilizará la *Fast Fourier Transform* (FFT), proveniente del método contrario *Inverse Fast Fourier Transform* (IFFT) y el teorema de la convolución. A continuación, la figura 2.16 representa como estas herramientas matemáticas interactúan con los dominios de la señal a convolucionar $x(n)$ y una respuesta al impulso $h(n)$:

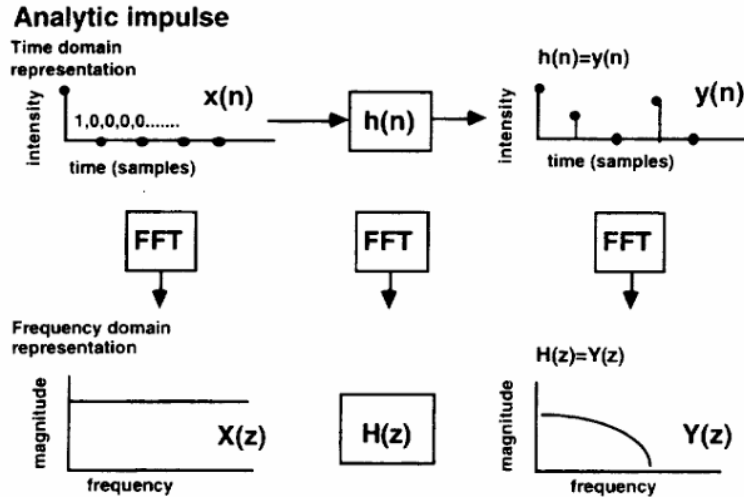


Figura 2.16: Utilización de FFT para convertir las distintas señales en el dominio frecuencial.

Nota. Adaptado de *3-D sound for virtual reality and multimedia*, por Begault (2000).

Pese a no estar incluido en la figura, IFFT corresponde lógicamente al proceso contrario $X(z) \rightarrow x(n)$. En conclusión, tanto FFT como IFFT aplican un cambio de dominio. Tras poder transformar las señales, no es sino con la aplicación del teorema de la convolución con el que se aplica la DSP descrita en la figura 2.16. El teorema de la convolución, mencionado de forma implícita por Begault (2000), establece que la convolución de dos señales en el dominio temporal $f(t)$ es equivalente a la multiplicación de sus FFT en el dominio $F(z)$. En consecuencia:

$$x(n) * h(n) = X(z) \cdot H(z) \quad (2.18)$$

Por último se deja constancia de dos cuestiones a resolver relacionadas con el mismo apartado. Que la convolución de una HRTF tendrá que aplicarse independientemente tanto para $HRTF_L$ como $HRTF_R$ y que se tendrá que elegir un método concreto dado que el impulso HRIR es más pequeño que la pista a convolucionar $x(n)$. Estas técnicas son *overlap-add* y *overlap-save*, ambas válidas para el procedimiento.

Finalmente el apartado teórico que será utilizado en la implementación del proyecto queda descrito. Se da paso a las tecnologías habilitadoras.

2.2. Tecnologías habilitadoras

A continuación, se detalla una lista de las tecnologías habilitadoras para el desarrollo del proyecto. Para cada una de ellas, se realizará una breve explicación sobre su función y aportación en la construcción del proyecto, junto con su justificación frente a otras alternativas posibles.

2.2.1. Python



Figura 2.17: Logotipo de Python

Python es un lenguaje de propósito general altamente utilizado en diversos ámbitos de la comunidad científica interpretando los conocimientos descritos por VanderPlas (2016). Python destaca por su sencillez y facilidad para la implementación de bloques de código o scripts sin tener que depender de una gran cantidad de ficheros externos. Es un lenguaje de sintaxis concisa y de alto nivel utilizado en la construcción de diversas librerías que, de forma idónea para los objetivos de este proyecto, implementan modelos de AI para la separación de un archivo de audio en distintos subarchivos llamados pistas o *stems*, contenedores para la construcción de una *Graphical User Interface* (GUI)⁶ y la implementación de algoritmos DSP.

Se han analizado otros lenguajes y entornos de trabajo como *Java* o *C++* para la construcción del sistema. A pesar de su capacidad para desarrollarse como tecnologías operantes, la justificación de Python frente a ellas reside en dos motivos. El primero es que todo el ecosistema del motor está cohesionado mediante este lenguaje. Tal y como se refleja en la tabla comparativa 2.1 posterior. El segundo, la librería elegida para la separación de pistas originalmente ha sido implementada en este lenguaje. De haberse optado por *C++* como posible lenguaje al ser nativo para otros muchos motores, se tendría que utilizar traducciones o *wrappers* de la librería original, causando así posibles fallos o latencias en el tiempo de procesamiento. Se escoge el lenguaje nativo de las herramientas que han sido empleadas en su desarrollo frente a otras que han sido traducidas a otros lenguajes.

⁶En castellano, *Interfaz gráfica de usuario*.

2.2.2. Blender



Figura 2.18: Logotipo de Blender

Blender es una suite de creación de software que permite generar animaciones, modelos y renderizaciones en un entorno tridimensional. Es un software de código abierto (Open Source), permitiendo el acceso y edición a su código fuente. Este motor dispone de un elenco de herramientas de construcción y visualización tales como: materiales para la creación de una GUI, inicialización de objetos desde código (pudiendo automatizar diversos procesos como la inicialización de las fuentes de sonido, siendo representadas por esferas, cubos, óvalos...) y obtención de trayectorias como curvas de Bézier o *splines*.

El mercado actual cuenta con una gran variedad de motores gráficos orientados para implementar software en 3D. Entre las opciones más destacadas del sector nos encontramos otros motores como *Unity Engine* o *Unreal Engine*, ambos caracterizados por su alta versatilidad. Entendiendo a los tres como posibles candidatos gracias al material presentado por Park y Choi (2025), la elección de Blender frente al resto de alternativas se sintetiza en la tabla 2.1.

Dada la naturaleza de Unity y Unreal existen dos factores que condicionan negativamente el desarrollo: las restricciones de licencia y la modificación del código del motor. En primer lugar, la restricción de licencia para sendos modelos recae en unas condiciones que restringen la libertad comercial mediante modelos de regalías o suscripciones tras alcanzar umbrales de facturación específicos. En segundo lugar, la modificación del código del motor es un aspecto determinante; dado que ninguno de ellos está específicamente pensado para una intervención profunda y gratuita en sus sistemas de gestión de audio o a niveles generales. En contraposición a estos factores, la naturaleza de Blender se define a favor para su elección.

Asimismo, el condicionante principal que justifica la elección de la primera opción ante el resto es que tanto la herramienta de separación de pistas como el paradigma del software 3D se implementan en Python. Se define entonces la unidad del proyecto mediante su propio lenguaje, promoviendo la convergencia tecnológica de las herramientas nativas del motor y la librería de separación, entre otras como *numpy* y la nativa de creación de *add-ons bpy*, como ya se ha explicado anteriormente.

Motor Característica	Blender	Unity	Unreal
Licencia	Código abierto	Pago	Pago
Propiedad del software	Código abierto	Pago	Pago
Herramientas para GUI	Python	C#	C++
Complejidad de desarrollo	Moderada	Alta	Alta
Integración con python	Nativa	C#	C++

Tabla 2.1: Comparativa de características entre los principales motores 3D.

2.2.3. Demucs



Figura 2.19: Logotipo de Meta, desarrolladores de Demucs.

Demucs es un proyecto desarrollado por Meta que se encarga de aislar las pistas de voz, baterías, bajos y melodías de una canción mediante el uso de AI. Sus versiones más recientes operan tanto en el dominio de la frecuencia como en el dominio del tiempo para minimizar los artefactos de separación introducidos durante el proceso, aparte de los ya existentes si se tuvieran los stems individuales de las pistas originales.

A modo de alternativa, existen otras tecnologías como *Moises* o *Spleeter* que tienen el mismo cometido que la herramienta elegida. Justificando el proceso de exclusión, la primera de ambas se descarta por su modelo de negocio de plataforma como servicio y por no disponer de una librería en Python donde poder utilizar su tecnología, más, si así se quisiera utilizar, la conexión a través de su *Application Programming Interface* (API)⁷ es una herramienta de pago.

La tecnología restante se sitúa en igualdad de condiciones con respecto a su función como librería para la integración en entornos de desarrollo. El resultado de su descarte reside en su operabilidad exclusivamente sobre el dominio frecuencial mediante espectrogramas. La inexistencia de la componente temporal en la separación de la tecnología de Spleeter hace

⁷En español, *Interfaz de Programación de aplicaciones*. Conjunto de protocolos, normas y definiciones que permiten la conexión de una tecnología de software con el resto sin que sepan su implementación interna.

que las señales resultantes no posean la calidad suficiente para el correcto desarrollo del proyecto. Demucs cuenta con valores de *Signal-To-Distorsion ratio* (SDR)⁸ (All) mayores a Spleeter, ya que Demucs alcanza 6.28 dB frente a los 5.91 dB de Spleeter (véase la tabla de la página 9 del artículo Défossez et al. (2019)).

2.3. Análisis del mercado

A continuación se procede a explicar las principales herramientas existentes en el sector de la generación de audio en tres dimensiones y cuáles son los factores diferenciadores del proyecto frente a ellas:

2.3.1. Tipos de herramientas existentes y limitaciones

Contextualizando brevemente, el auge del contenido multimedia inmersivo durante los últimos años ha popularizado el término 8D. Este es utilizado para referirse a cualquier tecnología que pretende emular y generar una audición espacial mediante el uso de auriculares. Actualmente esta oferta todavía no tiene la demanda que se esperaría de otros ámbitos como la realidad aumentada per se. Además, el factor concreto de que estas herramientas estén únicamente especializadas en el manejo y procesamiento de audio hace que se comprendan dentro de un segmento de mercado bastante específico. Por ello la oferta actual de herramientas está fragmentada entre soluciones de alta complejidad técnica y servicios automatizados con escaso control creativo y que sobre todo, no son de código abierto para todo el mundo. A continuación se enumeran y describen los grupos donde se incluyen algunas de las herramientas 8D existentes:

- **Plugins para *Digital Audio Workstation* (DAW)s:** comprenden las herramientas diseñadas para incluirse en estaciones de trabajo profesionales como *Ableton Live* o *Logic Pro* entre muchas otras. El ejemplo más notorio es *Sennheiser Ambeo Orbit*, una herramienta gratuita. Siendo su principal virtud la posibilidad de que cualquier persona pueda interactuar con el audio 8D, las pistas se han de mezclar en un espacio 2D. Entonces, si ya de por sí los DAW presentan una complejidad teórica en su funcionamiento, la necesidad de abstracción espacial del 2D al 3D es la condición limitante para muchas personas. A continuación un ejemplo de su interfaz:

⁸En español, *Relación señal distorsión*. Define la potencia de una señal deseada en comparación con la potencia de los artefactos de distorsión introducidos durante el procesamiento, transmisión o separación de fuentes.

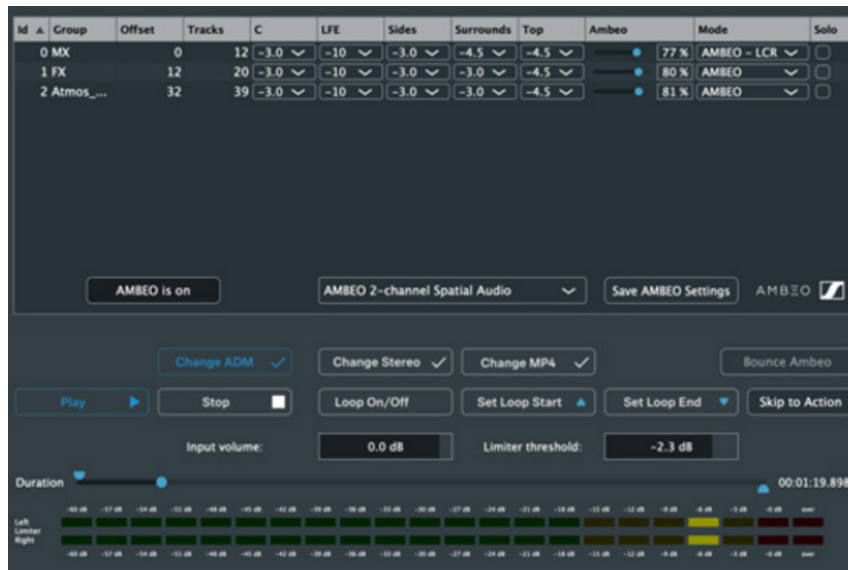


Figura 2.20: Interfaz principal de Sennheiser AMBEO Orbit.

Nota. Adaptado de *AMBEO spatial audio: Herramientas de audio inmersivo*, por Sennheiser (2024).

- **Motores de videojuegos o Middleware:** estas herramientas permiten gestionar el sonido de forma dinámica y envolvente dentro de un entorno virtual como *Unity* o *Unreal Engine*. Dentro de la oferta el más utilizado es el plugin **Steam Audio**, el cual se especializa en aportar una simulación espacial y aplicar efectos de oclusión basándose en la geometría del escenario. A pesar de su potencia técnica, su uso principal es la interactividad en tiempo real y la simulación de físicas, no tiene autoría con fines musicales, también carece de funciones nativas para la exportación final de un archivo en un formato de audio convencional, uno de los puntos clave a tratar por el sistema de la propuesta.
- **Convertidores web 8D:** al igual que existen servicios web que realizan la conversión entre distintos formatos, con el audio 8D sucede lo mismo. Estos operan transformando archivos estéreo a 8D de forma automatizada mediante procesos cerrados. Un ejemplo de ello la web de Audioalter (2026). Las limitaciones existentes se presentan en la modificación de las propiedades espaciales. Según la descripción en su página web el método que describe no permite posicionar libremente (o al menos con un cierto grado de libertad) las fuentes, aparte que no las separa, únicamente acepta un solo archivo de audio.

2.3.2. Aspectos diferenciadores de la propuesta

Conociendo varias de las limitaciones que tienen las tecnologías presentes hoy en día, la propuesta se diferencia en:

- **Pipeline unificado:** todas las tecnologías anteriores están limitadas o no incluyen distintos aspectos como la separación por pistas mediante AI, falta de libertad posicional en las fuentes y/o un apartado visual en 3D.

- **Autoría orientada a la exportación local:** mientras que en otros ámbitos como los motores de videojuegos están optimizados para la interactividad en tiempo real, esta herramienta se centra en la producción de una obra final exportable en formatos estándar que permitan la escucha directa.
- **Correlación visual-auditiva:** a diferencia de los DAW con interfaces esquemáticas en 2D, este sistema permite que el usuario pueda ver donde está posicionado el sonido gracias a *Blender*
- **Accesibilidad del procesado DSP:** el sistema automatiza procesos relacionados con la ingeniería acústica, concretamente el proceso de audición binaural. Esto permite que creadores de contenido sin formación técnica en acústica generen contenido inmersivo sin necesidad de conocimientos técnicos extensos.

Para un mejor entendimiento, se incluyen estos aspectos en la tabla 2.2 comparativa para las herramientas descritas en la sección 2.3.1:

Solución Característica	Plugins DAW	Steam Audio	Web	Propuesta
Separación AI	No	No	No	Sí(Demucs)
Interfaz 3D	2D	Sí	No	Sí(Blender)
Exportación local	Si	Limitada	Sí	Sí
Modelo de software	Propietario	Cerrado	Cerrado	Open Source

Tabla 2.2: Comparativa entre los ejemplos mencionados y la propuesta

En conclusión, el proyecto no pretende competir únicamente como un sistema de audio, sino como un sistema de reproducción multimedia que facilita el acceso a la creación de contenido 8D de calidad y como contribución dentro del ámbito de las tecnologías abiertas.

Capítulo 3

Planificación

El siguiente capítulo tiene como objetivo presentar la metodología de trabajo elegida, el desglose de las tareas, costes y viabilidad del proyecto.

3.1. Metodología de desarrollo

El desarrollo del proyecto se llevará a cabo mediante una **metodología incremental**. Su elección se fundamenta en que, para un reproductor de estas características, el procedimiento principal consiste en desarrollar un algoritmo matemático con el que reproducir música en 8D y no en un software cerrado para un cliente particular. La metodología tampoco contemplará la gestión de equipos ágiles, ya que será desarrollada por una sola persona, descartando el marco de trabajo SCRUM y las historias de usuario. La metodología se centrará en entregar un producto mínimo viable conforme a los objetivos del apartado 1.3 y los requisitos del apartado 4.1, el rendimiento algorítmico y la viabilidad. Por el contrario, el hecho de que sea incremental implica que la testabilidad de la interfaz estará más enfocada a entregar un producto funcional, que al hecho de proporcionar una *User Experience* (UX) sobresaliente. Se tiene más en cuenta la funcionalidad que el apartado visual de la *User Interface* (UI).

Los incrementos se justifican bajo la premisa del ciclo de ejecución del reproductor: la separación de pistas, la generación de una pista 8D, la reproducción y la exportación. Los incrementos solo tendrán en cuenta verificar que su acción se realiza correctamente, a causa de ser un proyecto orientado a código abierto, donde más personas podrían añadir sus módulos DSP y de separación.

Por último, el desarrollo del proyecto se realizará en paralelo a las tareas relacionadas con la redacción de la presente memoria técnica y la implementación del reproductor. A continuación, definida la metodología y sus características organizativas se desglosarán las fases del trabajo.

3.2. Estructura del desglose de trabajo

La *Estructura de desglose del trabajo* (EDT) del proyecto se desarrollará en cinco fases:

- **Fase 1: Investigación del estado del arte y viabilidad del proyecto.** Ejecución inicial del proyecto.
 - 1.1. Investigación sobre la audición binaural y su implementación dentro del entorno virtual.
 - 1.2. Análisis de las herramientas existentes del mercado.
 - 1.3. Investigación sobre librerías de implementación y API de Blender.
 - 1.4. Estimación de viabilidad, costes y riesgos.
 - 1.5. Redacción del estado del arte, planificación y riesgos de la memoria técnica.
- **Fase 2: Incremento 1 - Arquitectura base y módulo de separación con IA.** Estructuración inicial del proyecto.
 - 2.1. Definición de los requisitos funcionales y no funcionales.
 - 2.2. Diseño de la arquitectura general del sistema.
 - 2.3. Implementación general de la arquitectura y UI del sistema.
 - 2.4. Implementación y verificación del módulo de separación con AI.
 - 2.5. Redacción del análisis, diseño e implementación de la memoria técnica del incremento.
- **Fase 3: Incremento 2 - Integración de la escena 3D y módulo de espacialización 8D.** Sincronización inmersiva y generación del módulo de la pista 8D.
 - 3.1. Implementación de los elementos visuales de la escena.
 - 3.2. Diseño de la cadena del procesador 8D.
 - 3.3. Implementación y verificación del módulo de espacialización DSP.
 - 3.4. Redacción del análisis, diseño e implementación de la memoria técnica del incremento.
- **Fase 4: Incremento 3 - Exportación y discusión global.** Exportación y realización del conjunto de pruebas técnicas del proyecto.
 - 4.1. Implementación y verificación de la exportación de la pista 8D.
 - 4.2. Pruebas de rendimiento computacional.
 - 4.3. Pruebas de calidad de audio.
 - 4.4. Redacción de la implementación y de la discusión de resultados de la memoria técnica del incremento.
- **Fase 5: Conclusiones y cierre del proyecto.** Cierre y empaquetado de la memoria y del proyecto.
 - 5.1. Comparación empírica de la planificación, tiempo y costes reales frente a los estimados.

- 5.2. Redacción de resumen, conclusiones y trabajo futuro de la memoria técnica.
- 5.3. Revisión general del proyecto y de la memoria técnica.
- 5.4. Empaquetado y entrega del proyecto y de la memoria técnica.

La EDT comienza recabando el marco teórico y reconociendo el alcance del proyecto a nivel técnico y legal. Conocido su alcance, se implementan los tres incrementos relacionados con los tres procedimientos que realiza el reproductor. Cada uno de estos incrementos tendrá una verificación unitaria como evidencia de que la acción se realiza correctamente. El segundo incremento, al convertir una pista estéreo en 8D dando sentido a todo el proyecto, será al que se le realicen pruebas sobre su funcionamiento. Por consiguiente, la fase 4 discutirá el rendimiento de las pruebas realizadas para dicho incremento. Finalmente se empaquetará el proyecto y la memoria.

Con la EDT propiamente estructurada, se necesita asignar las distintas tareas a los distintos roles profesionales, independientemente si el trabajo ha sido realizado por una sola persona.

3.3. Asignación de perfiles profesionales

La asignación de los perfiles profesionales responde a las características de las distintas tareas. En la EDT, se diferencian varios tipos, resultando en los siguientes perfiles profesionales: **gestor de proyectos, ingeniero de audio e ingeniero backend**. El primero se encargará de todas las tareas relacionadas con la planificación, integración de la memoria, organización de tareas del resto de perfiles y entrega de los materiales entregables. El segundo, se encargará de las tareas que requieran aportar conocimientos sobre audio a nivel teórico y desarrollo técnico, sirviendo como apoyo al gestor para agregar calidad técnica a la memoria. Este ingeniero realizará y evaluará las pruebas de calidad de audio. El último, se encargará de integrar el proyecto a escala general, agrupando los distintos incrementos en el proyecto entregable final. Al igual que el anterior, también se apoya en el gestor para agregar la calidad técnica necesaria para la memoria. Este ingeniero realizará y evaluará las pruebas de rendimiento computacional.

3.4. Estimación temporal

Asignados los roles, se utilizará la **estimación por tres valores** para las tareas de la EDT. Su uso está justificado al no tener un equipo grande de desarrollo y ser un proyecto que mezcla varias competencias diferentes, pudiendo aportar una relación realista entre las tareas. Sin embargo, pese a ahorrar burocracia en la estimación, este método conlleva una alta carga subjetiva que dificulta la planificación inicial. Dicho esto, la fórmula empleada para el cálculo temporal es la siguiente:

$$t_e = \frac{(t_o + 4t_m + t_p)}{6} \quad (3.1)$$

$$t_e \rightarrow \text{tiempoEstimado} \quad (3.2)$$

$$t_o \rightarrow \text{tiempoMedio} \quad (3.3)$$

$$t_m \rightarrow \text{tiempoPromedio} \quad (3.4)$$

$$t_p \rightarrow \text{tiempoPesimista} \quad (3.5)$$

La estimación temporal se organiza según las actividades de cada fase, indicando al final la suma de todas ellas. El desglose temporal es el siguiente:

Actividad	t_o	t_m	t_p	t_e
Investigación sobre la audición binaural y su implementación dentro del entorno virtual.	2	4	7	4.17
Análisis de las herramientas existentes del mercado.	1	2	4	2.17
Investigación sobre librerías de implementación y API de Blender.	2	3	6	3.33
Estimación de viabilidad, costes y riesgos.	1	2	3	2.00
Redacción del estado del arte, planificación y riesgos de la memoria técnica.	2	3	5	3.16
Total de tiempo estimado en días	8	14	25	14.83

Tabla 3.1: Estimación temporal de la Fase 1

Actividad	t_o	t_m	t_p	t_e
Definición de los requisitos funcionales y no funcionales.	1	2	4	2.17
Diseño de la arquitectura general del sistema.	2	3	5	3.17
Implementación general de la arquitectura y UI del sistema.	3	5	8	5.16
Implementación y verificación del módulo de separación con AI.	4	6	10	6.33
Redacción del análisis, diseño e implementación de la memoria técnica del incremento.	2	4	6	4.00
Total de tiempo estimado en días	12	20	33	20.83

Tabla 3.2: Estimación temporal de la Fase 2

Actividad	t_o	t_m	t_p	t_e
Implementación de los elementos visuales de la escena.	2	4	7	4.17
Diseño de la cadena del procesador 8D.	3	5	9	5.33
Implementación y verificación del módulo de espacialización DSP.	6	10	18	10.67
Redacción del análisis, diseño e implementación de la memoria técnica del incremento.	3	5	8	5.16
Total de tiempo estimado en días	14	24	42	25.33

Tabla 3.3: Estimación temporal de la Fase 3

Actividad	t_o	t_m	t_p	t_e
Implementación y verificación de la exportación de la pista 8D.	2	3	6	3.33
Pruebas de rendimiento computacional.	2	3	5	3.17
Pruebas de calidad de audio.	1	2	4	2.17
Redacción de la implementación y de la discusión de resultados de la memoria técnica.	2	3	5	3.16
Total de tiempo estimado en días	7	11	20	11.83

Tabla 3.4: Estimación temporal de la Fase 4

Actividad	t_o	t_m	t_p	t_e
Comparación empírica de la planificación, tiempo y costes reales frente a estimados.	0.5	1	2	1.08
Redacción de resumen, conclusiones y trabajo futuro de la memoria técnica.	1	2	4	2.17
Revisión general del proyecto y de la memoria técnica.	2	3	5	3.17
Empaquetado y entrega del proyecto y de la memoria técnica.	0.5	1	1.5	1.00
Total de tiempo estimado en días	4	7	12.5	7.42

Tabla 3.5: Estimación temporal de la Fase 5

Fase del proyecto	t_o	t_m	t_p	t_e
Fase 1: Investigación del estado del arte y viabilidad del proyecto.	8	14	25	14.83
Fase 2: Incremento 1 - Arquitectura base y módulo de separación con IA.	12	20	33	20.83
Fase 3: Incremento 2 - Integración de la escena 3D y módulo de espacialización 8D.	14	24	42	25.33
Fase 4: Incremento 3 - Exportación y discusión global.	7	11	20	11.83
Fase 5: Conclusiones y cierre del proyecto.	4	7	12.5	7.42
Total general estimado en días	45	76	132.5	80.24

Tabla 3.6: Resumen de la estimación temporal total del proyecto

Como se puede ver en la tabla 3.6 la estimación total de las tareas de la EDT es de **80.24 días**, siendo el segundo incremento el que más coste temporal añade al proyecto. Sobre las estimaciones, las tareas que más incertidumbre representan son la investigación inicial, la verificación e implementación de todos los módulos, las tareas de diseño y las pruebas de rendimiento y calidad. Estos tiempos se han orientado a prever lo máximo posible el retraso que pudiera surgir durante el desarrollo. Dicho esto, la planificación temporal se representará en un *Diagrama de Gantt* (DG), representando el flujo de trabajo para una única persona que asume los distintos perfiles profesionales descritos anteriormente.

3.4.1. Diagrama de Gantt

El DG conforme a la EDT y la estimación temporal de la tabla 3.4 es incluido en la tabla 3.1. Las tareas del diagrama comienzan a partir del **02-02-2026** y todas consideran el valor t_e anterior. Por tanto, con la asignación de tareas cerrada, se estimarán los costes:

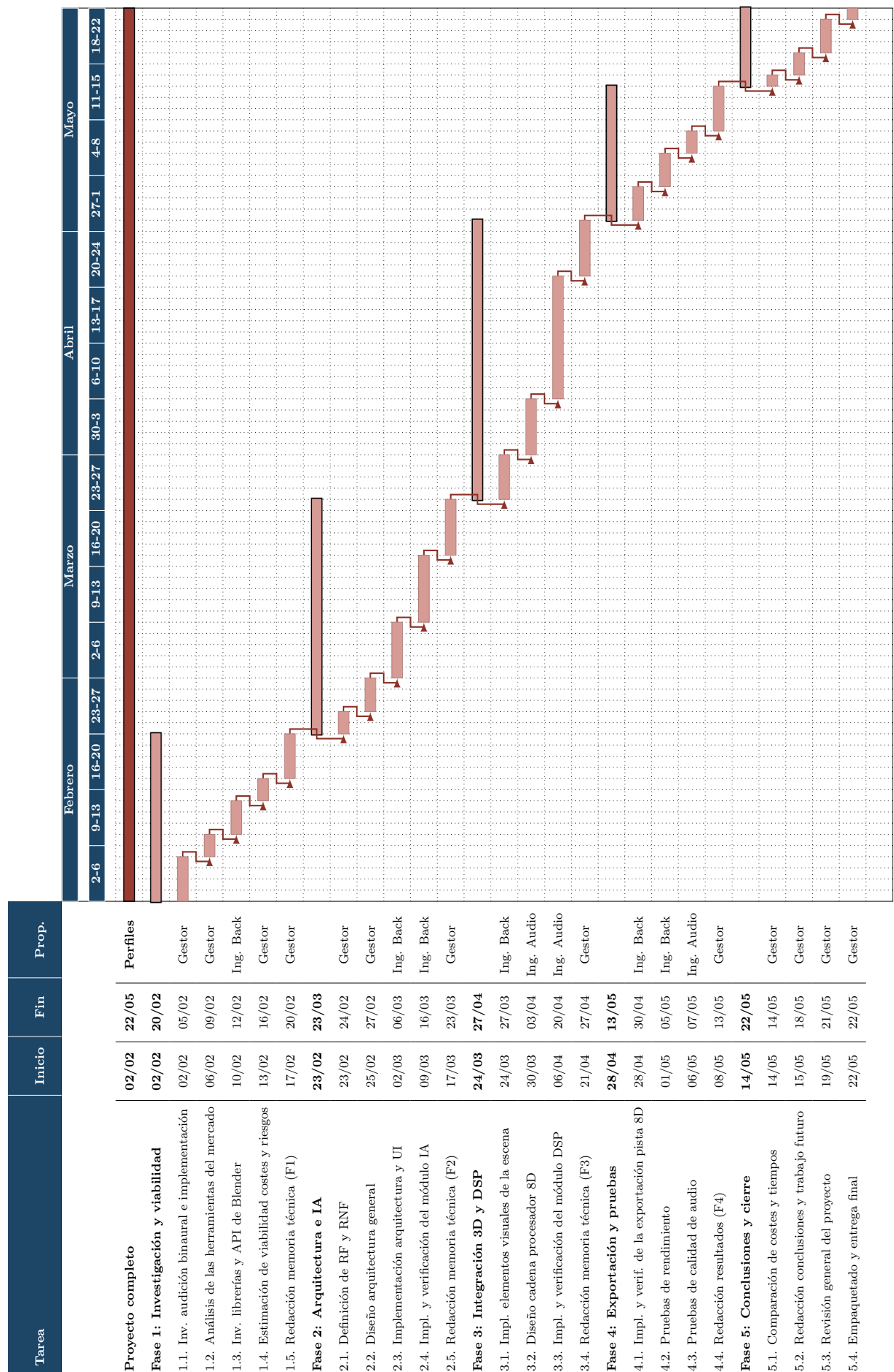


Figura 3.1: Diagrama de Gantt estimado del proyecto.

3.5. Estimación de costes

Los costes están divididos en dos categorías: personales y de software. El primero estima lo referido al pago del jornal por trabajo realizado a cada uno de los perfiles. El segundo estima el coste de licencia del software a utilizar, que en este caso será muy poco al utilizar herramientas de código abierto.

3.5.1. Costes personales

Para la obtención de los costes personales, la contratación de los distintos perfiles profesionales se ha realizado bajo un régimen general de trabajadores por cuenta ajena. Los tres perfiles responden a un mínimo de cinco años de experiencia en su sector y residen en la Comunidad Valenciana. El sueldo bruto medio de cada perfil profesional se ha estimado filtrando por sector, años de experiencia, localidad y salario anual en las guías salariales proporcionadas por Hays (2026). Bajo dicho régimen salarial, al sueldo bruto se le añadirá la parte porcentual de la cuota patronal de la seguridad social emitida por el Ministerio de la Presidencia, Justicia y Relaciones con las Cortes (2026). A partir de estos datos, los costes personales son los siguientes:

Profesional	Sueldo Neto	Seguridad Social (31 %)	Sueldo Bruto
Gestor de proyectos	31050.00 €	13950.00 €	45000.00€
Ingeniero de audio	35535.00 €	15965.00 €	51500.00 €
Desarrollador backend	35535.00 €	15965.00 €	51500.00 €

Tabla 3.7: Desglose de los costes directos personales para cada perfil profesional

En total, en disposición de los costes directos, el reparto de tareas específicas del DG y la cantidad de días estimados por la tabla 3.6 donde cada día lo conforman 8 horas laborables, el coste personal del proyecto es el siguiente:

Profesional	Horas Totales	Sueldo Neto	Seguridad Social	Sueldo Bruto
Gestor de proyectos	304	7600.00 €	2356.00 €	9956.00 €
Ingeniero de audio	144	4120.00 €	1277.20 €	5397.20 €
Ingeniero backend	192	5493.33 €	1702.93 €	7196.26 €
Total				22549.46 €

Tabla 3.8: Desglose de los costes personales

3.5.2. Costes de software

Los costes de software responden al material utilizado para la redacción de ciertos elementos contenidos en la memoria técnica y la implementación del reproductor 8D. El software utilizado durante el desarrollo es el siguiente:

- **Visual Studio Code:** Licencia Gratuita. Utilizada como entorno de desarrollo integrado para la codificación del proyecto.
- **Blender:** Licencia Gratuita. El motor 3D del proyecto.
- **Visual Paradigm 16.1:** Licencia Standard (\$19/mes $\rightarrow$ 16.55€/mes) . Utilizado para realizar los diagramas del proyecto.
- **Github:** Licencia Gratuita. Utilizado para guardar las actualizaciones y centralizar el código del reproductor.
- **Audacity:** Licencia Gratuita. Utilizado para realizar las pruebas de audio.

Como puede apreciarse, al ser un proyecto de código abierto, son muy pocos los costes de esta categoría. El desglose de los costes de software es el siguiente:

Software	Días de uso	Meses con suscripción	Precio Total
Visual Studio Code	35	∞	0.00 €
Blender	7	∞	0.00 €
Visual Paradigm 16.1 (Standard)	10	4	66.20 €
GitHub	35	∞	0.00 €
Audacity	5	∞	0.00 €
Total de Costes de Software			66.20 €

Tabla 3.9: Costes indirectos de software y licencias del proyecto.

3.6. Análisis de viabilidad del proyecto

La viabilidad del proyecto determina si este puede desarrollarse con garantías de éxito. Para esto, se considera la posible incertidumbre técnica, económica y legal existente.

3.6.1. Viabilidad técnica

Técnicamente, el contexto del proyecto representa la unión de distintos ejercicios comprendidos dentro de la ingeniería informática y del sonido. Dentro de las opciones del mercado, existen alternativas que ya implementan la separación de pistas, al igual que la conversión de pistas estéreo a 8D. La premisa sobre su viabilidad se fundamenta en la consecución por parte de terceros de estos ejercicios, debiendo ser estudiados los posibles riesgos técnicos al tratarse de la realización de particulares sin equipos ágiles en el tiempo total estimado.

3.6.2. Viabilidad económica

Económicamente, el proyecto no presenta ningún riesgo económico puesto que se está encargando un trabajo a distintos profesionales (independientemente de que haya sido realizado por una persona, se plantea como un ejercicio simulado.). La contratación por cuenta ajena representa la motivación personal del autor por llevar a cabo el proyecto, sin

depender de entidades externas que tengan que realizar una inversión de capital inicial. El régimen salarial propuesto no está sujeto a impagos de sueldos prorrateados a terceras personas.

3.6.3. Viabilidad legal

Legalmente, el proyecto trata las siguientes vertientes: el *Reglamento General de protección de Datos* (RGPD), la propiedad intelectual de los recursos técnicos empleados y la *Ley de Propiedad Intelectual* (LPI) sobre las pistas de audio tratadas por el reproductor. La primera de ellas no representa ningún riesgo, puesto que, aparte de las pistas introducidas, el sistema no necesita y/o pide ingresar datos personales y fiscales. El segundo responde a la GPL presentada y firmada al principio del documento. En última instancia, la LPI responde al tratamiento y uso de pistas originales en el sistema. A nivel legal, las pistas 8D resultantes responden a lo que se conoce como obras derivadas. El sistema actúa como un sistema pasivo que no altera la composición original, modificando únicamente sus propiedades posicionales. Cualquier uso no autorizado para la atribución de obras no originales a través del reproductor queda libre de toda responsabilidad por parte del sistema. Es responsabilidad del usuario determinar la propiedad de las pistas y/o masters que va a utilizar.

Por último, el conjunto de datos proporcionados por KEMAR para la implementación del procesador 8D ha sido atribuido a sus autores originales, quienes declaran de uso libre su base de datos. El conjunto de pistas utilizado para la medición de pruebas sobre la calidad de audio responde a una licencia *Creative Commons* (CCO), sea cual sea su fin. En resumen, legalmente no existen riesgos reales que alteren la viabilidad del proyecto.

3.7. Análisis de riesgos

Puesto que existen riesgos técnicos, se procede a su descripción:

- **Desbordamiento del uso de RAM del separador de AI:** la separación de pistas desborda el uso de RAM, dificultando o invalidando cualquier acción del equipo ajeno al reproductor.
- **Desbordamiento del coste computacional del procesador 8D:** la convolución matemática presenta un coste $O(n^2)$, dificultando o invalidando cualquier acción del equipo ajeno al reproductor.
- **Inestabilidad del hilo principal de la API de Blender:** la carga algorítmica de los procesos dificultan o invalidan la ejecución de otras acciones ajenas al reproductor en Blender.
- **Degradación acústica de la pista 8D:** la pista 8D resultante no se ajusta a la limitación sonora digital convencional que ha de tener, saturando y degradando la SDR entregada.
- **Artefactos audibles en la pista 8D:** la pista 8D introduce elementos audibles no deseados como clicks y artefactos, dificultando o invalidando la calidad de la pista original.

- **Incompatibilidad de las librerías externas en Blender:** el conjunto de librerías que permitiría desarrollar el trabajo no son compatibles con el entorno de Python de Blender.

Con los posibles riesgos analizados, se clasificarán según su *Nivel de riesgo* (NR) en base a la siguiente fórmula:

$$NR = \text{Probabilidad} \cdot \text{Impacto} \quad (3.6)$$

Aclarada la obtención del NR, los riesgos han sido clasificados según la siguiente escala¹. Recogidos en la tabla 3.10. Los valores de probabilidad e impacto han sido evaluados conforme a la incapacidad que otorgarían al proyecto.

ID	Riesgo	Probabilidad	Impacto	NR	Clasificación
R1	Desbordamiento del uso de RAM del separador de AI	3	4	12	Medio
R2	Desbordamiento del coste computacional del procesador 8D	4	4	16	Alto
R3	Inestabilidad del hilo principal de la API de Blender	3	5	15	Alto
R4	Degradación acústica de la pista 8D (Saturación)	2	3	6	Bajo
R5	Artefactos audibles en la pista 8D por separación	4	2	8	Medio
R6	Incompatibilidad de librerías externas en Blender	1	5	5	Bajo

Tabla 3.10: Evaluación de riesgos del proyecto.

Por último y con ello se cierra el capítulo de planificación, se establecen las siguientes medidas de contingencia para los riesgos descritos:

- **Desbordamiento del uso de RAM del separador de AI:** fomentar la separación de las pistas de forma asíncrona.
- **Desbordamiento del coste computacional del procesador 8D:** aplicación del teorema de la convolución para ahorrar tiempo de procesado, como ya se explica en el apartado 2.1.4.1.
- **Inestabilidad del hilo principal de la API de Blender:** encapsulamiento de los procesos más pesados en hilos del procesador distintos al hilo principal de Blender, evitando la mezcla de flujo de procesos entre las distintas tareas de forma asíncrona.
- **Degradación acústica de la pista 8D:** implementación de un techo de normalización de ganancia para la pista final, evitando que la pista se sature independientemente de la cantidad de subpistas que convolucione el reproductor.

¹Tanto la Probabilidad como el Impacto se evalúan en una escala de valores enteros del 1 al 5.

- **Artefactos audibles en la pista 8D:** aplicación de efectos de interpolación para transicionar entre las distintas HRIR cuando se procesa la pista 8D.
- **Incompatibilidad de las librerías externas en Blender:** forzar la instalación de las librerías en un entorno dedicado para la ejecución de las mismas, evitando colapsar la versión de Python nativa del motor 3D.

Capítulo 4

Análisis

En este capítulo se especifican las funcionalidades, restricciones técnicas y la secuenciación general del sistema de reproducción. En él, la **metodología de desarrollo incremental con validación técnica** descrita en el capítulo 3 influye directamente en la formulación de los apartados presentados.

4.1. Requisitos funcionales y no funcionales

Los *Requisito Funcional* (RF) y el *Requisito No Funcional* (RNF) representan, respectivamente, las acciones mínimas que deberá ejecutar el sistema de reproducción y cómo este deberá comportarse bajo una serie de restricciones técnicas. Este se centrará en el desarrollo de la pista binaural final, siguiendo los objetivos planteados en el capítulo 1.3 y las definiciones de los conceptos de las secciones teórica 2.1 y técnica 2.2 que posteriormente determinarán la fase de diseño del capítulo 5:

4.1.1. Requisitos funcionales

A efectos de cumplir los objetivos del proyecto, los RF se describen acompañando al subobjetivo que cumplen si es que tiene relación directa con el mismo. Por tanto, los RF son los siguientes:

- **RF-01 (Separación de audio con AI) → O2:** el sistema permitirá la carga de un archivo de audio estéreo y utilizará un modelo AI para dividir la señal original en cuatro pistas independientes que representarán distintos instrumentos de la pista original.
- **RF-02 (Aplicación de trayectorias predefinidas) → O3:** el sistema proporcionará un conjunto de trayectorias espaciales predefinidas que el usuario podrá asignar a las pistas de audio cargadas en el entorno.
- **RF-03 (Modificación de trayectorias) → O3:** el sistema permitirá al usuario editar de forma manual los parámetros de las trayectorias asignadas, validando que las modificaciones se mantengan dentro de los límites operativos restringidos por la interfaz.

- **RF-04 (Captura de coordenadas)** → **O3**: el sistema extraerá y calculará la posición relativa (r, θ, δ) de cada subpista en el espacio 3D de Blender en función de la posición determinada por su trayectoria.
- **RF-05 (Selección de respuesta al impulso)** → **O3**: el sistema obtendrá de una base de datos las respuestas al impulso HRTF que mejor se correspondan con las coordenadas espaciales extraídas en cada instante temporal (r, θ, δ) .
- **RF-06 (Procesamiento diferido)** → **O3**: el núcleo del sistema aplicará un algoritmo DSP en diferido a cada subpista individual, utilizando su respectiva trayectoria dinámica y las HRTF seleccionadas.
- **RF-07 (Sincronización visual)** → **O4**: si el procesamiento de la cadena DSP concluye con éxito, el sistema generará de forma automática la animación en la escena moviendo los objetos emisores que representan a las fuentes a través de las trayectorias previamente definidas para acompañar a la canción.
- **RF-08 (Almacenamiento del resultado)** → **O4**: el sistema almacenará la pista de salida generada en el almacenamiento local del sistema.

4.1.2. Requisitos no funcionales

A efectos de cumplir con el comportamiento del sistema y con la misma clasificación de subobjetivos, los RNF son los siguientes:

- **RNF-01 (Rendimiento del procesamiento)** → **O3**: el algoritmo de la cadena DSP deberá procesar la espacialización del audio, asegurando un tiempo menor a un factor de 0.5 respecto a la duración de la canción original.
- **RNF-02 (Eficiencia y consumo de memoria)** → **O3**: el sistema deberá cargar e instanciar la base de datos de respuestas al impulso HRTF una única vez, asegurando que el consumo máximo de memoria RAM en el equipo local no exceda 1 GB.
- **RNF-03 (Integración con el software 3D)** → **O1**: el *add-on* deberá estar desarrollado estrictamente en Python, donde su interfaz gráfica se integrará de forma nativa en el entorno 3D mediante la herencia exclusiva de las clases estructurales `bpy.` de Blender.
- **RNF-04 (Compatibilidad del formato de audio)** → **O1**: el motor de audio central procesará las señales acústicas garantizando la compatibilidad con archivos en formato *.WAV* sin compresión, trabajando a una frecuencia de muestreo de 44.1 kHz o superior.
- **RNF-05 (Aislamiento de procesos)** → **O3**: las excepciones o errores matemáticos originados en los *plugins* deberán ser capturados y controlados por el sistema para garantizar que un fallo no provoque el colapso de Blender.

Una vez descritos los RF y RNF, el siguiente paso en la etapa de análisis corresponde a definir la interacción del usuario con el sistema a través del *Diagrama de Casos de Uso* (D-CDU).

4.2. Diagrama de casos de uso

El D-CDU que muestra la interacción entre los distintos actores y el sistema de reproducción es el siguiente:

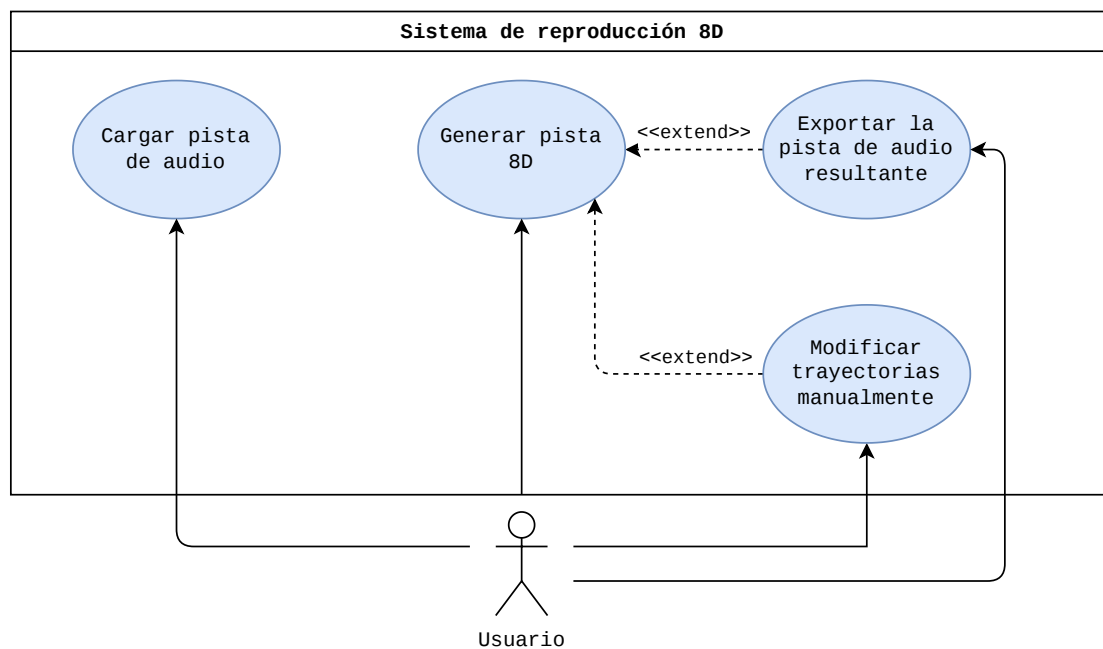


Figura 4.1: DCDU del sistema de reproducción 8D.

4.2.1. Descripción de los casos de uso

El conjunto de tablas 4.1 - 4.4 procede a desglosar cada *Caso de uso* (CU) conforme al diagrama de la figura 4.1, los RF y los RNF:

Identificador	CU-01
Nombre	Cargar pista de audio
Actor	Usuario
Propósito	Cargar una pista de audio estéreo dentro del sistema para su separación en subpistas y procesamiento.
Resumen	El usuario introduce una pista estéreo en el sistema la cual se procesará y otorgará en subpistas a través de su separación por AI.
Tipo	Primario
Referencias	RF-01, RNF-03, RNF-04 y RNF-05
Precondiciones	El <i>add-on</i> debe estar inicializado correctamente dentro del entorno de Blender.
Postcondiciones	El sistema obtiene las subpistas generadas por el motor de separación de AI.
Flujo principal	1. El usuario pulsa el operador correspondiente a la carga de una pista de audio. 2. El sistema abre el explorador de archivos local. 3. El usuario selecciona la pista estéreo que quiere pasar a ser procesada dentro del sistema. 4. El sistema recoge la pista y comprueba que corresponde a sus características. 5. El sistema realiza la separación mediante un módulo de AI otorgando las pistas resultantes y actualizando la interfaz.
Flujo alternativo	■ 4.a. Si la pista no corresponde a las características, el sistema captura la excepción y lanza un mensaje indicando el tipo de error producido.

Tabla 4.1: Especificación del CU-01: *Cargar pista de audio*

Identificador	CU-02
Nombre	Generar pista 8D
Actor	Usuario
Propósito	Realizar el proceso de espacialización de cada una de las subpistas.
Resumen	El usuario inicia el proceso de espacialización de las subpistas de audio. En él se calcula la posición de la trayectoria de cada una de ellas para después aplicar el procedimiento de la cadena DSP necesaria otorgando posteriormente la pista resultante final.
Tipo	Primario
Referencias	RF-04, RF-05, RF-06, RF-07, RNF-01, RNF-02, RNF-03, RNF-05
Precondiciones	El sistema ha cargado correctamente las subpistas junto con todos los datos de sus trayectorias.
Postcondiciones	El procesamiento de las pistas ha procedido correctamente permitiendo que se pueda realizar la exportación de la pista binaural resultante y la visualización a través de Blender.
Flujo principal	1. El usuario pulsa el operador correspondiente a la generación de las pistas 8D. 2. El sistema recoge los datos sobre la trayectoria de las subpistas. 3. El sistema recoge las HRIR de la base de datos correspondiente a las trayectorias de las subpistas. 4. El sistema aplica la DSP otorgando la pista binaural resultante y actualizando la interfaz.
Flujo alternativo	■ 1.a. Si el usuario desea ajustar la trayectoria de las fuentes antes de su procesamiento, el CU se extiende al CU-03, véase la tabla 4.3. ■ 4.a. Si durante el procesamiento DSP sucede un error, el sistema lo captura y lanza un mensaje indicando el tipo de error producido. ■ 4.b. Si el usuario desea guardar la pista resultante, el CU se extiende al CU-04, véase la tabla 4.4.

Tabla 4.2: Especificación del CU-02: *Generar pista 8D*

Identificador	CU-03
Nombre	Modificar trayectorias manualmente
Actor	Usuario
Propósito	Modificar las trayectorias de las subpistas manualmente según los intereses del usuario.
Resumen	El usuario modifica las trayectorias de cada una de las subpistas a través de las opciones predefinidas dentro de la interfaz.
Tipo	Secundario
Referencias	RF-02, RF-03, RNF-03
Precondiciones	El sistema ha cargado correctamente las subpistas generadas por el motor de separación de AI.
Postcondiciones	El sistema aplica las modificaciones pertinentes actualizando los nuevos datos listos para el procesamiento en el CU-02, véase la tabla 4.2.
Flujo principal	1. El usuario selecciona la subpista a la que le quiere aplicar las modificaciones. 2. El usuario modifica las características posicionales de la subpista interactuando con los controles de la interfaz. 3. El sistema actualiza internamente las propiedades de la trayectoria de la subpista conforme el usuario va realizando los cambios.
Flujo alternativo	■ 2.a. Si el usuario introduce o intenta asignar valores fuera de los rangos predeterminados por el sistema o la interfaz, este lo rechaza asignando los valores permitidos más cercanos.

Tabla 4.3: Especificación del CU-03: *Modificar trayectorias manualmente*

Identificador	CU-04
Nombre	Exportar la pista de audio resultante
Actor	Usuario
Propósito	Exportar la pista de audio resultante en el ordenador local del usuario.
Resumen	El usuario guarda la pista resultante en el espacio deseado.
Tipo	Secundario
Referencias	RF-08, RNF-03 y RNF-04
Precondiciones	El sistema ha procesado correctamente la pista binaural.
Postcondiciones	El sistema guarda un archivo de audio correspondiente a la pista binaural procesada.
Flujo principal	1. El usuario pulsa el operador correspondiente en la exportación de la pista de audio binaural resultante. 2. El sistema abre el explorador de archivos local. 3. El usuario selecciona donde quiere guardar la pista. 4. El sistema guarda la pista en el la dirección indicada.
Flujo alternativo	■ 3.a. Si el usuario decide cancelar la operación cerrando el explorador de archivos, el sistema aborta la exportación y vuelve a su estado anterior. ■ 4.a. Si ocurre un error de escritura, el sistema lo rechaza y lanza un mensaje indicando el tipo de error producido.

Tabla 4.4: Especificación del CU-04: *Exportar la pista de audio resultante*

Con la introducción de las siguientes tablas ha quedado registrado el flujo principal del sistema. Los flujos descritos destacan por recoger los errores que pudieran aparecer, anticipándose al colapso del sistema y utilizando los distintos módulos cuando se necesiten, teniendo en cuenta precondiciones básicas de instalación y la recogida de las propiedades generales del proyecto para realizar el procesamiento principal del CU-02. Este será descrito más adelante en el apartado 5.3.2.

4.3. Diagrama de clases conceptual

El *Diagrama de Clases Conceptual* (DCC) que muestra el modelo de dominio conceptual real de los componentes del sistema y sus relaciones lógicas, independientemente del trabajo a nivel de su implementación en código, es el siguiente:

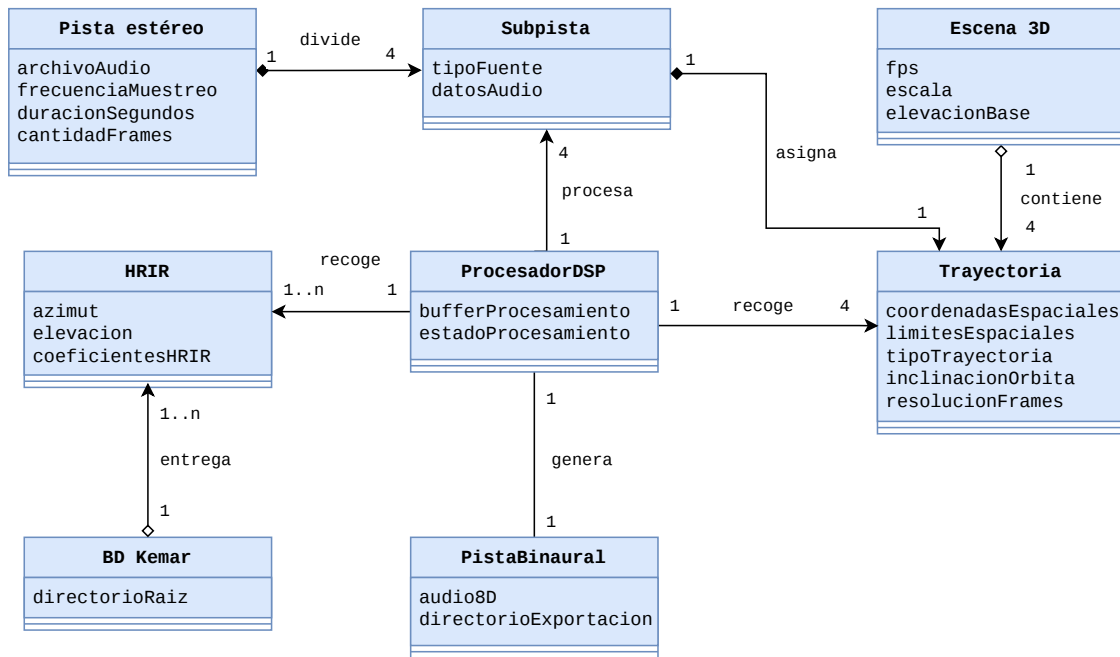


Figura 4.2: DCC del sistema de reproducción 8D

La descripción y justificación de cada una de las clases del diagrama 4.2 reside en:

- **Pista estéreo:** corresponde al audio original que introduce el usuario dentro del sistema para generar la pista binaural.
- **Subpista:** corresponde a las subpistas individuales separadas con AI pertenecientes a la pista estéreo introducida por el usuario, de carácter atómico, pues tienen las mismas características que la pista base, solo que separadas por instrumentos.
- **Escena 3D:** corresponde al entorno donde existen todos los elementos con los que interactúa el usuario. Esta contiene las trayectorias de cada subpista.
- **Trayectoria:** corresponde a las propiedades del itinerario de cada una de las subpistas, de carácter atómico, pues sin ellas no podrían asignarse y/o modificarse.
- **BD Kemar:** corresponde a la base de datos que entrega las HRIR con las que se obtiene la respuesta al impulso correspondiente.
- **HRIR:** corresponde a la respuesta al impulso de una posición (θ, δ) determinadas, que serán recogidas por el procesador 8D.
- **Procesador DSP:** corresponde al procesador de la información recogida por el sistema. Recoge el audio, trayectoria y HRIR en la forma resultante, la pista binaural.
- **Pista Binaural:** corresponde a la pista binaural generada por el modelo de procesamiento 8D.

La justificación tanto del DCC como de sus atributos recae en el procesamiento principal del sistema, obtener una pista binaural. Por ello las clases más extendidas son **Pista estéreo** al representar la pista inicial, **Trayectoria** por otorgar las propiedades posicionales

a las subpistas que hereda y **ProcesadorDSP** por realizar el procedimiento objetivo del sistema. El resto sirven como apoyo sin dejar de ser fundamentales para la consecución del objetivo, consideradas como clases secundarias de igual valor. Otro punto a destacar es el hecho de no incluir al módulo de separación con AI como clase directa, pues es más un proceso puente que forma parte del proceso completo. Las clases tienen una serie de atributos que se han de conocer para comprender qué propiedades tiene el sistema de reproducción más en profundidad.

4.3.1. Descripción de atributos del DCC

La tabla con la descripción de los atributos del DCC para cada una de las clases del sistema junto con sus características es incluido en la tabla **A.1** del apéndice a efectos de maquetado y estilo.

Acompañando la explicación sobre el DCC, los atributos representan las características esenciales; unas pistas, unos atributos posicionales, un procesador y distintas variables que necesita el proceso de convolución. Finalmente, dejando de lado la parte sobre el procesador y la implementación de la binauralidad, se tiene una clase que comprende los atributos relacionados con la **Escena 3D**, el visor que transmite la inmersividad al sistema. Sus atributos sirven como apoyo para mejorar la coherencia y comprensión visual al conjunto 3D y al movimiento inmersivo de la música.

4.4. Diagrama de secuencia conceptual

Por último, el *Diagrama de Secuencia Conceptual* (DSC) que muestra la interacción principal entre el usuario y el sistema completo es el siguiente:

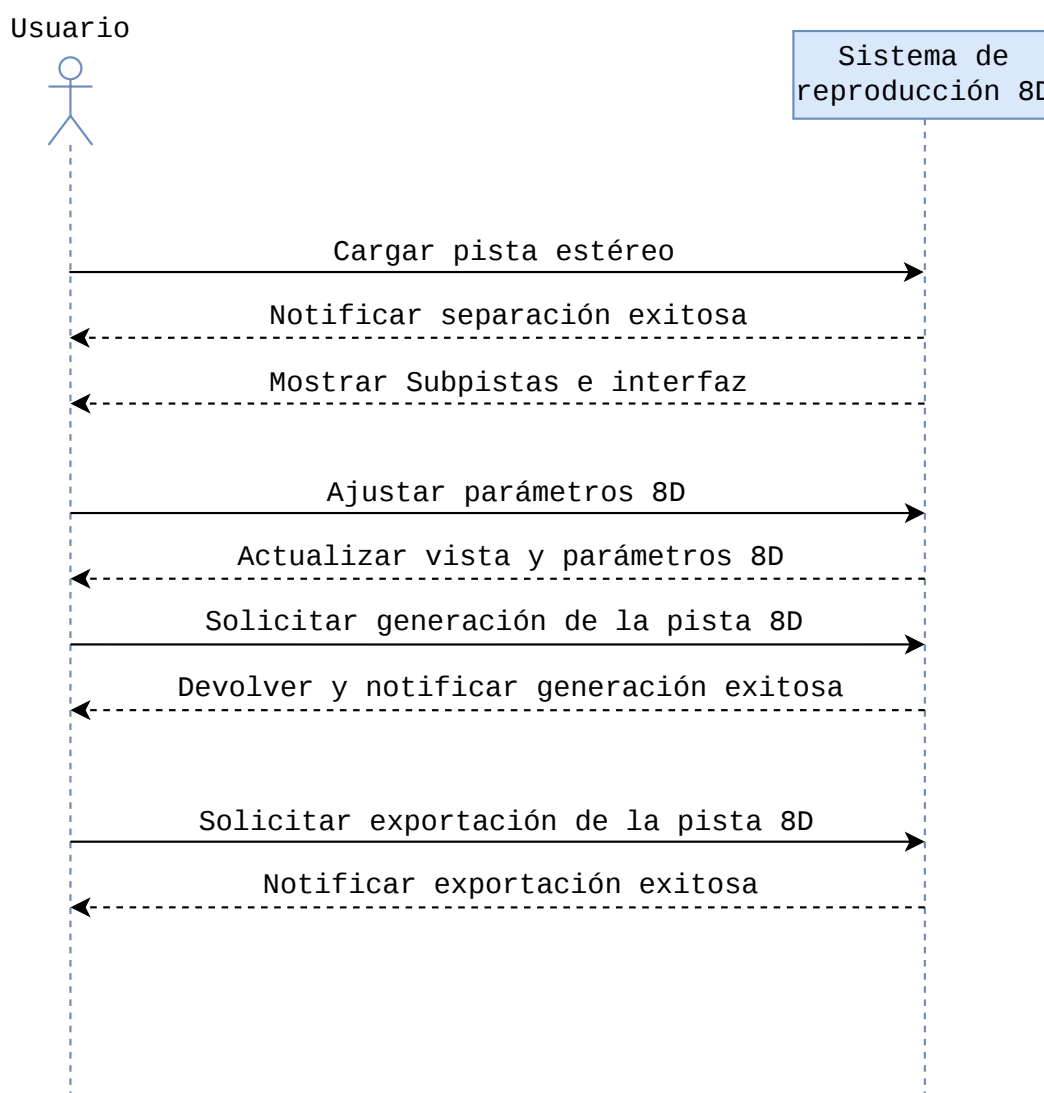


Figura 4.3: DSC del sistema de reproducción 8D

La figura 4.3 muestra cómo el usuario puede interactuar con tres procedimientos ligados en orden: la separación por pistas, la generación de la pista 8D y la exportación de la pista 8D. Estos procesos están relacionados con los cuatro CU señalados en el D-CDU que tienen asignación directa con el usuario. El punto más importante a reflejar en este flujo es la necesidad de que la interacción con el sistema sea ordenado. Este será uno de los puntos a tratar durante el proceso de diseño debido a que, sin una pista estéreo no hay subpistas, y sin subpistas no hay pista 8D ni exportación final. La arquitectura propuesta para implementar el DSC deberá tener en cuenta la posibilidad de alterar el orden de estas acciones, evitando así posibles errores en el funcionamiento del reproductor. Estos aspectos se abordarán en el capítulo a continuación.

Capítulo 5

Diseño

En este capítulo se desglosará el apartado de diseño del sistema de reproducción. Si el capítulo anterior comprende *qué* características tiene el sistema, este presenta la abstracción sobre *cómo* se desarrollará.

5.1. Arquitectura

La arquitectura presentada ha de responder en partes iguales tanto al proceso de análisis como a la optimización de las características del propio diseño. A continuación, se presenta una descripción sobre algunos tipos de arquitecturas de software sintetizadas en el material bibliográfico de Richards y Ford (2020), adaptándolas al sistema de reproducción:

- **Por capas:** representa una arquitectura monolítica¹ que se divide en capas independientes separadas por funcionalidades con una responsabilidad única por capa, pero que a su vez dependen unas de otras según la prioridad de las peticiones del sistema. El uso de esta arquitectura para el sistema de reproducción implicaría que la interfaz se acople teniendo toda la relevancia sobre el manejo de los datos de las trayectorias de las pistas y el acceso a la base de datos entre muchas más funcionalidades. A pesar de ser íntegra y fácil de comprender, cualquier cambio definido por la interfaz implicaría una extensión de errores por todo el sistema y un cambio sistemático para que cumpla con lo dictaminado por ella.
- **Microkernel:** representa una arquitectura monolítica donde su topología tiene una única unidad central desde la que se acoplan componentes adicionales o *plugins*. Es útil a la hora de construir sistemas altamente personalizables que necesitan añadir características nuevas constantemente sin tener que modificar el núcleo del sistema. El uso de esta arquitectura en el sistema de reproducción implicaría delegar las tareas a las posibles funcionalidades independientes, restando en escalabilidad en cuanto al uso de usuarios activos simultáneos y una tolerancia a fallar.
- **Basada en servicios:** representa una arquitectura distribuida que divide la aplicación en servicios independientes. Aporta una gran flexibilidad al tratar estos servicios por separado, pero mantiene una menor complejidad al compartir una única interfaz de usuario y base de datos para todo el proyecto. El uso de esta arquitectura para el

¹Arquitectura donde todo el sistema está acoplado en una misma unidad de despliegue.

sistema de reproducción implicaría entender el tratamiento de audio para múltiples usuarios conectados al mismo tiempo, utilizando el procesador y las propiedades conjuntamente.

- **Microservicios:** representa una arquitectura distribuida que aísla por completo cada enfoque del sistema. Cada servicio que propone es muy pequeño y tiene un propósito único que opera de manera totalmente independiente. El uso de esta arquitectura en el sistema de reproducción implicaría desacoplar cada una de las microfuncionalidades dentro de cada CU como el servicio de separación con AI, la consulta de las propiedades posicionales, la obtención de las HRIR entre otras muchas operaciones.

Habiendo resumido el caso donde se utilice el diseño propuesto por el material bibliográfico, en la tabla 5.1 se presenta una comparativa entre las distintas arquitecturas respetando la valoración² impuesta por los mismos autores Richards y Ford (2020). En la tabla siguiente no se recogen todas las características, solamente las que corresponden a los intereses del proyecto y que por tanto, resultan en los puntos a enfocarse:

Arquitectura Característica	Por capas	Microkernel	Servicios	Microservicios
Escalabilidad	1	1	3	5
Rendimiento	2	3	3	2
Bajo coste	5	5	4	1
Modularidad	1	3	4	5
Simplicidad	5	4	3	1
Testabilidad	2	3	4	4
Tolerancia a fallos	1	1	4	4

Tabla 5.1: Comparativa entre las distintas arquitecturas

El análisis de la comparativa 5.1 arroja información de interés. En primer lugar, los factores que más se han valorado conforme a las características presentadas son: que el sistema de reproducción es individual para cada persona (no se trata la reproducción para varios sistemas a la vez), que ha de tener un rendimiento comprometido a la arquitectura local del sistema que lo va a utilizar, que esta sea simple de entender, que sea de bajo coste y que exista una capacidad para modularizar el sistema al ser de código abierto. Entonces, la *escalabilidad* en las arquitecturas basadas en servicios y microservicios es favorable, sin embargo, no es uno de los factores cruciales a tener en cuenta puesto que la reproducción del sistema no es simultánea entre usuarios. El *rendimiento* es favorable tanto en la basada en servicios y en la microkernel. El *bajo coste* es favorable en la arquitectura por capas y microkernel al estar bajo estructuras monolíticas centralizando el trabajo. La *modularidad* es favorable para las arquitecturas de servicios y microkernel. El sistema de capas es débil puesto que sus componentes dependen unos de otros, siendo una

²Valoración en una escala [1-5] donde 1 representa peor rendimiento y 5 el mejor rendimiento de dicha característica.

mala opción para un sistema compartimental ya referido anteriormente con la redacción de los RF y RNF. La *simplicidad* de la propia arquitectura es favorable para las mismas arquitecturas que presentan un bajo coste de implementación. La *testabilidad* es favorable para las arquitecturas basadas en microservicios, aunque microkernel también es valorada de forma positiva al proporcionar cierto grado de libertad entre sus módulos. Finalmente, la *tolerancia a fallos* representa un punto débil para las dos primeras arquitecturas debido a su estructura monolítica, pues si el sistema falla en cierto punto el error se extenderá al resto de la estructura integradora, por otro lado, los microservicios distribuyen el peso de los fallos entre sus componentes.

Bajo estas premisas, la que más se acopla a las directrices e intereses del proyecto y sobre la que se construirá el sistema de reproducción es la **arquitectura microkernel**. La elección se fundamenta en su isomorfismo³, el cual facilita una integración natural con el motor 3D. Al estar el proyecto concebido como un *add-on* para Blender, replica de forma exacta la topología a construir. Luego, a pesar de sus deficiencias tanto en escalabilidad como en tolerancia a fallos, el hecho de que el sistema esté destinado a un uso individual local, sin procesamiento simultáneo de múltiples usuarios, reduce el impacto de estas características.

En último lugar, la arquitectura utilizará un patrón **Modelo-Vista-Controlador (MVC)**. Aparte de ser una técnica ampliamente utilizada y conocida por el autor, aplica perfectamente a la arquitectura debido a que separa las responsabilidades a nivel de código. A continuación se introduce la estructura de MVC aplicando esta arquitectura:

- **Modelo:** por un lado representa el conjunto de propiedades generales y específicas del sistema y por otro lado, todo el procesamiento interno en cuanto a la delegación de responsabilidades y lógica recogidas en los directorios `core/` y `plugins/` respectivamente. El primero dictamina qué plugins van a orquestrar las acciones recogidas y cuáles son las partes del modelo de datos que ha de entregar para que funcione, aislando por completo dicha responsabilidad al segundo, donde los plugins se encargarán del motor de procesamiento matemático del sonido espacial y la separación por pistas con AI. Esto permite aislar los procesos, puesto que cada plugin tiene la responsabilidad de otorgar la información que se pide según los requisitos del sistema de forma independiente.
- **Vista:** representa la capa de presentación al usuario recogida en el directorio `ui/`. La vista se encargará exclusivamente de dibujar los paneles y la interfaz gráfica dentro del motor 3D, sin poseer conocimiento sobre el modelo y los plugins.
- **Controlador:** representa la capa de operadores que se encargan de establecer un puente entre las propiedades que se muestran en `ui/`, contenidos en el directorio `operators/`. Estos se encargan de capturar los eventos de los botones de la interfaz e invocar los procesos de `core/` necesarios para ejecutar su acción.

5.2. Diseño lógico

La siguiente sección incluye los diagramas que representan la distribución lógica y de clases según la arquitectura microkernel y el patrón MVC escogidos para el sistema de

³Característica de una arquitectura que se da cuando la forma del problema o del ecosistema coincide directamente con la topología del estilo arquitectónico elegido.

reproducción. El primer apartado muestra y discute la estructura de ficheros.

5.2.1. Diagrama de paquetes

El *Diagrama de Paquetes* (DP) para el sistema de reproducción es el siguiente:

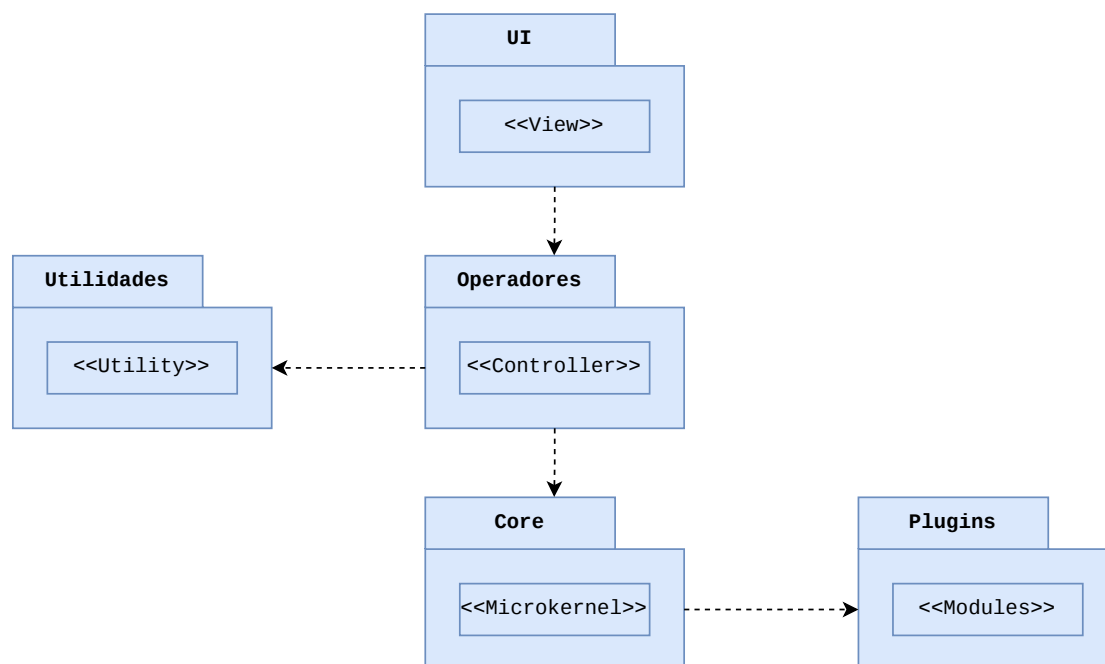


Figura 5.1: DP del sistema de reproducción 8D

El DP recoge lo parcialmente descrito en el capítulo anterior al presentar la arquitectura y el patrón MVC. Los directorios se estructuran en una serie de carpetas que recogen la UI (*Vista* → **View**), el conjunto de controladores (*Operadores* → **Controller**) que establecen un puente entre la interfaz, funciones auxiliares o (*Utilidades* → **Utility**) y el (*Core* → **Microkernel**). Este último se encargaría de ejecutar los (*Plugins* → **Modules**). En resumen, todas ellas tienen el propósito de comunicar la acción al módulo correspondiente aislando el proceso de transmisión de datos lo máximo posible. La última parte a comprender del DP corresponde al modelo, el cual está implementado en *Core* actuando como orquestador para las tareas que realizarán los *Plugins*.

Por el contrario, a pesar de que la arquitectura donde el *Core* y los *Plugins* pudieran crecer demasiado restando el factor de responsabilidad única para dicha clase, el conjunto organizativo permitiría poder realizar mejoras o cambios con relativa facilidad, al tener un flujo estructural fácil de comprender.

5.2.2. Diagrama de clases

El *Diagrama de Clases* (DC) de la figura 5.2 no es más que una extensión del DCC presentado en el apartado 4.3, incluido a continuación:

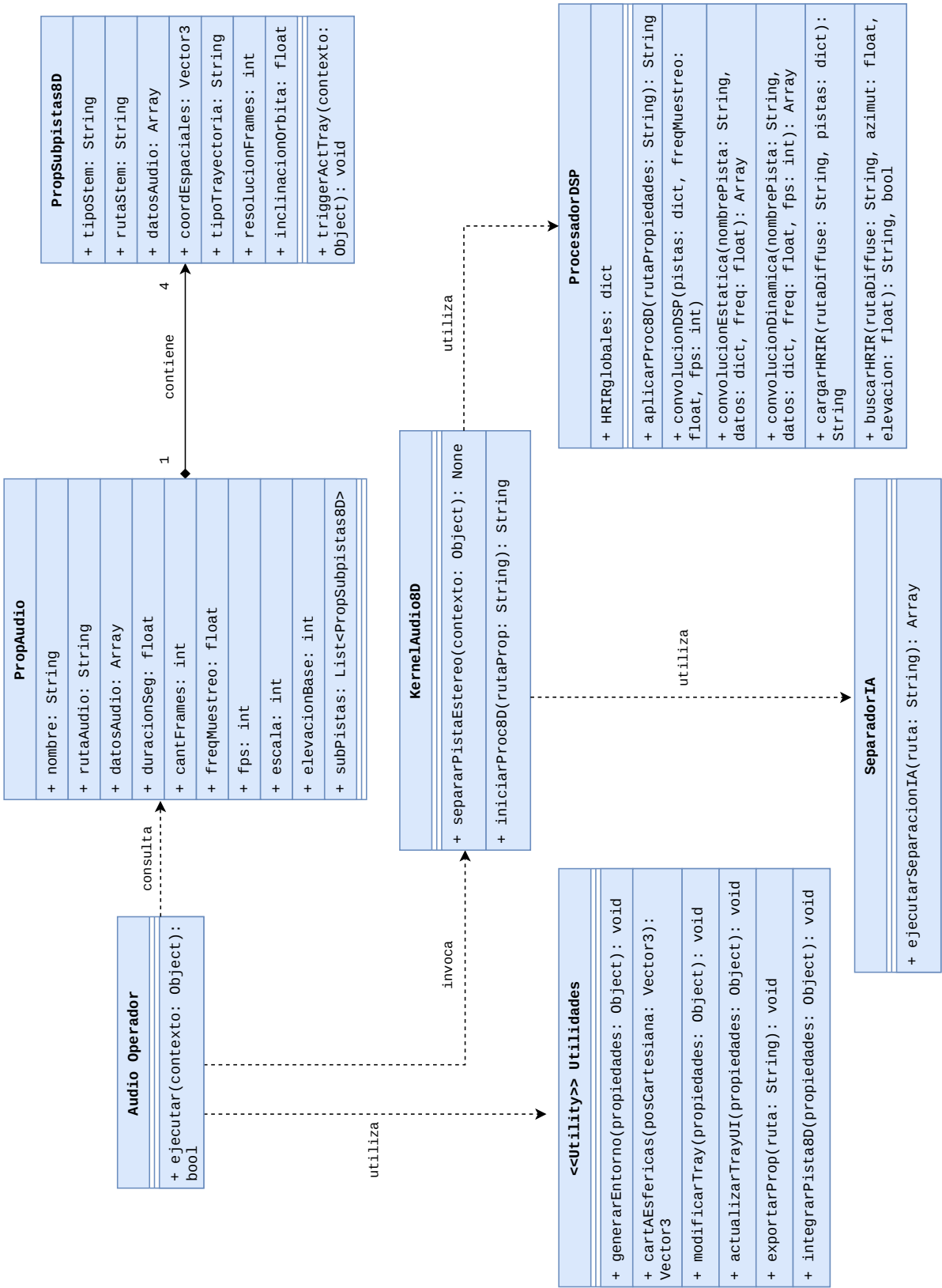


Figura 5.2: Diagrama de clases del sistema.

En primer lugar, el DC responde a la agrupación de varias clases conceptuales del DCC. La primera es **PropAudio**, que une las propiedades de la escena 3D y las propiedades de la pista que va a reproducir, sin tener ningún método. Esta decisión se justifica por la necesidad de tener una clase que represente al modelo y contenga las propiedades básicas de la escena 3D y la pista base.

De forma consecuente, **PropSubpistas8D** une las subpistas con las propiedades posicionales de su trayectoria. Esta unión se fundamenta debido a que son las subpistas, y no la pista principal, las que interactúan de forma directa con la convolución y aplicación de una HRTF. Son estas las que tienen las coordenadas esféricas de posición y el resto de propiedades de composición de trayectorias, como por ejemplo el atributo *inclinacionOrbita*. El único método utilizado por el modelo es un trigger para actualizar constantemente su trayectoria, teniendo una relación de forma indirecta en el diagrama, pero sí a nivel de código y con la interfaz. Estas dos clases funcionan principalmente con rutas simbólicas (definidas por *rutaAudio* y *rutaStem* debido a su intención modular extrapolable a otros motores 3D). En conjunto, ambas trabajan para dar toda la información necesaria de la pista base que se quiere convertir en una pista 8D.

La siguiente clase es **Utilidades**, clasificada como «Utility», una clase de apoyo al sistema. A pesar de manejar atributos intermedios, estos no se reflejan porque son los de las clases anteriores con los que verdaderamente trata el sistema. Al ser una clase de apoyo, solo tiene métodos. Estos comprenden muchas de las funcionalidades indispensables, como la conversión del sistema cartesiano a esférico, la generación y actualización de la escena 3D y la exportación de propiedades para el manejo del módulo DSP. Existe la posibilidad de extender la clase debido a la inclusión de más métodos que apoyen el desarrollo del sistema, siendo indispensables los descritos en su clase.

La clase **Audio Operador** describe la clase controladora del patrón MVC para consultar las propiedades de las dos primeras clases descritas y la utilización de los métodos de apoyo para mandar a ejecutar el módulo correspondiente. Sirve como puente entre la petición del usuario y la ejecución del sistema, dependiendo del kernel para ejecutarse.

La primera clase representativa de un módulo de la arquitectura microkernel es **SeparadorIA**. Esta recoge el contexto de la escena para conocer la ruta de la pista base actual y devolver los datos del audio y las rutas de las subpistas separadas con AI. Estas rutas son las mismas que las pertenecientes a las clases **PropSubpistas8D**, representadas como vocales, batería, bajo y otros.

La segunda clase representativa de un módulo, **ProcesadorDSP**, y la más importante, es la que realiza todo el proceso de convolución. Esta recoge todo el modelo conocido gracias a *rutaPropiedades* para iniciar el procesamiento DSP. En ella se agrupan las clases HRIR, BDKemar y la pista binaural final. Esta aplica dos procedimientos: uno de búsqueda y agrupación de HRIR, y otro de aplicación de la convolución. El primero justifica la aparición de los métodos *cargarHRIR* y *buscarHRIR*, junto con un diccionario *HRIRglobal* que relaciona una coordenada esférica con su HRIR a causa de las características de la base de datos elegida. Esta obliga en cierta medida a buscar, cargar y cambiar las propiedades de los impulsos. El segundo procedimiento corresponde a la obtención de la pista binaural cuando ya se tienen todos los datos necesarios. Principalmente, ejecuta una convolución dinámica o estática para todas las **PropSubpistas8D** según su *tipoTrayectoria*. El módulo devuelve la ruta con la pista binaural masterizada y lista para exportar a la escena.

La última clase, **KernelAudio8D**, representa el core según la arquitectura microkernel. Esta se encarga de aislar la lógica sobre la construcción de la escena 3D y de evitar que

los fallos de los módulos se extiendan al resto de las clases. En consecuencia, al igual que el operador necesita al kernel para funcionar, el kernel necesitará sus módulos.

El resto de propiedades y métodos para las clases por ahora no se desarrollarán debido a que están mejor comprendidos en el capítulo 6.

5.3. Diseño de procesos

La siguiente sección recoge el conjunto de apartados de diseño que se centran en guiar al usuario por la interfaz del sistema de reproducción, el flujo principal del sistema y el procesamiento de la DSP.

5.3.1. Mockup de la UI

Con el objetivo de representar el orden propuesto por el DSC se necesita construir una UI y un flujo visual con el que el usuario pueda establecer rápidamente cuáles son los procesos que ha de ejecutar y en qué orden. Conociendo esta cuestión, la UI propuesta es la siguiente:

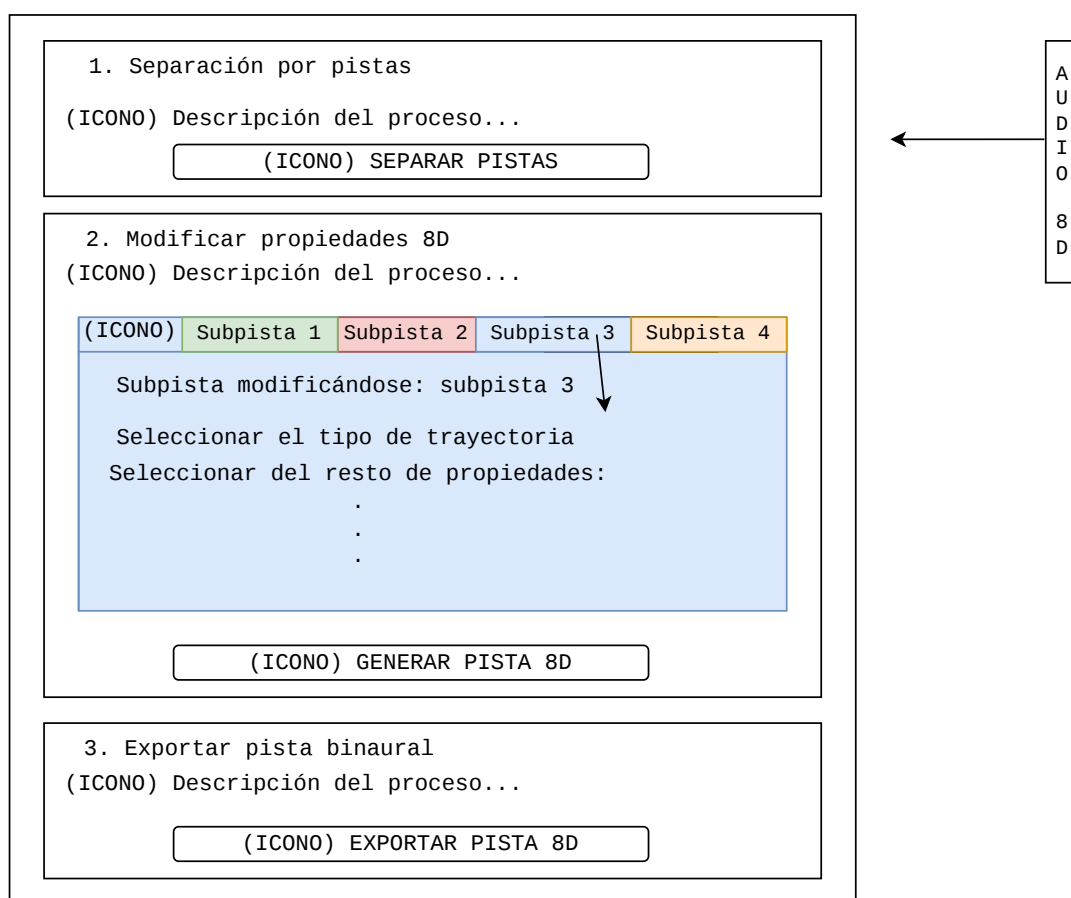


Figura 5.3: Mockup UI del sistema de reproducción 8D

La UI propuesta nace como un panel incrustado en la sección derecha de la vista de Blender. Al desplegar el panel *Audio8D* se tienen tres paneles que representan los tres procesos descritos en el DSC. Cada uno de estos paneles tiene un identificador numérico para indicar el paso que se está siguiendo, un título, una descripción y un botón al final que ejecuta su acción. Los iconos seleccionados ayudan a comprender la acción que realiza y así reducir la carga cognitiva de texto en la UI. El segundo panel representa el CU-02 y CU-03. Para tratar la modificación de trayectorias, este tendrá un subpanel que permitirá elegir que propiedades se aplican a qué subpista. Cada una de las subpistas tiene un icono que permite relacionar visualmente qué pista se está modificando. Para acompañar esta decisión, el panel de modificaciones, común a todas las subpistas, tendrá el mismo color que la pista que se está modificando. Tanto el color de la interfaz como la de los elementos que representen a las subpistas en la escena 3D, será común, aportando coherencia a la UI. El diseño también incluye una serie de mensajes para dar a conocer al usuario qué paso tiene que realizar en ese momento, evitando futuros errores y desbloqueando la interfaz verticalmente a medida que se utiliza el sistema.

5.3.2. Diagrama de actividad del CU-02

El *Diagrama de Actividad* (DA) del CU-02 que muestra el paso crucial entre una pista estéreo a una pista 8D está incluido en la figura **A.1** del apéndice a efectos de maquetado y estilo.

El diagrama de actividad destaca por separar las competencias del operador, kernel y el procesador. Con esto se pretende encapsular las responsabilidades cuando se trata de trasladar las acciones entre los implicados en la ejecución. Entonces, el **operador** únicamente registra la acción del usuario en su estado inicial y prepara los datos mínimos con los que deberá operar el plugin al que llame, sin tener conocimiento en ningún momento del procedimiento para el que fue asignado. Después, el **kernel** orquesta qué plugin tiene que actuar frente al operador que le llama, entregando los datos necesarios para la ejecución. El **procesadorDSP** obtiene la información que le entrega el kernel y procede a ejecutar su cadena DSP. La separación de estas responsabilidades recae en los RF y RNF del CU, principalmente el RNF-05, con el objetivo de aislar lo máximo posible sus procesos.

Cabe mencionar que el DA del CU-02 ha sido diseñado en base a varios de los objetivos descritos en el capítulo 1.3, específicamente la posibilidad de poder integrarse con motores externos al sistema de reproducción. Sin embargo, este posible CU generado a partir de un objetivo no tiene sentido que sea registrado pues no comprende al desarrollo del proyecto, pero sí se ha de orientar a los fines de consecución del mismo como se ha querido extrapolar.

5.3.3. Diagrama de secuencia general

El *Diagrama de Secuencia General* (DSG) de la figura 5.4 que muestra cómo tiene que ser el comportamiento del sistema frente a las peticiones del usuario es el siguiente:

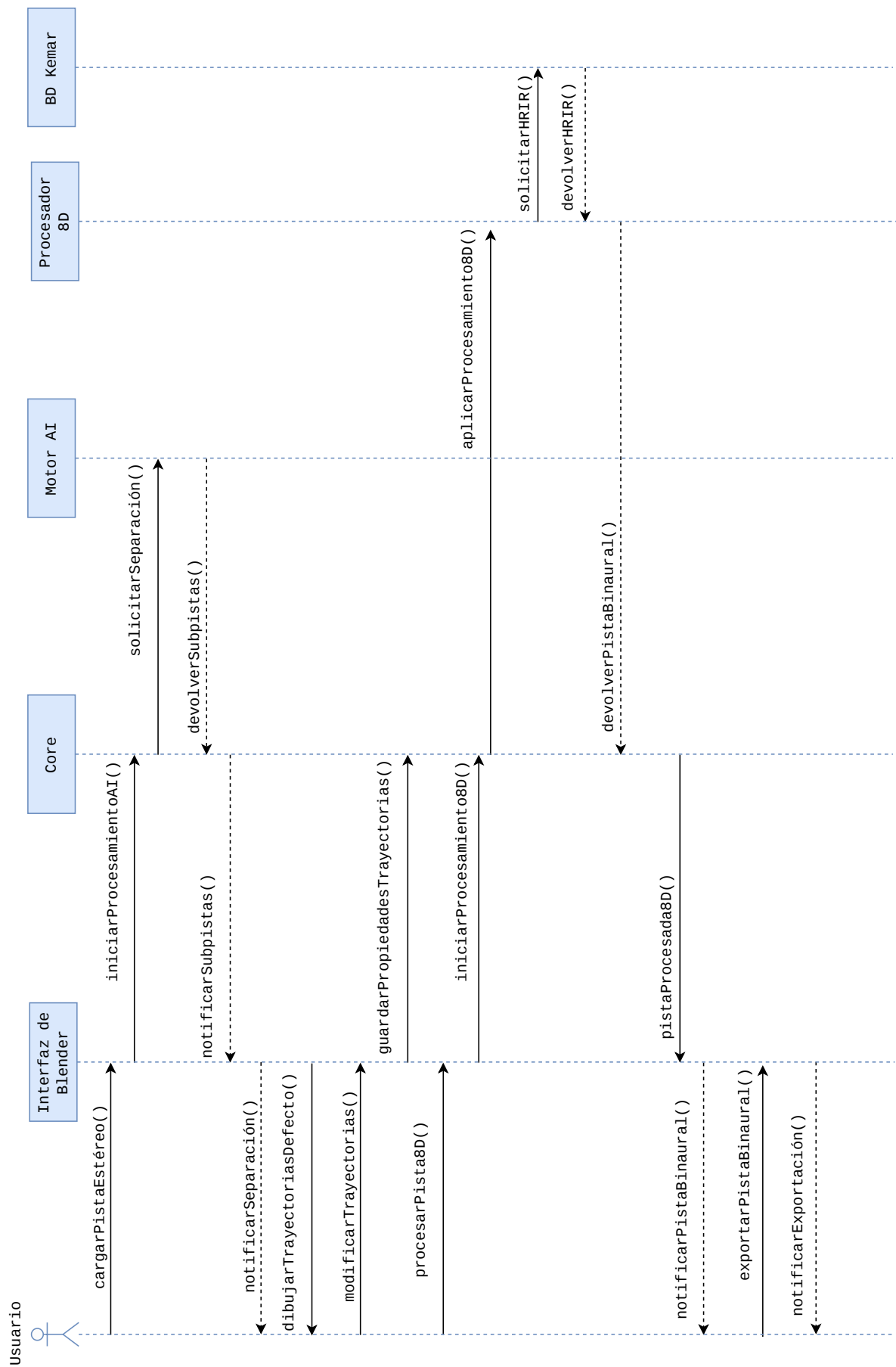


Figura 5.4: Diagrama de secuencia general del sistema.

Al igual que el DC es una evolución del DCC, lo mismo sucede con el DSG para el DSC. El diagrama se extiende para el usuario haciendo que entable relación con la interfaz de blender. Esta interfaz (que incluye a los operadores), es la que responde a los tres procesos del DSC y a las actualizaciones en la modificación de las propiedades binaurales de las subpistas. Entre las peticiones de la *Interfaz de Usuario* y los plugins, está el *Core* de la arquitectura, orquestando y delegando las acciones hacia los módulos *Motor AI* con `cargarPistaEstéreo()` y *Procesador 8D* con `procesarPista8D()`, actuando como puente. Ambos módulos lo notifican por si existe algún error y a su vez, a la interfaz, para que el usuario reciba una respuesta. Respecto al procesamiento de la pista binaural, el DSG se extiende, incluyendo la *BD Kemar* para recoger las HRIR con las que otorgará las propiedades posicionales.

Algunos de los métodos que no dependen de los módulos conectados al kernel son los que serán ejecutados por la clase *Utilities* incluida en el DC. Sin embargo, para que el sistema funcione mínimamente, tendrán que aparecer los siguientes: `modificarTrayectorias()`, `dibujarTrayectoriasDefecto()` y `exportarPistaBinaural()`. Estos métodos son los más importantes dentro del conjunto de funciones auxiliares y construyen el proceso base sobre el que el sistema puede operar mínimamente. El resto de métodos auxiliares no están reconocidos debido a que permiten controlar la asignación de las rutas entre otras necesidades operativas. Cada uno de los procesos entre dos clases de la secuencia informa sobre el estado final de su acción, permitiendo capturar el error y aislar los procesos lo máximo posible.

5.3.4. Diseño de la DSP

El diseño de la DSP es en parte interpretado por el bloque de actividad del *Procesador DSP* del diagrama A.1. El diseño incluye únicamente todo lo referido al tratamiento del audio de las subpistas, la obtención y carga de las HRIR y la mezcla de todas las subpistas finales para la pista binaural. La figura 5.5 con el DA de la cadena DSP es el siguiente:

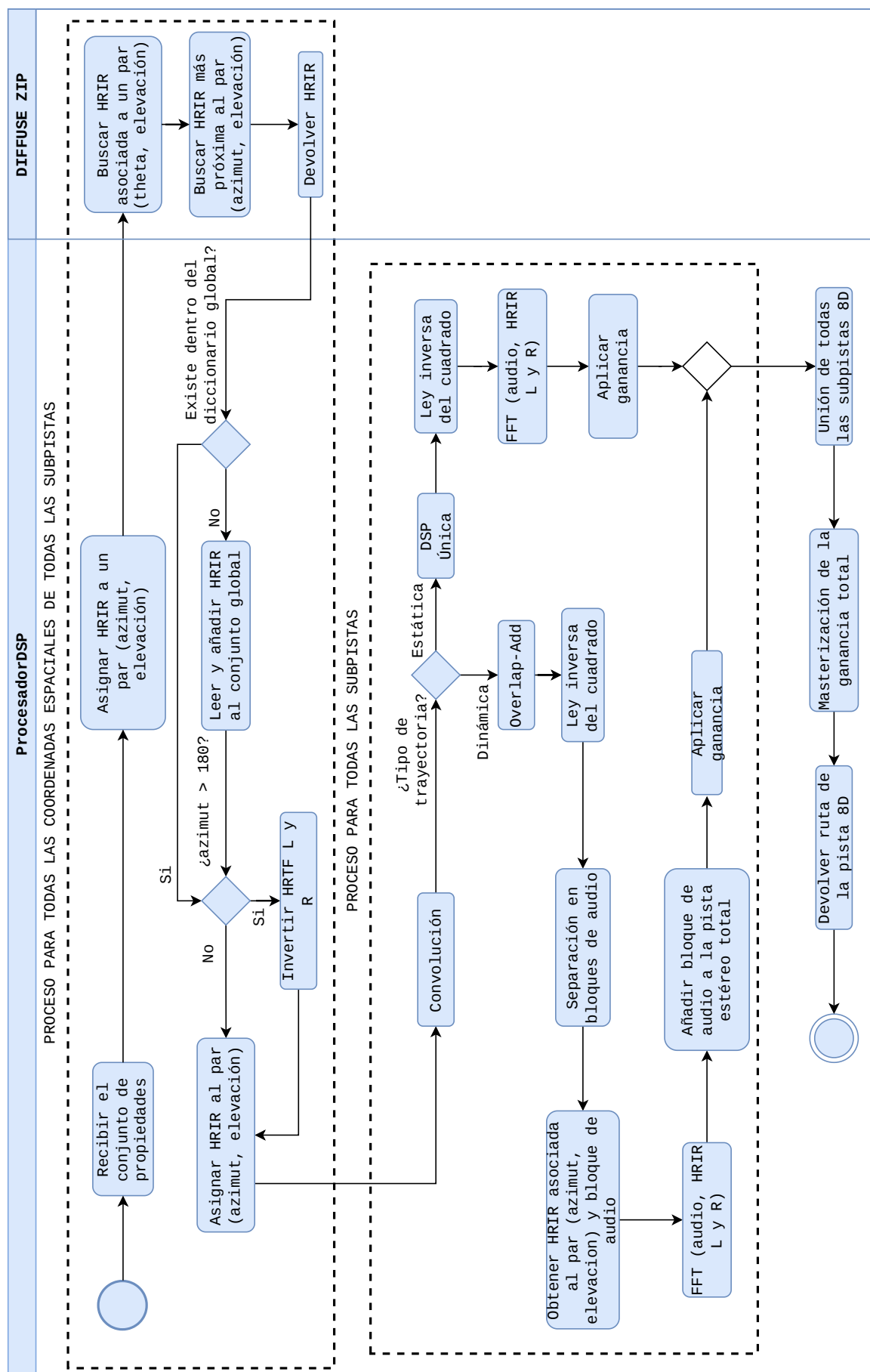


Figura 5.5: Diagrama de actividad del procesador DSP.

Partiendo de la base en la que se tienen todos los datos necesarios para aplicar la DSP y que el procesado es offline y no en tiempo real, el diseño planteado se divide en dos subprocesos: el preprocesamiento de las HRIR y la convolución. La primera de ellas está orientada a optimizar las operaciones de lectura de los impulsos. Para ello, se tiene que conocer cómo está estructurado el conjunto de HRIR que empleará el sistema.

Entre los sets HRIR medidos por Gardner y Martin (1994) se encuentran: `full.zip`, `compact.zip`, `diffuse.zip` y `diffuse32K.zip`. La selección se basa en cuánto de preparadas están para optimizar el tiempo de cómputo y así poder cumplir varios RNF. `full.zip` contiene el conjunto crudo de datos sin post procesar, introduciendo el ruido producido por los micrófonos usados en la captación. `compact.zip` está pensada como un formato comprimido donde la duración de las HRIR es menor en comparación con el resto, reduciendo la calidad de la huella posicional en el sonido a la que se aplica. `diffuse.zip` tiene un tamaño de respuesta igual a `full.zip`, aparte, está ecualizado para introducir una respuesta plana para cualquier auricular. `diffuse32K.zip` es igual que la anterior solo que su frecuencia de muestreo es de $32kHz$, quedando descartada por no cumplir el RNF-04. En conclusión, el conjunto que mejor se aproxima a las necesidades del reproductor es `diffuse.zip`. Por un lado, respeta el RNF-04 y entrega una respuesta al impulso lo suficientemente larga como para añadir una huella espacial notable, a su vez, está optimizada según el modelo esférico en el que se basa el reproductor. La característica más importante de este set es que el δ está limitado a $[-40^\circ, 90^\circ]$, lo cual imposibilita posicionar pistas por debajo del usuario. Los valores azimutales llegan hasta 180° , cuestión que es fácil de resolver sabiendo que el sistema propuesto es simétrico en la forma de ambos oídos. El factor limitante de δ es uno de los mayores inconvenientes cuando se utiliza el set KEMAR, restando propiedades binaurales.

El segundo subproceso comienza obteniendo el factor de ganancia que se le aplicará a la subpista utilizando la regla inversa del cuadrado. Después se obtendrá el audio convolucionado. En este punto se diferencia entre una convolución estática o dinámica, debido a que su procedimiento no es el mismo. La convolución estática, aplicada a las subpistas con una trayectoria del mismo tipo, utiliza una sola HRIR para toda la pista. La convolución dinámica, al tener que interpolar varias HRIR, utilizará *overlap-add* ya que es más sencillo de comprender que *overlap-save*. La estática no se tiene que centrar en entender particiones de audio porque la subpista ya actúa como un único bloque, aplicando la FFT entera con la librería *scipy*. La dinámica tendrá que particionar la canción en un número de bloques igual a la cantidad de coordenadas (θ, δ) , teniendo en cuenta que la HRIR introduce una reflexión a cada bloque final al tener la misma longitud en muestras. Esto permite que el procedimiento se sincronice directamente con la posición que tenga la subpista en la escena 3D. El método *overlap-add* describe que los bloques de audio se sumarán en una pista estéreo, superponiendo una parte de su HRIR, interpolando las respuestas sin producir cambios audibles entre el cambio de huellas espaciales. Al finalizar se aplicará la ganancia al conjunto final. Por último, puesto que la distancia entre el usuario y la subpista no cambia, la ganancia es constante para toda la subpista. Para ambos procedimientos se aplicará su $HRIR_L$ y $HRIR_R$, respetando el aislamiento entre los canales, para reconstruirse nuevamente.

Tras acabar el proceso de integración de la huella binaural, se normaliza la ganancia total para que la pista 8D tenga un volumen lo más acorde posible a la pista original y el factor r . Este último proceso responde a la etapa final de masterización de la pista 8D, respetando la pista original y terminando con la DSP.

5.4. Diseño de datos

El modelo de datos recoge principalmente los atributos de las clases **PropAudio**, **PropSubpistas8D** y **ProcesadorDSP**. Los datos más importantes son los archivos de audio, tanto de las pistas como de las HRIR, los cuales se tratan como rutas **String** que apuntan a los propios **Arrays** de audio. Esta decisión otorga mucha más libertad a la hora de manejar la información y elimina la dependencia del usuario a tener que adaptarse a un uso del sistema de forma específica. Como ya se mencionó anteriormente, la clase **PropAudio** contiene la lista de las cuatro subpistas en forma de **<List>**, realizando el hecho de que todas están sujetas a las propiedades de la escena 3D. El modelo también contiene variables de control que permitirán establecer las precondiciones para utilizar el sistema en el orden esperado e indicar al sistema qué subpista está recibiendo modificaciones en formato **Bool**. Por último, esta clase contiene las variables **escala** y **elevacionBase** con las que situar los elementos de la escena y ajustar el aspecto de visualización cómodamente.

Con respecto a las subpistas, la trayectoria que acompaña a cada una de **PropSubpistas8D** se define a través de un vector con la posición (r, θ, δ) que indica una posición cualquiera de la subpista. Para las pistas dinámicas se tiene una variable **tipoTrayectoria** tipo **String** que determina si esa subpista es estática o no. Es de tipo **String** debido a que existe más de un tipo de trayectoria dinámica. **resolucionFrames**, utilizado únicamente para la convolución dinámica, afecta directamente al tamaño del bloque de audio al darnos la cantidad de puntos que hay en un único segundo, siendo su rango $[1, 60]$. A mayor resolución, más corto es el bloque, aumentando la interpolación de las HRIR. Esta variable afecta directamente al rendimiento del procesador.

El enfoque principal que justifica muchos de los datos del modelo es la sincronización visual entre las trayectorias y la escena. Las variables **fps**, **freqMuestreo**, **duracionSeg**, **cantFrames** y **resolucionFrames** son las que resuelven este problema. En primer lugar, se ajusta la canción a una cantidad de frames con $cantFrames = duracionSegundos \cdot fps$. Con la duración de la canción base en frames, se obtiene el número de muestras que tendrá un bloque de audio, ajustándose siempre a la duración anterior y la resolución espacial con $nMuestrasBloque = cantFrames \cdot 1 \div resolucionFrames$. La resolución actúa como elongador del tamaño del bloque. Entonces, conocido el bloque, la cantidad de estos viene determinada por $nBloques = longitudAudioFrames \div nMuestrasBloque$. Con esto se ajusta toda la trayectoria a la cantidad de frames determinada por la canción y el número de bloques del procesamiento dinámico. Por último, este diseño de datos se ha configurado para que las subpistas den una única vuelta desde su posición de origen, por ello el ajuste de segundos a frames y frames a bloques de audio.

5.5. Documentación para addons en Blender

La última fase de diseño comprende una breve explicación sobre las directrices que proporciona la documentación de la API de Blender en Blender Foundation (2026) y que serán de obligado cumplimiento en el diseño. Si se analiza lo que ejemplifica el mockup 5.3.1 y el DP 5.1, los elementos a proporcionar por Blender serán la estructura de inicialización del proyecto (incluyendo el instalador), elementos de construcción de UI y los operadores.

La inicialización del proyecto comienza con la declaración de una estructura **blInfo** que

indique los metadatos del proyecto como el nombre, versión del addon, dónde se ubica... Eso sí, no hace falta que incluya todas los parámetros. Todo ello registrando y desregistrando todos los elementos que la conforman:

```
bl_info = {
    "name": "Example Add-on Preferences",
    "author": "Your Name Here",
    "version": (1, 0),
    "blender": (2, 65, 0),
    "location": "SpaceBar Search -> Add-on Preferences Example",
    "description": "Example Add-on",
    "warning": "",
    "doc_url": "",
    "tracker_url": "",
    "category": "Object",
}

# Código incrustado...

import bpy

# Registration
def register():
    bpy.utils.register_class(OBJECT_OT_addon_prefs_example)
    bpy.utils.register_class(ExampleAddonPreferences)

def unregister():
    bpy.utils.unregister_class(OBJECT_OT_addon_prefs_example)
    bpy.utils.unregister_class(ExampleAddonPreferences)
```

Código 5.1: Estructura de inicialización y metadatos de un addon en Blender

Nota. Adaptado de *Blender Python API documentation*, por Blender Foundation (2026).

En cuanto a los paneles, estos se declararán con la siguiente estructura:

```
import bpy

class HelloWorldPanel(bpy.types.Panel):
    bl_idname = "OBJECT_PT_hello_world"
    bl_label = "Hello World"
    bl_space_type = 'PROPERTIES'
    bl_region_type = 'WINDOW'
    bl_context = "object"

    def draw(self, context):
        self.layout.label(text="Hello World")

bpy.utils.register_class(HelloWorldPanel)
```

Código 5.2: Creación de un panel básico utilizando la API de Blender

Nota. Tomado de *Blender Python API documentation*, por Blender Foundation (2026).

Los operadores tendrán que ser declarados de la siguiente manera:

```
import bpy

class HelloWorldOperator(bpy.types.Operator):
    bl_idname = "wm.hello_world"
    bl_label = "Minimal Operator"

    def execute(self, context):
        print("Hello World")
        return {'FINISHED'}

# Registro de la clase en el sistema
bpy.utils.register_class(HelloWorldOperator)
```

Código 5.3: Creación de un operador básico utilizando la API de Blender

Nota. Adaptado de *Blender Python API documentation*, por Blender Foundation (2026).

Con esta última aportación de diseño, todo el capítulo ha justificado y explicado como se ha procedido a crear la arquitectura lógica y algorítmica. A continuación, se realizará la descripción del proceso de implementación del reproductor.

Capítulo 6

Implementación

En este capítulo se describe la integración e implementación de la arquitectura general del reproductor y de los tres incrementos estimados en el apartado metodológico del desarrollo. Los apartados para cada incremento incluyen la justificación de su verificación, detallando la resolución de los problemas más relevantes del mismo.

6.1. Incremento 1: Arquitectura base y módulo de IA

Para la implementación del primer incremento, se ha establecido el contenido del archivo `properties.py`, del directorio `core/`, varias de las funciones del `utils/` y varios elementos de las carpetas `ui/` y `operators/`.

La arquitectura base comienza con la inicialización de las dependencias y las conexiones mínimas necesarias, iterando por las versiones de las librerías que mejor se ajustan a las tareas del reproductor. A través de la investigación de las diferentes librerías, se establece una verificación como método de contingencia para invocar los directorios que contienen la arquitectura base. Por el contrario, si el usuario no hubiera descargado las dependencias del *addon* antes, se incluye un operador para iniciar la descarga. Este procedimiento se realiza mediante un `wheel`, la forma estándar de distribuir módulos de Python, quedando así el `init` principal del *addon*.

```

__INIT.PY__
if dependencias.MODULES_AVAILABLE:
    from .core import properties
    from . import operators
    from . import ui
    modules = (
        dependencias,
        properties,
        operators,
        ui,
    )
else:
    modules = (
        dependencias,
    )

# Registro y desregistro de todas las clases del addon
def register():
    """Registra todos los modulos del addon"""
    for module in modules:
        if hasattr(module, "register"):
            module.register()

def unregister():
    """Desregistra todos los modulos del addon"""
    for module in modules:
        if hasattr(module, "unregister"):
            module.unregister()

if __name__ == "__main__":
    register()

#DEPENDENCIES.PY
#...
try:
    #Instalación del método que permite desempaquetar los archivos .whl
    subprocess.check_call([python_exe, "-m", "pip", "install", "wheel"])

    #Instalación de todas las librerías necesarias
    subprocess.check_call([
        python_exe, "-m", "pip", "install",
        "--no-cache-dir",
        "torch==2.1.2+cpu", "torchaudio==2.1.2+cpu",
        "demucs", "pandas", "scipy", "soundfile", "diffq",
        "--extra-index-url", "https://download.pytorch.org/whl/cpu"
    ])

except subprocess.CalledProcessError as e:
    self.report({'ERROR'}, f"Fallo en la instalación: {e}")
    return {'CANCELLED'}

self.report({'INFO'}, "Instalación completada. Reinicia Blender.")
return {'FINISHED'}

```

Código 6.1: Instalación base del addon según sus dependencias

Nota. Elaboración propia.

Con ello quedan iniciados y disponibles el resto de directorios con los que se empezará a construir el reproductor. Tanto los operadores como los paneles declarados se registran en su respectivo directorio `operators` y `ui`, al igual que el archivo de propiedades de la carpeta `core/`.

Lo primero que se ha realizado a efectos reales en el reproductor es definir el modelo, el cual servirá para enmarcar el proyecto bajo las condiciones de todas las variables que lo conforman. El modelo construido queda así:

```
#Trigger para actualizar las propiedades que se necesiten
def trigger_actualizacion_trayectoria(self, context):
    generacion_trayectorias.actualizar_trayectoria_ui(self, context)

#Propiedades de las subpistas
class PropiedadesSubpistas(bpy.types.PropertyGroup):
    ruta_stem: bpy.props.StringProperty(name="Ruta", subtype='FILE_PATH')
    audio: bpy.props.PointerProperty(name="Audio", type=bpy.types.Sound)

    distancia: bpy.props.FloatProperty(name="Distancia (r)", default=1.0, min
        =0.1, update=trigger_actualizacion_trayectoria)

    azimut: bpy.props.FloatProperty(name="Azimut (Theta)", default=0.0, min=0.0,
        max=359.0, step=500, update=trigger_actualizacion_trayectoria)

    elevacion: bpy.props.FloatProperty(name="Elevación (Delta)", default=0.0,
        min=-90.0, max=90.0, step=500, update=trigger_actualizacion_trayectoria
    )

    #Resto de propiedades de las subpistas...

class Audio8DProperties(bpy.types.PropertyGroup):
    #Metadatos de la pista base
    nombre_original: bpy.props.StringProperty(name="Nombre pista original",
        subtype="FILE_NAME")
    ruta_original: bpy.props.StringProperty(name="Pista original", subtype='
        FILE_PATH')

    #Modelo de datos de cada una de las subpistas
    vocals: bpy.props.PointerProperty(type=PropiedadesSubpistas)
    drums: bpy.props.PointerProperty(type=PropiedadesSubpistas)
    bass: bpy.props.PointerProperty(type=PropiedadesSubpistas)
    other: bpy.props.PointerProperty(type=PropiedadesSubpistas)

    #Resto de datos del entorno/pista base...
```

Código 6.2: Modelo de datos del sistema

Nota. Elaboración propia.

Lo más importante aquí es cómo se establece la relación de agregación diseñada, donde la clase contenedora de las propiedades generales incluye una copia de la clase de cada subpista. El modelo incluye un único método, `trigger`, que actualizará el sistema de trayectorias del reproductor a medida que el usuario maneja la interfaz.

Tras implementar el modelo, se han inicializado los tres paneles y operadores en base

al mockup de la figura 5.3. Todos se registran/desregistran como los directorios a los que hace referencia el `init` que inicia el proyecto. El bloque de código que representa la inicialización de los paneles está incluido en el bloque **A.1** del apéndice a efectos de maquetado y estilo.

Los paneles añaden los elementos de selección de propiedades, botones e iconos a través de un `layout` general utilizando un sistema de `box`, `rows` y `columns`, ejemplificados en el bloque **A.2** del apéndice. Los operadores contienen una clase auxiliar `Import/ExportHelper` que permite abrir el explorador de archivos de Blender, con ella podemos obtener las rutas de los archivos de la pista a separar y la pista 8D a exportar. `Export` utiliza una función `invoke` para establecer qué metadatos tendrá la pista binaural final, como el formato del nombre y extensión `.WAV`. Tras la maquetación de todos los paneles, la UI final del reproductor es la siguiente:

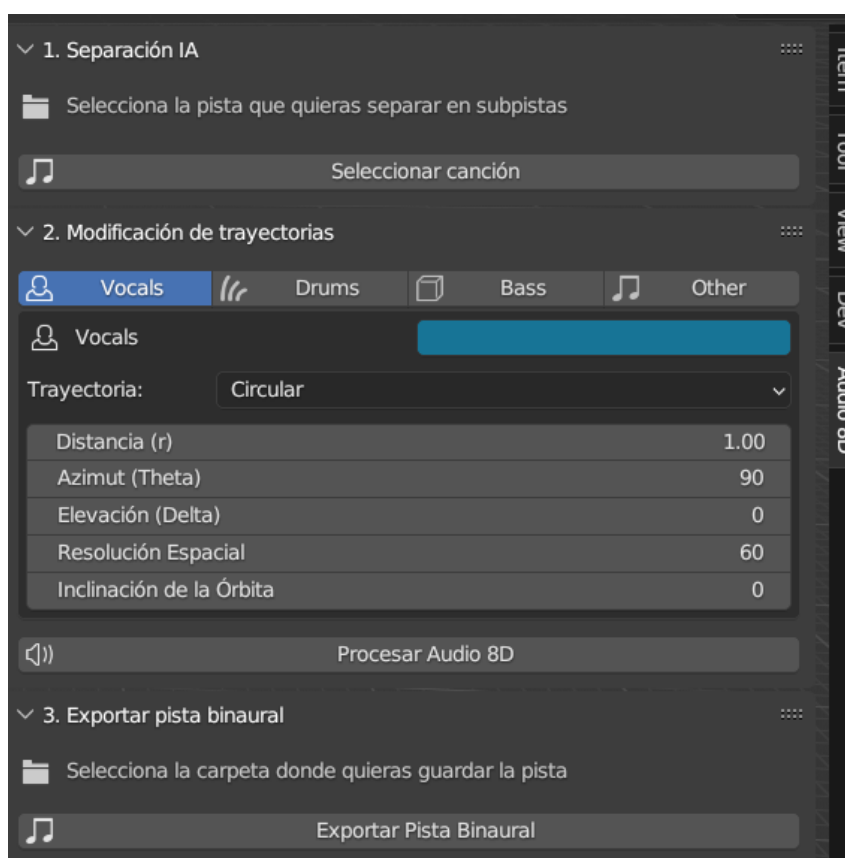


Figura 6.1: Mockup UI del sistema de reproducción 8D

Como puede apreciarse, la UI se parece bastante al mockup diseñado, contando con las varias de las características diseñadas. Con todos estos elementos, se procede a la implementación del kernel y el módulo de separación por AI.

6.1.1. Módulo de separación de pistas por AI

El módulo de separación de pistas es la primera conexión de un plugin con el kernel. Este procedimiento también se realizará para el procesador DSP, quedando de la siguiente manera:

```

#OPERADOR SEPARACIÓN IA
class AUDIO8D_OT_separar_audio(bpy.types.Operator, ImportHelper):
    #Propiedad para filtrar archivos .wav en ImportHelper
    file_filter: bpy.props.StringProperty(default="*wav", options={'HIDDEN'},
        maxlen=255, )
    def execute(self, context):
        ruta = self.filepath
        context.scene.audio8d.ruta_original = ruta
        if not ruta or not ruta.lower().endswith(".wav"):
            return {'CANCELLED'}
        #Selecciona una ruta .wav obligatoriamente, si no, aborta el proceso
        with wave.open(ruta, 'rb') as archivo_wav:
            frecuencia = archivo_wav.getframerate()
            #Obtiene y comprueba que la fMuestreo >= 44.1kHz...

        #Llamada a la operacion del kernel que carga la pista de audio
        stems = kernel.KernelAudio8D.cargarPistaEstereo(ruta)

#KERNEL
class KernelAudio8D:
    @staticmethod
    def cargarPistaEstereo(ruta_pista):
        try:
            print(f"[KERNEL]: Iniciando separación de IA para: {ruta_pista}")
            stems = separadorIA.ejecutar_separacion(ruta_pista)

            if not stems:
                raise ValueError("El plugin de Demucs no devolvió los stems
                    esperados.")

        except Exception as error_separacion:
            raise RuntimeError(f"Error interno en el procesamiento del plugin IA:
                {str(error_separacion)}")

        return stems

#PLUGIN IA
def ejecutar_separacion(ruta):
    #Proceso de separación...
    return stems

```

Código 6.3: Conexión operador IA, kernel y plugin.

Nota. Elaboración propia.

Tras seleccionar la pista y asignar sus metadatos al modelo, el cual es guardado y accedido a través del `context.scene.audio8d` de la escena, es el controlador quien determina que la ruta de la pista elegida tenga una extensión `.WAV` y una `fMuestreo` mínima de `44,1kHz`. Al tener la clase `ImportHelper`, se evita una sobreingeniería del patrón MVC siendo el controlador quién filtra la ruta del archivo. Finalmente acaba siendo recogida y trasladada del kernel al plugin, aislando el procedimiento en un archivo `.py` aparte.

El plugin de separación de AI encuentra la ruta de la canción que se quiere separar y enruta los archivos necesarios que necesita para operar. Después determina si la canción seleccionada ya ha sido separada y si es así asigna el audio de las subpistas en una

carpeta que reside en el mismo directorio que el archivo `.blend` en el que se encuentra el proyecto actual, centralizando todo el trabajo. Puesto que se puede abandonar el proyecto y continuarlo en cualquier momento o dar un error en la elección de la canción a separar, se incluye una condición de búsqueda para determinar si ya se encuentran disponibles las subpistas de la pista seleccionada, ahorrando tiempo de computo. Si no, la separación se ejecuta mediante la ejecución de `demucs` a través de un comando que especifica qué modelo se utiliza y con qué características. En este punto es donde verdaderamente se cuantifica la variación de duración del módulo, pues dependerá de las características del ordenador que lo utilice. Los apartados más importantes del módulo son los siguientes:

```
#PLUGIN SEPARACION CON IA
def ejecutar_separacion(ruta):

    #1. Encontrar la carpeta origen y el nombre de la canción
    carpeta_origen = os.path.dirname(ruta)
    nombre_cancion = os.path.splitext(os.path.basename(ruta))[0]
    #...

    #3. Enrutamiento al módulo FFMPEG que necesita demucs
    ruta_script = os.path.dirname(__file__)
    ruta_ffmpeg_bin = os.path.join(ruta_script, "bin")
    ruta_ffmpeg_exe = os.path.join(ruta_ffmpeg_bin, "ffmpeg.exe")

    if not os.path.exists(ruta_ffmpeg_exe):
        #No se puede ejecutar la separación

    #6. Ejecución del proceso y asignación de rutas a las futuras subpistas
    ruta_base_stems = os.path.join(carpeta_origen, "mdx_extra_q", nombre_cancion)
    stems = {
        "vocals": os.path.join(ruta_base_stems, "vocals.wav"),
        "drums": os.path.join(ruta_base_stems, "drums.wav"),
        "bass": os.path.join(ruta_base_stems, "bass.wav"),
        "other": os.path.join(ruta_base_stems, "other.wav")
    }

    #7. Comprobación por si la pista ya estuviera separada
    ya_procesada = all(os.path.exists(pista) for pista in stems.values())
    if ya_procesada:
        print(f"[SEPARADOR IA]: '{nombre_cancion}' ya está separada! Se recogen
              sus stems")
        return stems

    print(f"[SEPARADOR IA]: Ejecutando demucs en: {nombre_cancion}...")

    #7. Ejecución del proceso
    comando = [ruta_python, "-m", "demucs", "-n", "mdx_extra_q", "-d", "cpu", "-j", "4", "--out", carpeta_origen, ruta]
```

Código 6.4: Plugin de separacion con IA.

Nota. Elaboración propia.

`Demucs` destaca por tener dos métodos con los que procesar, mediante CPU o GPU. El procedimiento por CPU asegura que todos los ordenadores modernos puedan utilizarlo, al contrario, el proceso por GPU requiere una tarjeta gráfica externa dedicada en el ordenador. Pese a que el método por GPU reduce en gran parte el tiempo de cómputo, el plugin

operará con la máxima capacidad de rendimiento posible de la CPU. Las subpistas son guardadas en un directorio para cada canción dentro del directorio donde está contenida la escena `.blend`, sirviendo como biblioteca para las pistas ya separadas. A fin de verificar que la separación funciona, y devuelve las pistas, se ha realizado la siguiente separación para una canción del dataset **MUSDB18HQ** de Rafii et al. (2019) cambiando el nombre de la pista `mixture` por su nombre real para que el separador funcione correctamente. Estos han sido los tiempos de separación y obtención de subpistas en base al procesamiento con 2 y 4 núcleos para una canción de 3:20 minutos:

```

Librerías cargadas correctamente!
[OPERADOR IA]: Solicitando separación de: AI James - Schoolboy Facination.wav. El tiempo dependerá de las características del
ordenador
[KERNEL]: Iniciando separación de IA para: C:\Users\CARLOS RUEDA\Desktop\pruebaTFG\AI James - Schoolboy Facination.wav
[SEPARADOR IA]: Ejecutando demucs en: AI James - Schoolboy Facination...
[SEPARADOR IA] Finalizado en: 6 min y 23 seg.

2 Librerías cargadas correctamente!
[OPERADOR IA]: Solicitando separación de: AI James - Schoolboy Facination.wav. El tiempo dependerá de las características del
ordenador
[KERNEL]: Iniciando separación de IA para: C:\Users\CARLOS RUEDA\Desktop\pruebaTFG\AI James - Schoolboy Facination.wav
[SEPARADOR IA]: Ejecutando demucs en: AI James - Schoolboy Facination...
[SEPARADOR IA] Finalizado en: 5 min y 41 seg.

[OPERADOR IA]: Solicitando separación de: AI James - Schoolboy Facination.wav. El tiempo dependerá de las características del
ordenador
[KERNEL]: Iniciando separación de IA para: C:\Users\CARLOS RUEDA\Desktop\pruebaTFG\AI James - Schoolboy Facination.wav
[SEPARADOR IA]: 'AI James - Schoolboy Facination' ya está separada! Se reconocen sus stems

```

Figura 6.2: Tiempo de separación para la pista *AI James - Schoolboy Facination*

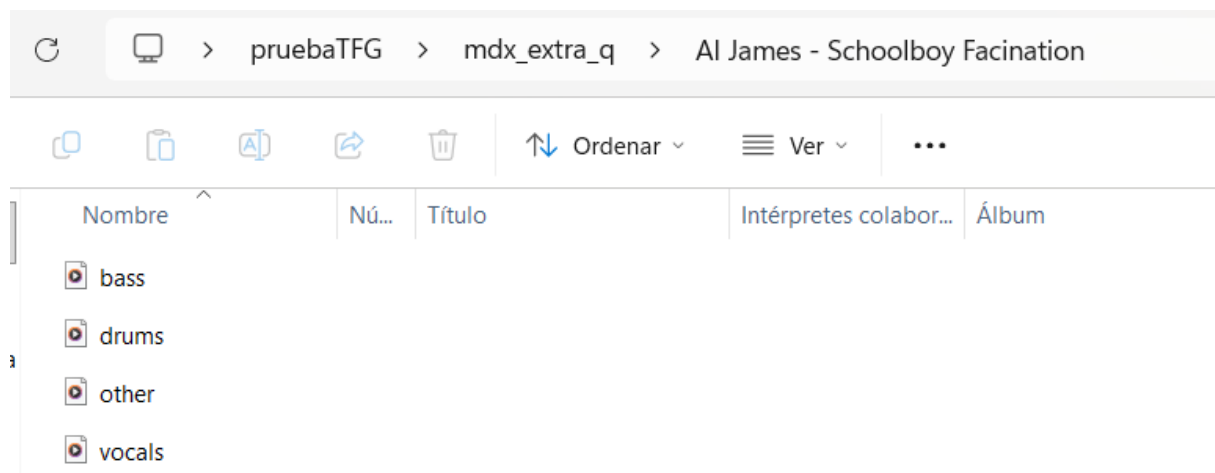


Figura 6.3: Verificación de las subpistas de *AI James - Schoolboy Facination*

Con ello, queda registrada la verificación de la separación exitosa de una canción con una calidad aceptable y recogiendo una mejoría de 39 segundos frente a la opción de separación menos óptima. Los elementos correspondientes al primer incremento quedan registrados, pasando a la implementación del segundo incremento.

6.2. Incremento 2: Escena 3D y procesador 8D

El segundo incremento es el más extenso de todos ellos a causa de representar el proceso más importante del proyecto, la generación de la pista 8D y la sincronización visual del resultado final. En primer lugar, se explicará el proceso de selección y optimización de los elementos visuales que conforman la escena hasta llegar al procesador DSP.

6.2.1. Implementación de los aspectos visuales de la escena

Para hacer entender al usuario el posicionamiento se necesita una cabeza con la que permitir al usuario orientar las subpistas correctamente. Su construcción ha tenido en cuenta las referencias de las mediciones KEMAR, las geometrías definidas en el apartado 2.1, la escala y una elevación respecto al suelo de la escena 3D para simular la altura promedio de un oyente. Esta función pertenece a una de las funciones contenidas en el directorio `utils/`. La cabeza se construye de la siguiente manera:

```
#GENERAR OYENTE.PY -> UTILS
def generar_oyente(propiedades):
    col_oyente = configurar_coleccion("Audio8D_Escena")
    escala = propiedades.escala
    z_head = propiedades.elevacion_base * escala

    #1. Cabeza del oyente
    bpy.ops.mesh.primitive_uv_sphere_add(radius=0.0875 * escala, location=(0, 0,
        z_head))
    #Se añade a la colección del oyente...

    #2. Orejas del oyente heredando a cabeza
    #Para cada oído...
    bpy.ops.mesh.primitive_uv_sphere_add(radius=0.015 * escala, location=(
        posX, 0, 0))
    oido = bpy.context.active_object
    oido.name = f"Oyente_Oido_{lado}"
    #Dependencia a la cabeza y la colección...

    #3. Nariz del oyente apuntando al frente (+Y) heredando a cabeza
    loc_y_nariz = (0.0875 + 0.02) * escala
    bpy.ops.mesh.primitive_cone_add(radius1=0.015 * escala, depth=0.04 * escala,
        location=(0, loc_y_nariz, 0))
```

Código 6.5: Generación del modelo esférico en Blender

Nota. Elaboración propia.

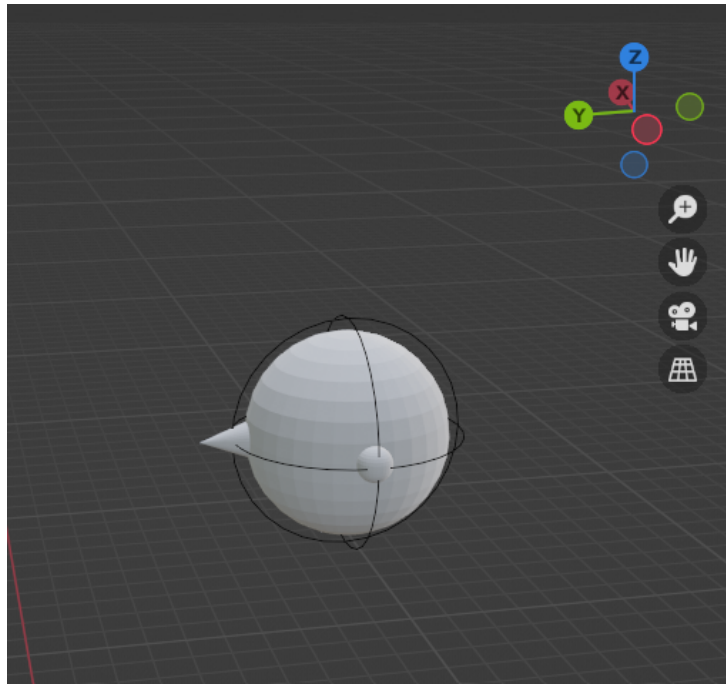


Figura 6.4: Representación de la cabeza en Blender

Al tener una esfera simétrica, el cono permite al usuario ubicar el frente correctamente. Lo siguiente será discutir cómo se han integrado los elementos que permitirán modificar las propiedades de las trayectorias dinámicas y estáticas.

Para generar los elementos visuales de las subpistas se inicializa una esfera para cada subpista según el color asignado por la propiedad `colorUi` del modelo, en relación a los colores vistos en la UI. La posición relativa de cada esfera corresponde a un pivote situado en el centro de la esfera de la cabeza, simulando el centroide de la escena 3D. El posicionamiento inicial, asumiendo una posición inicial estática cartesiana, tiene en cuenta los factores de `escala`, `elevacionBase`. La posición inicial se convertirá a coordenadas esféricas con la función `cartesianasAEsféricas` para comenzar a asignar el modelo esférico y obtener un posicionamiento inicial. El código y resultado de esta aproximación es la presentada en el código 6.6 a continuación:

```
#ENTORNO BASE.PY -> UTILS
def generar_subpistas(propiedades):
    #Posición de pistas inicial
    pistas_config = {
        "Vocals": (propiedades.vocals, (0.0, 0.5, 1.0, 1.0), (0, 1, 0)),}
    #Resto de asignaciones para las subpistas...
    #Generación y asignación de cada una de las subpistas
        props.color_ui = color
        #Inicialización de la distancia según su posición cartesiana
        distancia = props.distancia * escala
        loc_x = dirX * distancia
        #Lo mismo para el resto de coordenadas YZ...
        #Conversión del plano cartesiano al esférico.
        _, azimuth, elevacion = cartesianas_a_esfericas(dirX, dirY, dirZ)
        props.azimut = azimuth
        props.elevacion = elevacion
        #Generación de pivotes para cada escena...
        #2. Crear la Esfera de la subpista en su posición estática final
        bpy.ops.mesh.primitive_uv_sphere_add(radius=0.05 * escala, location=(
            loc_x, loc_y, loc_z))
        esfera = bpy.context.active_object
        esfera.name = f"Pista_{nombre}"
        #Asignación del color para cada esfera y asignación del pivote...
        esfera.parent = pivote
        col_oyente.objects.link(esfera)
    #...
```

Código 6.6: Inicialización de subpistas en Blender.

Nota. Elaboración propia.

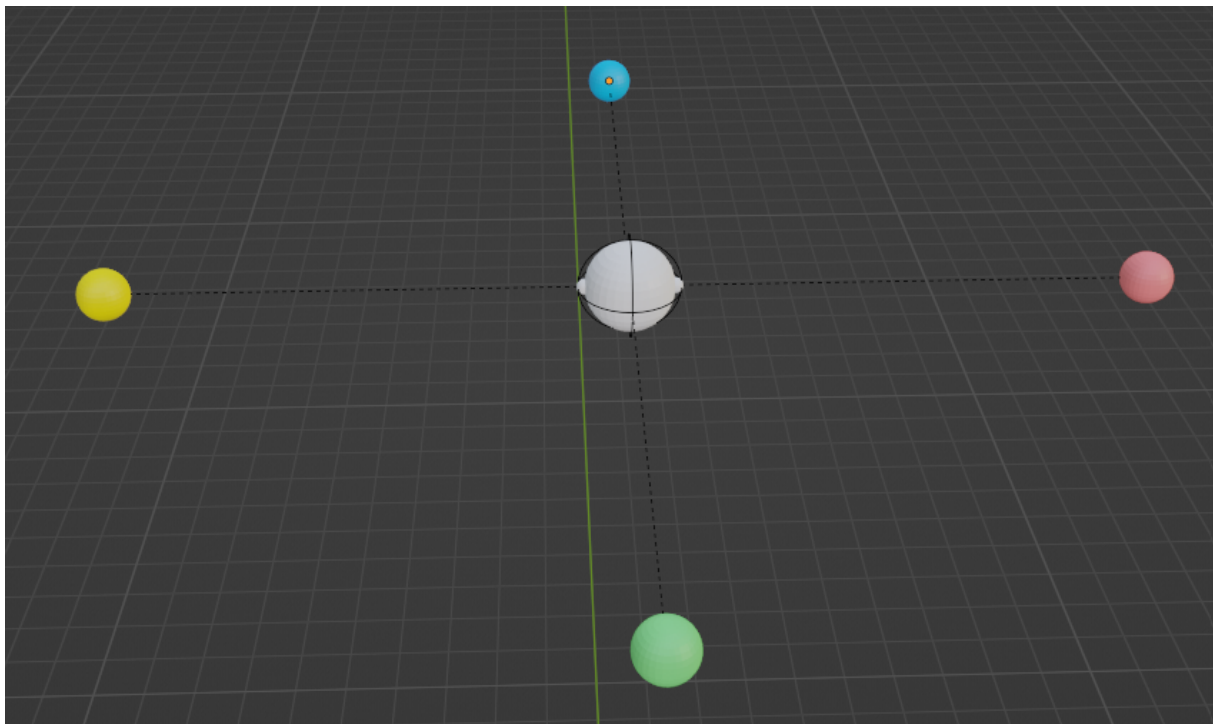


Figura 6.5: Representación de las subpistas estáticas en blender

Con las pistas inicializadas y sujetas al centroide, se utilizaron dos aproximaciones para construir el modelo visual que permita asignar y manejar la trayectoria dinámica de cada subpista. La primera consistió en asignar una colección de esferas independientes, representando la posición donde se calcularía una HRTF, que cambiase su posición dinámicamente para cada subpista al modificar los parámetros de su trayectoria. Esta fue rápidamente descartada al entender que se generaría una cantidad de vértices muy elevada. Si los cambios de la subpista quieren actualizarse visualmente en tiempo real (que no el audio, que se procesa en diferido), rápidamente la escena tendría un conjunto elevado de vértices para cada esfera que tendrían que recalcularse constantemente, y todo sin contar que hay cuatro subpistas. Es por esta razón que se optó por la búsqueda de elementos que puedan optimizar esta situación. Tras la búsqueda de elementos de la API de Blender, se llegó a la siguiente solución.

La estructura para una subpista con trayectoria dinámica es conformada por los siguientes elementos: una curva que representa el camino que seguirá la pista, un conjunto de vértices y una malla invisible. La curva responde a la geometría dictaminada por el tipo de trayectoria, circular o cuadrada, que hereda su radio desde el centroide. A esta curva se le añade la esfera con la restricción `FollowPath`, haciendo que la esfera quede anclada a la trayectoria definida por la propia curva, ahorrando cálculos manuales de interpolación entre posiciones. A esta curva, se le añade una malla que contendrá todos los vértices que representan los puntos de cálculo para el procesador basándose en `resolucionFrames`. El cálculo de los vértices de la malla es el punto más importante para la asignación de la trayectoria, siendo distinta para los dos tipos, circular y cuadrada. A partir de los valores (θ, δ) dados por `resolucionFrames` se establece un cálculo trigonométrico, con fines de computar la posición de cada vértice. Para reducir este número y que no suceda como la primera aproximación, la propia malla o `MeshInstancer` optimiza este proceso. La malla carga únicamente un icosaedro en memoria, la cual ya de por sí tiene menos vértices que una esfera, escalando y situando todos los vértices obtenidos asignados previamente. La malla carga un único icosaedro en varias posiciones sin tener que ser renderizados de forma independiente. Esta es la principal diferencia respecto a la aproximación anterior, la optimización de la carga de un único objeto pero con vértices y posiciones distintas.

La asignación de las propiedades `inclinacionOrbita`, `azimut` y `delta` influyen en la posición final de los vértices de la malla contenida en la curva. La inclinación afecta directamente al grado de inclinación del plano XY y las coordenadas esféricas definen el grado de inclinación respecto al plano en el que está contenida la curva, obteniendo el conjunto (x, y, z) . Dicho esto, parte del código que genera y actualiza el sistema de trayectorias estáticas y dinámicas del reproductor está incluido en el 6.7:

```

#GENERACIÓN DE TRAYECTORIAS ESTÁTICAS
#Datos del modelo para realizar los cálculos
radio = subpista.distancia * escala
azimut_rad = math.radians(subpista.azimut)
elevacion_rad = math.radians(subpista.elevacion)
pivote = bpy.data.objects.get(f"Pivote_{nombre_pista}")

#Desbloqueo temporal de la esfera hasta añadir nuevas coordenadas esféricas
esfera.lock_location = (False, False, False)

#---SIN TRAYECTORIA (NO SE DIBUJAN CURVA Y MESH)---
if subpista.tipo_trayectoria == 'ESTATICA':

    #1. Reset de las dependencias a la trayectoria anterior
    if pivote and esfera.parent != pivote:
        esfera.parent = pivote
        #Reset de la posición pasada con respecto al antiguo pivote o curva
        esfera.matrix_parent_inverse = mathutils.Matrix.Identity(4)

    #2. Esféricas -> Cartesianas
    x = radio * math.sin(azimut_rad) * math.cos(elevacion_rad)
    y = radio * math.cos(azimut_rad) * math.cos(elevacion_rad)
    z = radio * math.sin(elevacion_rad)

    #3. Asignación de la posición y bloqueo al pivote de la cabeza
    esfera.location = (x, y, z)
    esfera.lock_location = (True, True, True)
    return

```

Código 6.7: Generación de la trayectoria estática en Blender.

Nota. Elaboración propia.

El código con los apartados de código más importantes referidos a las trayectorias dinámicas está incluido en el apartado **A.3** a efectos de maquetado y estilo. El último cálculo para obtener el vector con la posición del vértice, consiste en multiplicar la rotación que sufre la inclinación de la trayectoria por la posición sobre el plano XY de su inclinación. Por último, se clarifica que los elementos de la escena no necesitan estar en coordenadas esféricas porque solo será el conjunto de vértices el que tenga impacto real a la hora de generar la pista binaural. El resultado de este proceso se ve representado en la figura 6.6 en Blender tal que así:

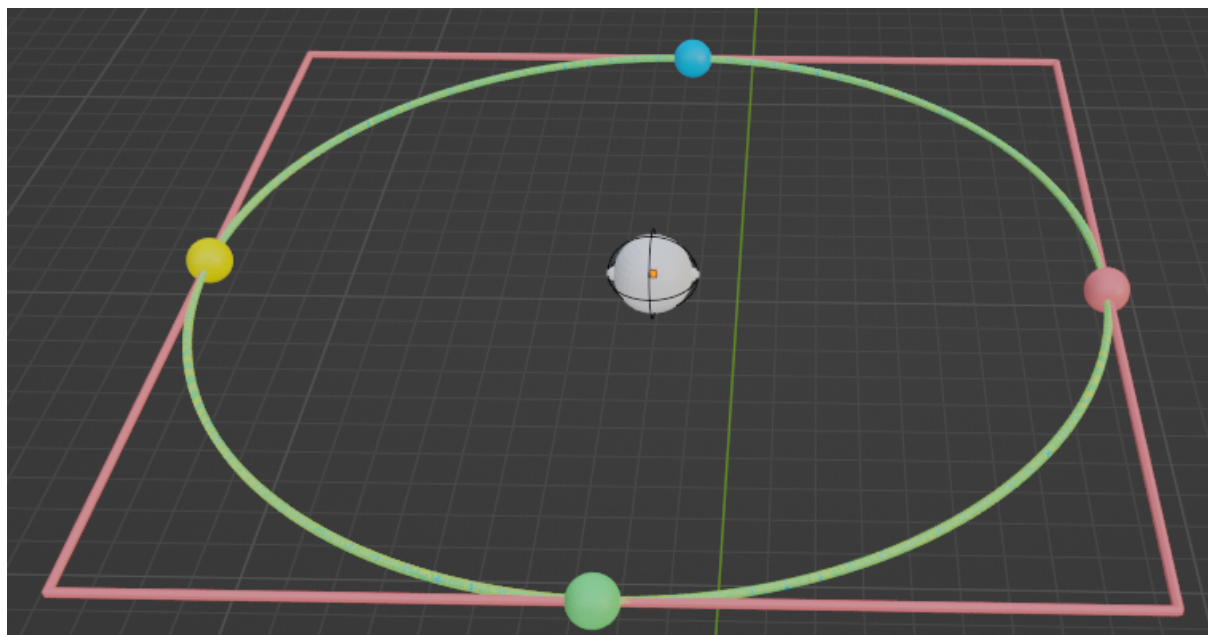


Figura 6.6: Representación de las subpistas dinámicas en blender

Por último, la implementación de la función `trigger` que permite actualizar en tiempo real simplemente recoge el estado del modelo y llama a la función `modificarTrayectorias` de `utils/` para borrar las antiguas trayectorias y generar las nuevas automáticamente. Al utilizar elementos de Blender y no tener una cantidad de elementos excesivamente alta como en la primera aproximación, la actualización visual se renderiza en tiempo real, pudiendo modificar las trayectorias y personalizar cada subpista correctamente. Con el sistema de trayectorias implementado, se procede a implementar el módulo más importante del reproductor, el procesador 8D.

6.2.2. Implementación del procesador 8D

La conexión entre el operador que genera la pista binaural, el kernel y el plugin se realiza de la misma forma en la que se conecta el separador de pistas por AI a su operador. Sin embargo, para implementar el procesador8D se necesitan más datos aparte de la ruta de las subpistas. En consecuencia, se ha empaquetado un archivo JSON con toda la información necesaria, siendo la ruta a este archivo con la operará el procesador. Este archivo JSON se recoge mediante la función `exportarPropiedades` del directorio `utils/`. Esta función exporta el conjunto de vértices de la malla o la posición exacta del centro de la esfera de la subpista. Para procesar las pistas se recogen las propiedades de la escena para sincronizar el movimiento de las esferas con la convolución final. Luego, para cada subpista, se recogerá una o varias coordenadas dependiendo del tipo de trayectoria, incluyendo la resolución espacial para las dinámicas. Entonces, la forma final del JSON es la siguiente:

```
#COORDENADAS_DSP.JSON
{
  "contexto_global": {
    "frecuencia_muestreo": 44100,
    "fps_escena": 30
  },
  "pistas": {
    #Subpista estática
    "Vocals": {
      "ruta_stem": "#Ruta...pruebaTFG\\mdx_extra_q\\AI James - Schoolboy
        Facination\\vocals.wav",
      "coordenadas": [
        {
          "r": 1.0,
          "azimut": 75.0,
          "elevacion": 40.0
        }
      ]
    },
    #Subpista dinámica
    "Drums": {
      "ruta_stem": "#Ruta...\\pruebaTFG\\mdx_extra_q\\AI James - Schoolboy
        Facination\\drums.wav",
      "coordenadas": [
        {
          "r": 1.0,
          "azimut": 90.0,
          "elevacion": 0.0
        },
        #Resto de coordenadas...
      ]
    },
    "resolucion_espacial": 60
  }
}
```

Código 6.8: JSON con las propiedades de la escena y subpistas.

Nota. Elaboración propia.

Con este archivo el primer paso que realiza el procesador es desempaquetar y leer el JSON y asignar las propiedades comunes a la convolución de todas las subpistas, independientemente de que sea estática o dinámica. Tras ello se cargan los impulsos HRIR para todas las subpistas. Este proceso fue en parte explicado a través de la justificación del diseño de datos en el apartado 5.4. En profundidad, su proceso se divide en **cargarHRIR()** y **buscarHRIR()**. El primero, para todas las coordenadas de todas las subpistas, recoge y guarda las HRIR reales que se utilizarán en la convolución, sabiendo que el mismo impulso podría ser utilizado por pares de coordenadas distintas. Esta premisa es la que determina al procesador a optimizar el proceso de búsqueda, integrando una lista de impulsos que serán asignados a pares (θ, δ) únicos. Se ha de diferenciar entre el impulso y la coordenada específica. Las coordenadas siempre serán distintas, las HRIR no tienen por qué. Al utilizar un modelo esférico, **cargarHRIR()** realiza un proceso de inversión entre los canales L y R, debido a que el conjunto KEMAR está optimizado para dar respuestas hasta $\theta \geq 180^\circ$ extendiendo el rango azimutal hasta $\theta = 359^\circ$. Después de preparar la HRIR única para cada coordenada, la lista auxiliar se elimina, ahorrando memoria. El bucle principal de **cargarHRIR()** que realiza este proceso es el siguiente:

```

#PROCESADOR DSP -> CARGARHRIR()
#1. Obtenemos los sets de coordenadas únicas para todas las pistas
for nombre_pista, datos_pista in pistas.items():
    coordenadas = datos_pista.get("coordenadas", [])

    for punto in coordenadas:
        coordenadas_unicas.add((punto["azimut"], punto["elevacion"]))

#2. Obtenemos las HRIR únicas más cercanas a todas las coordenadas de las
pistas
for az, el in coordenadas_unicas:
    #3. Búsqueda de HRIR para dicho punto
    ruta_hrir, invertir = buscarHRIR(ruta_diffuse, az, el)
    #4. Si no ha sido asignada se guarda por primera vez en el conjunto
    auxiliar
    if ruta_hrir not in aux_hrir:
        audio_hrir, fs_hrir = sf.read(ruta_hrir)
        aux_hrir[ruta_hrir] = audio_hrir
    #5. Obtenemos la HRIR (Par único ruta-audio)
    audio_hrir = aux_hrir[ruta_hrir]
    #6. Modificación de la HRIR base para dicha coordenada espejo en L y R (
    ej. 45 grados = 315 grados)
    if(invertir):
        audio_hrir = audio_hrir[:, [1, 0]]
    #Guardamos el HRIR al par azimut, elevación
    hrir_globales[(az, el)] = audio_hrir
#Limpiamos diccionario auxiliar
del aux_hrir

```

Código 6.9: Bucle de carga de las HRIR.

Nota. Elaboración propia.

La función `buscarHRIR()` determina los límites de búsqueda para (θ, δ) , independiente de que la UI permita elegir valores que la base de datos del MIT no estima. El apunte sobre el rango de coordenadas disponible en el set fue descrita al diseñar el modelo de datos. Aún así, la UI sigue ofreciendo la selección de estos valores, por si a posteriori se terminase insertando una base de datos que sí los comprenda. Aclarado el apunte, la función devuelve la ruta de la HRIR seleccionada y si esta ha de intercambiar los canales L y R para el par espacial específico. La función queda de la siguiente manera:

```

#PROCESADOR DSP -> BUSCARHRIR()
#1. Límites de elevación KEMAR (-40 como elevación inferior máxima y elevación
    en saltos de 10 en 10)
el_kemar = max(-40, el)
el_kemar = round(el_kemar / 10) * 10
ruta_el = f"elev{int(el_kemar)}"
ruta_carpeta = os.path.join(ruta_diffuse, ruta_el)

#2. Límites azimutales KEMAR (según aproximación al valor más cercano)
az_kemar = az
invertir = False

if 180 < az < 360:
    az_kemar = 360 - az
    invertir = True

#3. Obtenemos todas las HRIR para dicha elevación
archivos = os.listdir(ruta_carpeta)
az_disponibles = []

for archivo in archivos:
    if archivo.endswith(".wav"):
        az_archivo = archivo.split('e')[1].split('a')[0]
        az_disponibles.append(int(az_archivo))

#4. Obtenemos el archivo con menor valor azimutal
az_kemar_final = min(az_disponibles, key = lambda x: abs(x - az_kemar))
nombre_archivo = f"H{int(el_kemar)}e{int(az_kemar_final):03d}a.wav"
ruta_hrir = os.path.join(ruta_carpeta, nombre_archivo)

return ruta_hrir, invertir

```

Código 6.10: Búsqueda de una HRIR en diffuse.zip.

Nota. Elaboración propia.

La carpeta está organizada en orden de acceso δ y θ . La elevación, al estar separados los tramos de 10 en 10, `buscarHRIR()` se aproxima al valor más cercano, asignado a la variable `elKemar`. El valor azimutal responde a un proceso parecido pero diferente. Después de determinar si HRIR se tiene que invertir, se filtran los ángulos azimutales contenidos en la carpeta de `elKemar`. Esto se debe a la organización del dataset en la cantidad de puntos que hay por carpeta, los cuales no siempre están separados a la misma distancia, como sí sucede en la elevación. Con el conjunto obtenido, se asigna el valor azimutal más cercano, obteniendo la diferencia entre las dos HRIR entre las que se encuentre el azimut a asignar. Para obtener la diferencia de distancia entre los dos posibles valores azimutales, se utiliza una función lambda. Esta función determina el filtro que tiene que pasar el azimut buscado, si hubiera alguno a menor distancia, se cambiaría por el que se tiene ahora. Obtenido el par, se obtiene la ruta de la HRIR a asignar, que después se comprobaría si tiene que leerse a nivel de array de audio. Esta última es la operación más costosa de las dos, por eso se opta siempre por leer el dataset, pero no siempre leer el impulso HRIR.

El segundo procedimiento principal responde a la cadena DSP pura. Lo primero es iterar por las cuatro subpistas, determinando si la convolución a aplicar es estática o dinámica.

Si al iterar sobre la pista la longitud del vector de coordenadas del JSON para dicha subpista es 1, es estática. Para cualquier otra pista con un número de coordenadas mayor, se le aplicaría una convolución dinámica. La convolución estática es relativamente sencilla. Su proceso recoge el audio y coordenada, convierte la pista a mono, aplica la ganancia utilizando la ley inversa del cuadrado y aplica la FFT al audio y la HRIR L y R. Al final se devuelve el array de audio binaural de la subpista. El proceso es el siguiente:

```
#PROCESADOR DSP -> CONVOLUCION ESTÁTICA
#1. Variables globales
global hrir_globales
ruta_stem = datos_pista["ruta_stem"]
coordenadas = datos_pista.get("coordenadas", [])
audio_stem, fs_stem = sf.read(ruta_stem)

#2. Conversión a mono para aplicar el impulso (Espacialización por
    lateralización)
audio_stem = np.mean(audio_stem, axis=1)

#3. Obtención de las coordenadas
r = coordenadas[0]["r"]
az = coordenadas[0]["azimut"]
el = coordenadas[0]["elevacion"]

#4. Aplicación de la inversa del cuadrado conforme la amplitud y obtención
    de HRIR
ganancia = 1.0 / r
hrir = hrir_globales[(az, el)]
hrir_L = hrir[:, 0]
hrir_R = hrir[:, 1]

#5. Aplicación de FFT
audio_conv_L = fftconvolve(audio_stem, hrir_L, mode='full')
audio_conv_R = fftconvolve(audio_stem, hrir_R, mode='full')

#6. Únimos ambas señales en una matriz estéreo y aplicamos la ganancia
audio_salida = np.column_stack((audio_conv_L, audio_conv_R))
audio_salida *= ganancia

return audio_salida
```

Código 6.11: Proceso de convolución estática

Nota. Elaboración propia.

Por otro lado la convolución dinámica es el procedimiento más laborioso del procesador. Para ello, se obtienen las mismas variables que se utilizan para la convolución estática solo que se tendrá que tener en cuenta la `resolucionFrames`, `fps` y el bloque de audio que se está procesando. A nivel de proceso es muy parecido al estático puesto que se sigue aplicando la FFT de la misma manera. El factor a tener en cuenta es que, al aplicar *overlap-add*, se tiene que calcular el bloque de audio y la cola dejada por la superposición de los impulsos, factor relevante al método *overlap-add*. Entonces, conforme a la resolución espacial se obtiene el número de frames de muestras que se tendría en un bloque con la frecuencia de muestreo, `fps` y `resolucion`. Como ya se mencionó anteriormente, a mayor

resolución, menor tamaño de bloque y por tanto más calidad en la interpolación entre bloques de audio. Con el número de muestras por bloque, se obtienen el número de bloques a procesar totales. Una simple multiplicación. Después, otro procedimiento que difiere al estático es el cálculo de la longitud del audio de salida binaural a causa del método convolutivo empleado. Con todos estos valores se procede a aplicar cada HRIR a cada coordenada de la pista, la cual coincide exactamente con el número de bloques de audio obtenidos y actualizados por iteración. En resumen, el proceso de convolución dinámica está incluido en el bloque **A.4** del apéndice a efectos de maquetado y estilo.

Finalmente, el último proceso que realiza el procesador está relacionado con la masterización de la pista binaural. Después de convolucionar cada pista, se normaliza el audio para evitar que la pista final se sature y evalúe los volúmenes de cada subpista consecuentemente a su volumen original y el factor introducido por r . Al final de todo el proceso de convolución se guarda la pista en el mismo directorio que el plugin a efectos posteriores de tratamiento y exportación al timeline de Blender, donde se podrá reproducir y sincronizará con el movimiento de las subpistas. Para la exportación de la pista final se utiliza un codec *PCM-16*, un formato que exporta las pistas sin degradar su calidad. El bloque de código que realiza el procesador es el siguiente:

```
#PROCESADOR DSP -> CONVOLUCION DSP FINAL
#Convolución de cada subpista
#3. Creación de la pista con el master final
master_mix = np.zeros((longitud_maxima, 2))
for pista in pistas_procesadas:
    master_mix[:len(pista)] += pista
#4. Normalización de la ganancia
pico_maximo = np.max(np.abs(master_mix))
master_mix = master_mix / pico_maximo
#5. Se obtiene el nombre de la canción para el archivo temporal final
primera_pista = list(pistas.values())[0]
ruta_stem_ref = primera_pista.get("ruta_stem", "")
nombre_cancion = os.path.basename(os.path.dirname(ruta_stem_ref))
nombre_archivo_8d = f"{nombre_cancion}_8D.wav"
#6. Exportación temporal a la carpeta del plugin para manejar el Video
Sequencer
directorio_actual = os.path.dirname(os.path.abspath(__file__))
ruta_pista_binaural = os.path.join(directorio_actual, nombre_archivo_8d)
sf.write(ruta_pista_binaural, master_mix, frecuencia_muestreo, subtype='
    PCM_16')

return ruta_pista_binaural
```

Código 6.12: Proceso de masterización de la pista 8D

Nota. Elaboración propia.

Implementado el sistema de trayectorias y el procesador DSP, solo queda exportar la pista binaural resultante desde Blender.

6.3. Incremento 3: Módulo de exportación

Este incremento tiene como objetivo exportar a Blender la pista binaural para reproducirla y el proceso contrario, permitir al usuario exportar la pista al directorio que desee. La primera implementación de este módulo representa el último paso ejecutado por el operador del procesador 8D. Este paso ya no tiene nada que ver con el proceso anterior, pero sí con el flujo principal del reproductor. Tras obtener la pista binaural, se invoca una función que inserta la pista dentro del secuenciador de video o `VideoSequencer`. La función `integrarPistaBinaural`, carga la pista en el secuenciador de Blender desde el primer frame:

```
#INTEGRAR PISTA BINAURAL -> UTILS
def integrar_pista_binaural(scene, ruta_pista_binaural):
    #1. Asegurarnos de que el editor de está inicializando
    if not scene.sequence_editor:
        scene.sequence_editor_create()
    #2. Añade la pista a la secuencia con su nombre
    nombre_pista = os.path.splitext(os.path.basename(ruta_pista_binaural))[0]
    seq = scene.sequence_editor.sequences
    #3. Eliminar renders anteriores para evitar superposiciones de las canciones
    for strip in seq:
        if strip.name == nombre_pista:
            seq.remove(strip)
    #4. Insertar la nueva pista de audio en el primer frame
    pista_sonido = seq.new_sound(
        name=nombre_pista,
        filepath=ruta_pista_binaural,
        channel=1,
        frame_start=1
    )
    pista_sonido.show_waveform = True
    return True
```

Código 6.13: Proceso carga de la pista 8D al timeline de Blender

Nota. Elaboración propia.

La función determina si el secuenciador está iniciado y si ya había una pista 8D cargada anteriormente, eliminándola en su caso, para evitar superposiciones y no escuchar dos pistas binaurales a la vez. El resultado de esta función es visible en la figura 6.7 al acceder al `VideoSequencer`.

Finalmente, se inicializa el mismo procedimiento de conexión interfaz-operador que se ha implementado en el resto de incrementos, pero evitando el kernel al poder exportar de forma más sencilla con el útil `ExportHelper` de Blender. La función `exportarPista()` ejecutada por el operador recoge el `.WAV` de la pista binaural resultado del procesador DSP, contenida en el directorio `core/plugins/procesador8D`. Si no la encuentra, devuelve un error. Si es así, copia el archivo y lo guarda en la ruta seleccionada previamente por el usuario. El motivo de la llamada `invoke()` es justamente este motivo, obtener el nombre de la pista y evitar que el usuario lo cambie, ahorrando tiempo pues se supone que lo guardará bajo el pseudónimo 8D, pista binaural, entre otros. Esta implementación es simplemente puntual. El código que realiza este proceso es el mostrado en el bloque de código ??

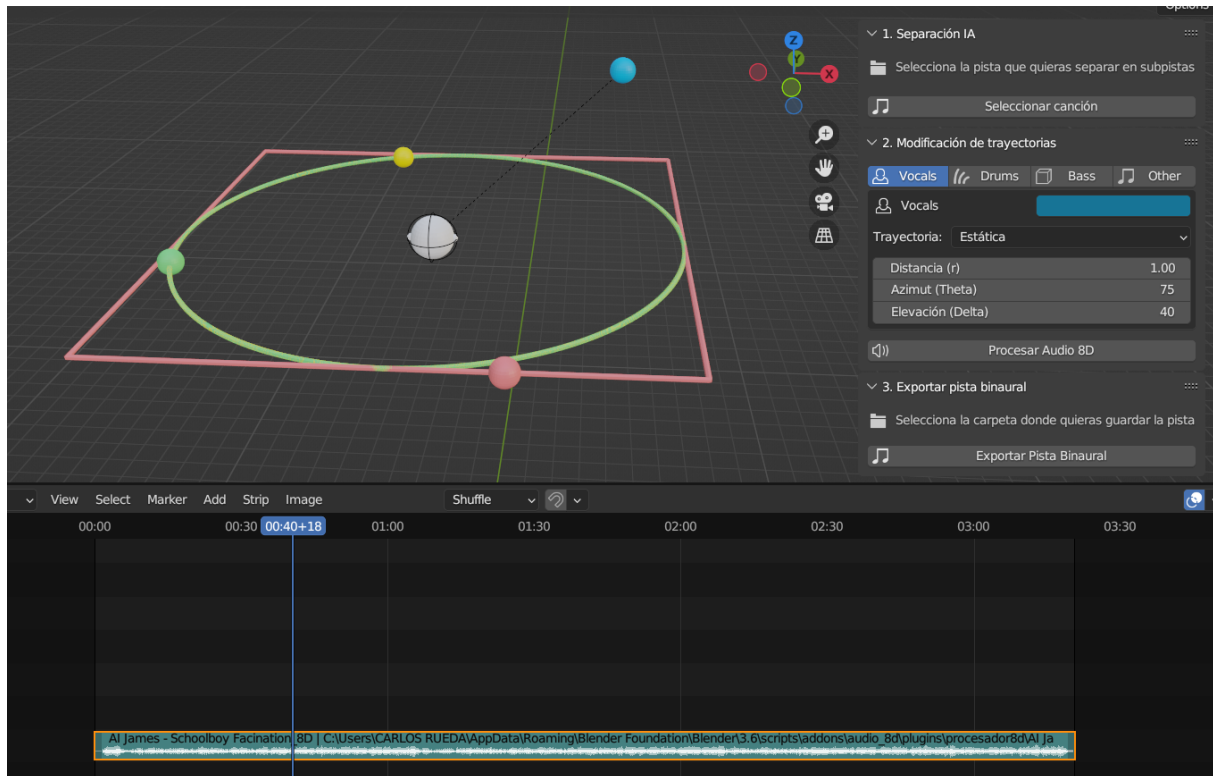


Figura 6.7: Vista del addon con la pista 8D cargada

```
#EXPORTAR PISTA BINAURAL -> UTILS
#1. Localización de la carpeta con el plugin
directorio_base= os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
carpeta_procesador= os.path.join(directorio_base, "plugins", "procesador8d")
ruta_origen = None
#2. Buscar el archivo .wav dentro de esa carpeta
if os.path.exists(carpeta_procesador):
    for archivo in os.listdir(carpeta_procesador):
        if archivo.endswith(".wav"):
            ruta_origen = os.path.join(carpeta_procesador, archivo)
            break # Cogemos el primero que encuentre (la mezcla 8D)
#3. No se encuentra el archivo procesado
if not ruta_origen or not os.path.exists(ruta_origen):
    raise FileNotFoundError("No se encontró ningún archivo de audio en la
    caché interna.")
#3. Exportación de una copia al directorio seleccionado por el usuario
try:
    shutil.copy2(ruta_origen, ruta_destino)
    return True, "Audio guardado exitosamente."
except Exception as error_exportacion:
    raise RuntimeError(f"Fallo al copiar el archivo: {str(error_exportacion)}")
```

Código 6.14: Proceso de exportación de la pista 8D del Video secuenciador

Nota. Elaboración propia.

Con el objetivo de verificar la exportación de la pista, la misma que se utilizó para separar

en pistas y generar la pista 8D, será la utilizada para exportarla a un directorio cualquiera. El resultado de la acción es la siguiente:

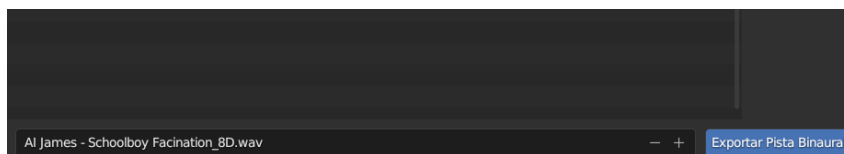


Figura 6.8: Ejecución del operador de exportación

mdx_extra_q	22/06/2026 16:51	Carpeta de archivos	
TFG-CRM	16/06/2026 18:49	Carpeta de archivos	
A Thousand Miles(cut)	10/06/2026 17:59	Archivo WAV	12.046 KB
Al James - Schoolboy Facination	22/06/2026 16:01	Archivo WAV	34.544 KB
Al James - Schoolboy Facination_8D	17/06/2026 20:33	Archivo WAV	12.046 KB
coordenadas_dsp	23/06/2026 0:02	Archivo de origen ...	2.602 KB

Figura 6.9: Vista del explorador de archivos al exportar una pista 8D

Cómo se puede apreciar en las figuras 6.8 y 6.9, se verifica el objetivo del operador, exportar la pista 8D que se está reproduciendo en todo momento.

El último añadido a todos los incrementos, aunque se ha realizado en esta fase, es la implementación de variables de control para obligar a pasar por todos los procesos, indicado por mensajes. Al comenzar la ejecución, la vista del addon se verá así:

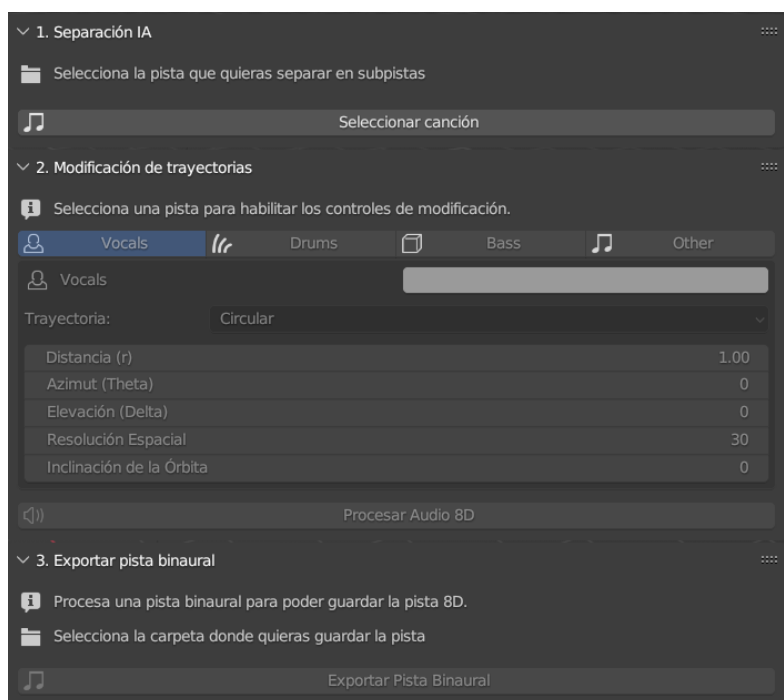


Figura 6.10: Vista del reproductor al iniciar el addon por primera vez

Con los tres incrementos implementados, finaliza este capítulo. A continuación, se realizarán las pruebas de rendimiento y calidad de audio.

Capítulo 7

Pruebas

Las pruebas descritas y analizadas en este capítulo se han orientado al módulo que genera la pista binaural. Más concretamente, al segundo incremento descrito en la planificación. En este capítulo se realizarán pruebas de rendimiento y de calidad de audio.

7.1. Pruebas de rendimiento

Las pruebas de rendimiento están sujetas a determinar si los valores estimados por los RNF-01 y 02 respecto al tiempo total y uso de RAM del procesador no son excedidos. Los datos utilizados para la medición son: duración de la pista t_c en minutos, tipo de trayectoria de las subpistas y su resolución espacial r_{esp} ¹ si las trayectorias fueran dinámicas. Las métricas que se obtendrán son: el tiempo de ejecución del procesador t_{dsp} en segundos, el ratio de tiempo según la canción procesada r_t y su uso de memoria RAM $cantRAM$ medido en MB.

Las pruebas se catalogan con la siguiente nomenclatura $P1$, $P2$, $P3$ y $P4$. Respectivamente, cada una de ellas tendrá en cuenta una situación de los datos medidos, describiendo un caso óptimo, promedio, otro promedio y de estrés, a fin de promediar el total de sus métricas y obtener conclusiones comparándose con la estimación de los RNF. Sobre cada prueba, $P1$ solo tendrá en cuenta trayectorias estáticas para una $t_c > 2min$. $P2$ solo tendrá en cuenta trayectorias dinámicas, pero con distintos valores de r_{esp} y un $t_c > 3min$. $P3$ tendrá las mismas condiciones que $P2$ solo que aumentando su r_{esp} hasta el máximo permitido por el sistema. Por último, $P4$ tendrá las mismas condiciones que $P3$ solo que con un $t_c > 5min$. Dicho esto, para cada una de las pruebas se escoge una pista del dataset que cumpla las condiciones de t_c . Para obtener los resultados de cada prueba, se han utilizado las librerías *time* y *tracemalloc* entre los intervalos de comienzo y final de la función principal de `procesadorDSP()`. La fórmula para obtener la media de las métricas utilizadas y r_t es la siguiente:

$$\bar{t}_{dsp} = \frac{(t_{dsp})_{P1} + (t_{dsp})_{P2} + (t_{dsp})_{P3} + (t_{dsp})_{P4}}{4} \quad (7.1)$$

$$\overline{cantRAM} = \frac{cantRAM_{P1} + cantRAM_{P2} + cantRAM_{P3} + cantRAM_{P4}}{4}; r_t = \frac{t_{dsp}}{t_c} \quad (7.2)$$

¹catalogadas tal que: v = vocals, d = drums, b = bass y o = other

La tabla 7.1 con el desglose del conjunto de datos utilizados y los resultados obtenidos es la siguiente:

Métrica Prueba	Pista	Duración t_c	Trayectorias	Resolución r_{esp}	Tiempo t_{dsp}	Ratio r_t	RAM $cantRAM$
P1	Hollow Ground - Ill Fate	02:21	Estáticas	Ninguna	3.70 s	0.026	572.24 MB
P2	Angels In Amplifire - I'm Alright	03:00	Dinámicas	v1, d15, b30 y o60	45.85 s	0.255	733.91 MB
P3	Bobby Nobody - Stitch Up	03:41	Dinámicas	v60, d60, b60, o60	94.67 s	0.219	929.83 MB
P4	BKS - Bulldozer	05:37	Dinámicas	v60, d60, b60, o60	99.56 s	0.295	1412.20 MB

Tabla 7.1: Resultados empíricos de las pruebas de rendimiento del motor DSP.

En base a los resultados de cada prueba, las métricas con las que se discutirá el rendimiento del sistema son las siguientes:

Métricas	Tiempo Promedio $\bar{t}_{dsp}$	Ratio Promedio $\bar{r}_t$	RAM Promedio $\overline{cantRAM}$
Promedio Global	60.95 s	0.251	912.05 MB

Tabla 7.2: Promedio global de las métricas de rendimiento evaluadas en las cuatro pruebas.

En base a los resultados de las tablas 7.1 y 7.2 se pueden realizar varias afirmaciones. Primero, el tiempo promedio en el que se procesa una pista es de 60,95s aproximadamente. Esto se debe a que el coste computacional para las pistas estáticas reduce el $\bar{t}_{dsp}$ al procesar dos FFT y cuatro HRIR L y R para cada subpista. Este factor no se traduce bien a la $cantRAM$, donde el uso de más de cuatro impulsos como mínimo ya supondría una carga de $cantRAM$ de 572,24MB. Entonces, al tener dos casos promedio donde la duración aumenta a valores de duración de la música estándar y la necesidad de recoger más de cuatro HRIR, el sistema casi sobrepasa el consumo de RAM permitido de 1GB del RNF-02. Pese a esta sobrecarga, el sistema responde bien en cuanto al $\bar{r}_t$, con unos valores promedio de 0,251, la mitad de lo estimado por el RNF-01. Aún así, la verdadera limitación del sistema sigue siendo el consumo de RAM, factor que depende mucho de la cantidad de puntos en los que coincidan la trayectoria de las subpistas. Por supuesto, el coste de cómputo para el caso P4 excede los límites en 412,20MB.

En resumen y en base a las anteriores afirmaciones, el sistema procesa las canciones en un tiempo estimado sobresaliente, pero a cambio consume una cantidad de RAM que sobrepasaría fácilmente el consumo de RAM estimado en el RNF-02. Por tanto, al haber consagrado los puntos fuertes y débiles del rendimiento del procesador, se necesita saber si la calidad de audio otorgada reduciría la compensación en cuanto al consumo de RAM. A continuación se realizarán pruebas para determinar la calidad de audio.

7.2. Pruebas de calidad de audio

El análisis y ejecución de las pruebas de audio estarán sujetas a tres escenarios o posicionamientos. Cada uno de estos escenarios tiene como propósito conocer distintas características de las pistas binaurales que se pueden generar. Para la obtención y discusión

de resultados se utilizarán espectrogramas analizados con Audacity que permiten observar cómo cambia la frecuencia a lo largo del procesamiento.

7.2.1. Escenario 1: Posicionamiento estático

El primer escenario responde a un posicionamiento estático de las subpistas. La justificación del escenario está orientado a comprobar si el procesador es capaz de aplicar el impulso correspondiente a los parámetros de la prueba y demostrar las diferencias entre ITD e ILD introducidas por la HRIR a un par (θ, δ) . Para la prueba, se ha utilizado el siguiente conjunto de datos para cada subpista:

Subpista	Distancia (r)	Azimut (θ)	Elevación (δ)	Tipo de Trayectoria	Resolución Espacial
Vocals	1.0	0°	0°	Estática	Ninguna
Drums	1.0	180°	0°	Estática	Ninguna
Bass	1.0	270°	-20°	Estática	Ninguna
Other	1.0	90°	45°	Estática	Ninguna

Tabla 7.3: Parámetros de la escena y de las subpistas del Escenario 1.

La tabla 7.3 sitúa cada pista en los puntos críticos del cono de confusión, frente, detrás, izquierda y derecha, con cierta variación en la elevación para *Bass* y *Other*. La canción del set de Rafii et al. (2019) (perteneciente al mismo set utilizado para las pruebas de rendimiento) seleccionada para el escenario es *AI James - Schoolboy Facination*. Para el análisis de la calidad de audio y la aplicación correcta de las HRIR, se ha obtenido el espectrograma perteneciente a los primeros 15 segundos de la canción procesada para los canales L y R de las cuatro subpistas.

Antes de continuar, para las figuras donde aparezcan, a causa de la incapacidad de Audacity para mostrar las unidades directamente sobre la figura del espectrograma puro, se ha declarado la siguiente configuración del eje vertical OX y horizontal OY antes de obtener sus métricas, especificada en la figura 7.1. El vertical, pertenece al tiempo medido en segundos. El horizontal, pertenece al espectro audible del ser humano con rango $[20Hz - 20kHz]$.

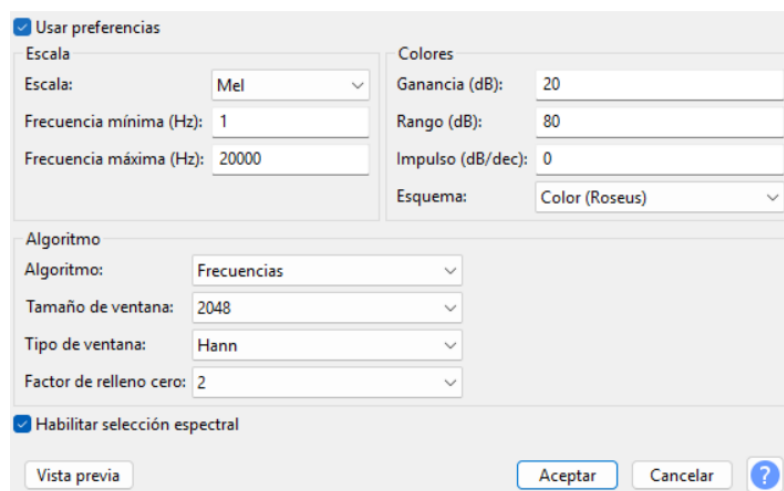


Figura 7.1: Unidades de los espectrogramas analizados en los tres escenarios

Mencionado este aspecto, los espectrogramas de cada subpista procesada son los siguientes:

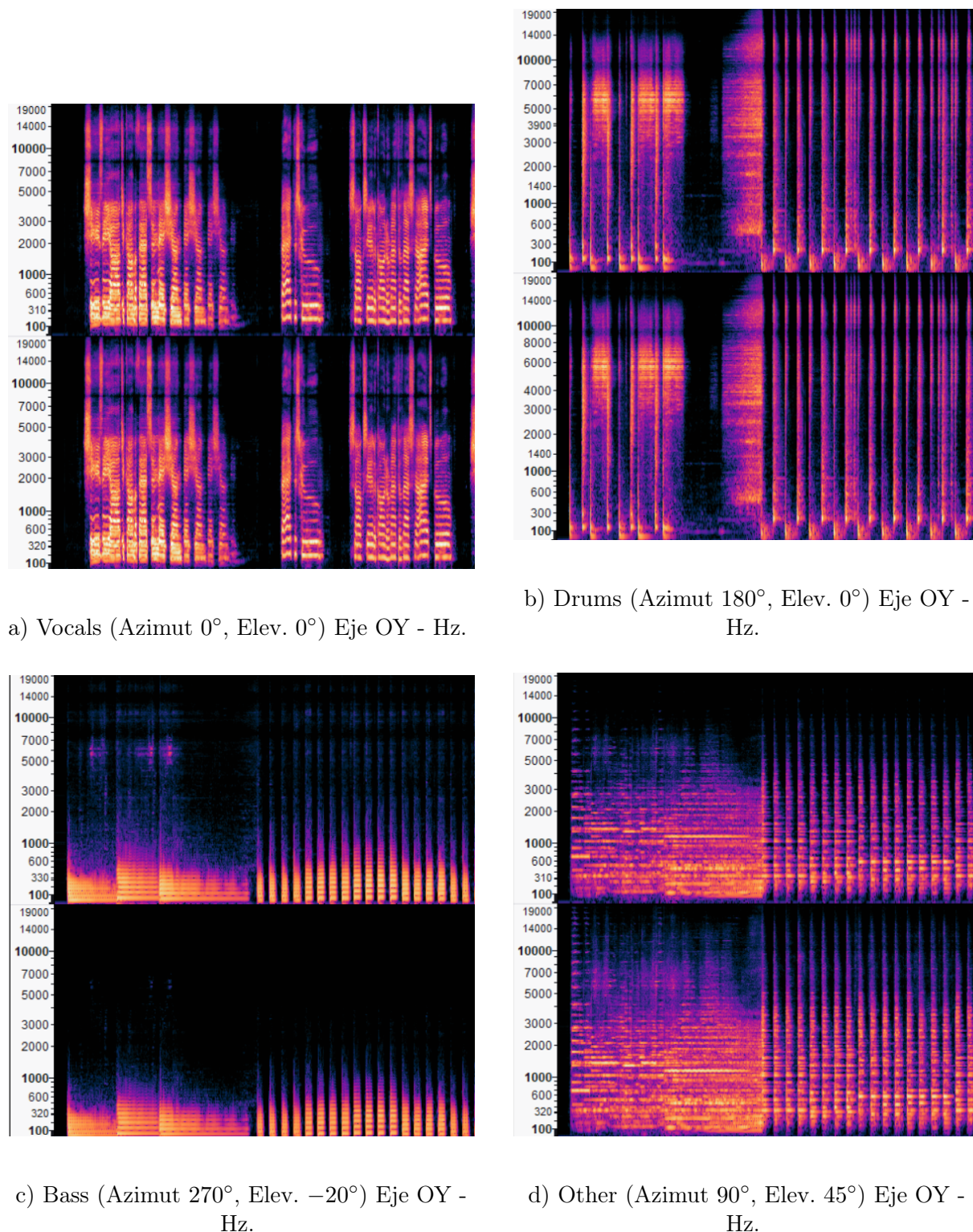


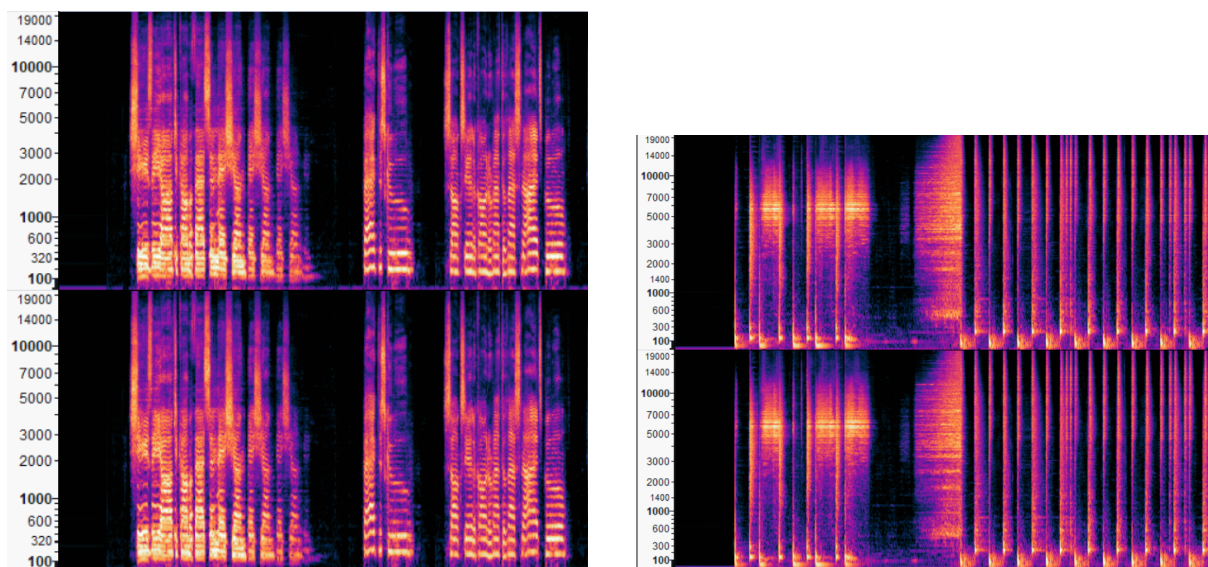
Figura 7.2: Espectrogramas resultantes de la convolución estática para cada subpista del Escenario 1.

Fuente: Elaboración propia.

De cada uno de los espectrogramas se obtienen los siguientes datos y afirmaciones. Para

las subfiguras c) y d) se justifica la aplicación del impulso comparando la reducción de las frecuencias entre los canales L y R. Para la subpista *Bass*, que está posicionada a la izquierda del oyente, se aprecia una reducción de las frecuencias agudas a partir de $3kHz$ significativamente en el canal R. Con ello, se puede afirmar que, aunque el corte frecuencial en el valor $f_{limiteILD}$ que se obtuvo en el apartado 2.1.2.2 no se ajusta exactamente a estos resultados, sí que se produce un filtro claro para las frecuencias agudas, otorgando a la subpista la propiedad binaural deseada. Aparte, el bajo es un instrumento que aporta frecuencias graves en su totalidad, por lo que las pocas frecuencias agudas que genera son mitigadas por la HRIR de la HRTF. Por otro lado, la subfigura d) presenta similitudes a la subfigura c), solo que la reducción frecuencial comienza a partir de $10kHz$ en el canal L. Continuando, el hecho de tener un valor δ elevado, permite que la reducción no sea tan drástica al tener una ITD más corta que si tuviera que recorrer todo el plano sagital para llegar al oído izquierdo. La huella del impulso se ve reflejada en estos aspectos, otorgando las propiedades binaurales deseadas.

Continuando, el espectrograma a) de las vocales de la figura 7.2 presenta los mismos valores de intensidad para los canales L y R. Visualmente, sus espectrogramas son iguales. Esto permite afirmar que los niveles de ITD e ILD son los mismos para cada oído. Así pues, como ya se discutió anteriormente en el estado del arte, cuando $\theta = 0^\circ$ o 180° , no existen diferencias de tiempo ITD y presión interaural ILD. Al tener las vocales situadas en el frente, apenas se verá reflejada una reducción de sus frecuencias que permita justificar la impronta del impulso HRIR. Por ello, se ha de comparar con el espectrograma de las subpistas separadas por el módulo de separación, para que tenga en cuenta la calidad de audio desde la que se parte antes de procesar la pista 8D. Los espectrogramas a) y b) de la figura 7.2.1 responden a las mismas propiedades temporales que los espectrogramas de la figura 7.2:



a) Vocals separados por IA. Eje OY - Hz

b) Drums separados por IA. Eje OY - Hz

Figura 7.3: Espectrogramas resultantes de la separación de las subpistas vocals y drums del Escenario 1.

Fuente: Elaboración propia.

Al comparar las subpistas de las vocales y la batería, previa y posteriormente a la aplicación de los impulsos, se obtienen los siguientes datos y afirmaciones. En primer lugar, las vocales no sufren apenas una alteración visible, los dos espectrogramas antes y después del procesamiento son prácticamente idénticos en L y R, menos una cuestión que se desarrollará más adelante. Al situar la pista en el frente exacto coincidiendo con la orientación de los pliegues de la oreja, el rango frecuencial inducido por el impulso no cambia. De esta manera se justifica la aplicación del filtro correctamente para el caso más neutral que existe, cuando θ y δ son iguales a 0° . Por otra parte, la batería si sufre una variación en su espectro frecuencial al estar situado detrás del oído. Debido a que los pliegues externos de la oreja en el filtrado de Pinna, explicado en el apartado 2.1.2.4, están orientados al contrario que de donde se sitúa la pista se produce una reducción en las frecuencias graves entre $[100Hz - 500Hz]$. Los pliegues introducen una reflexión y difracción que reduce la carga energética del impacto del sonido de la pista, alterando su huella espectral. Es con este dato y en comparación a que la pista situada al frente no varía, que la HRIR introducida en la batería resuelve el eje frente-detrás al ser procesada.

La cuestión mencionada en la explicación del espectrograma de las vocales y que también apoya a la justificación de la correcta aplicación de los impulsos por el primer escenario, es el corte frecuencial de $10kHz$ para todos los espectrogramas de la figura 7.2. Esto se debe a la introducción de las muescas espectrales introducidas en el mismo apartado 2.1.2.4. Las muescas espectrales ayudan a identificar mejor el tipo de pista espectral que percibe el cerebro cuando se trata de otorgar propiedades posicionales.

Habiendo justificado los datos empíricos del primer escenario, se afirma que las subpistas recogen las propiedades posicionales otorgadas por los impulsos HRIR, sobre todo para cambios de azimut. Estos son mucho más audibles y fáciles de identificar que los cambios en la elevación, hechos visibles en los espectrogramas vistos en este apartado. Por este mismo hecho, la calidad de audio percibida es proporcional al cambio que produce en el rango frecuencial de forma correcta antes y después de construir la pista 8D final. El siguiente escenario tiene como objetivo determinar la calidad de audio para huellas posicionales dinámicas.

7.2.2. Escenario 2: Posicionamiento dinámico

El segundo escenario responde a un posicionamiento dinámico de las subpistas. La justificación del escenario está orientado a determinar si el audio del procesador responde correctamente a los cambios continuos de impulso. Los valores (θ, δ) están más orientados a ofrecer variaciones drásticas de elevación, puesto que el escenario anterior determinó la calidad de audio para valores azimutales.

Subpista	Radio (r)	Azimut (θ)	Elevación (δ)	Inclinación de la Órbita	Resolución Espacial
Vocals	1.0	$0^\circ \rightarrow 360^\circ$	$-45^\circ \rightarrow +45^\circ$	45°	1
Drums	1.0	$0^\circ \rightarrow 360^\circ$	$-40^\circ \rightarrow +40^\circ$	90°	30
Bass	1.5	$0^\circ \rightarrow 360^\circ$	$-20^\circ \rightarrow +20^\circ$	0°	45
Other	2.0	$0^\circ \rightarrow 360^\circ$	$-45^\circ \rightarrow +45^\circ$	45°	60

Tabla 7.4: Parámetros de la escena y de las subpistas del Escenario 2.

La tabla 7.4 sitúa cada pista en diferentes puntos que recorren una amplia cantidad de variaciones de elevación. Debido a que las trayectorias dinámicas están sujetas a recorrer su trayectoria una vez durante toda la longitud de la canción, se han recogido dos puntos del espectrograma de la batería con duración de 5 segundos, a fin de comparar la huella espectral que existe en momentos distintos de su trayectoria. Para esta prueba se ha elegido la segunda canción del set, *AM Contra - Heart Peripheral*, a fin de comparar el procesamiento para distintas canciones y añadir variación al conjunto de pruebas. La trayectoria que seguirá cada pista está representada en la figura 7.4:

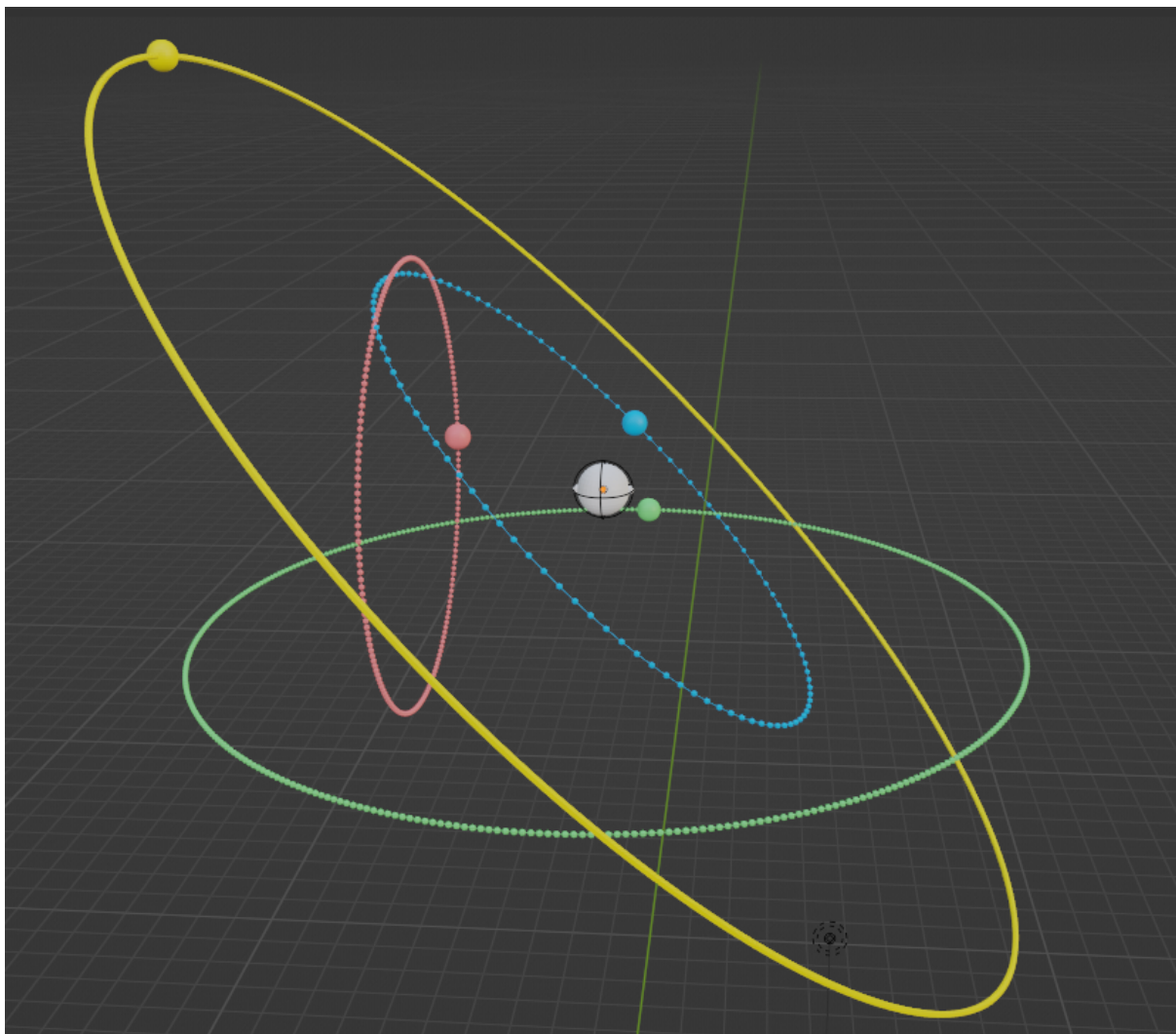
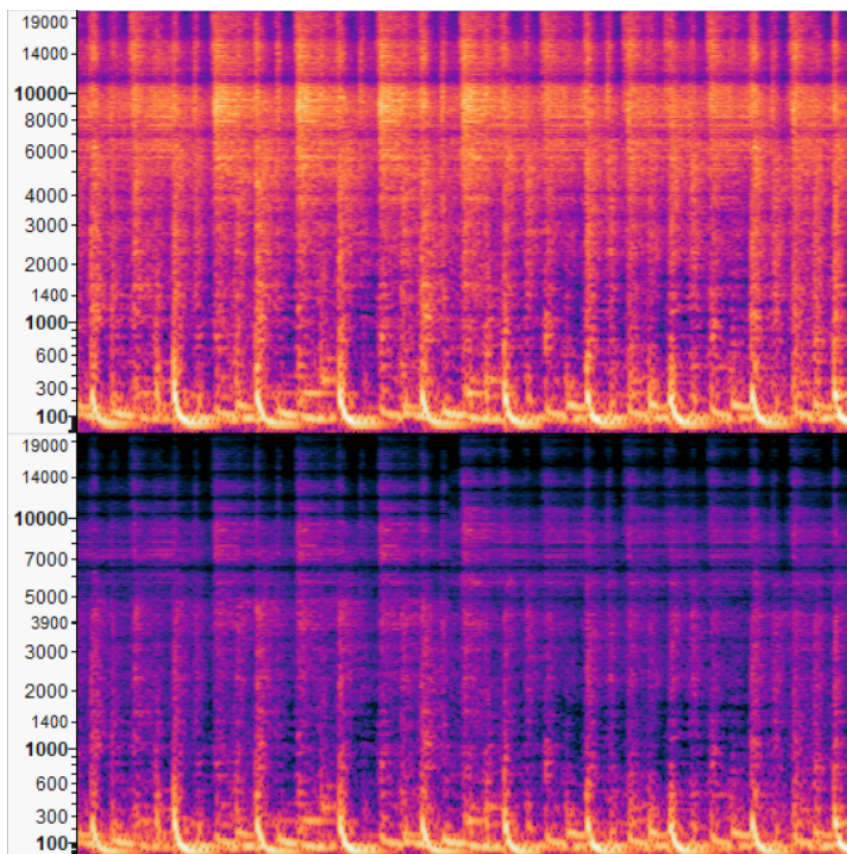
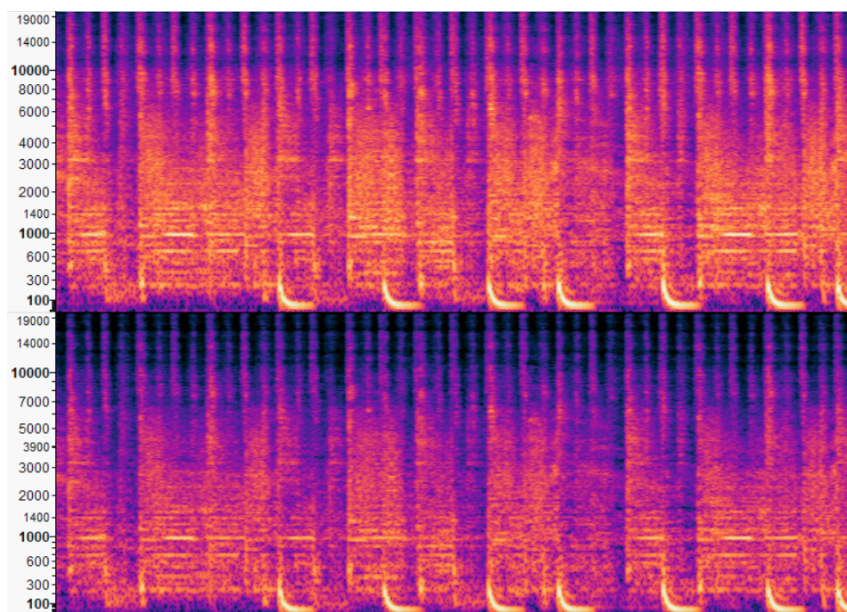


Figura 7.4: Vista del escenario 2

Tras procesar la pista binaural, se obtienen los siguientes espectrogramas, representados en la figura 7.5:



a) Drums (Azimut 270° , Elev. -45°) Eje OY - Hz



b) Drums (Azimut 270° , Elev. 45°) Eje OY - Hz

Figura 7.5: Espectrogramas L y R correspondientes a distintos puntos de elevación de la pista drums del Escenario 2.

Fuente: Elaboración propia.

Al comparar los espectrogramas a) y b), aparte de que ambos tienen más carga energética

en el canal L, en el espectrograma a) se pueden ver dos muescas espectrales sobre los $7kHz$ y $11kHz$. Esto se debe a que el ángulo de incidencia de las muescas para frecuencias cortas incide directamente con el lóbulo y el torso del cuerpo humano. Estos elementos generan las difracciones que reducen la cantidad de energía que realmente llega al tímpano para esos rangos frecuenciales. Este concepto ya fue explicado en el apartado de dependencia de la elevación 2.1.2.4. Entonces, apoyando la justificación en dicha explicación, las muescas del espectrograma a) coinciden en el rango de $[6kHz - 12kHz]$ de la explicación, que a su vez se mueven en el espectrograma b), resaltando el dinamismo en la aplicación de los impulsos HRIR. En el espectrograma b) se desplaza la muesca hasta valores frecuenciales aproximados a $11kHz$. Esto sucede al estar las orejas más orientadas hacia $\delta = 45^\circ$ y no tener el torso, hombros y el lóbulo como elementos que interfieran, permitiendo que no difracten y lleguen con mas energía las frecuencias bajas.

Continuando, este escenario también ha tenido en cuenta cómo afecta el factor r al volumen general de una pista 8D procesada. Para justificar su aplicación, simplemente se deberá comparar la pista separada original con la procesada. La subpista analizada será Other durante un tramo de un minuto en el intervalo $[30s - 90s]$. Esta es la subpista más alejada al centroide de las cuatro. Los espectrogramas de la pista procesada por AI 7.6 y la pista 8D procesada final 7.7 son los siguientes:

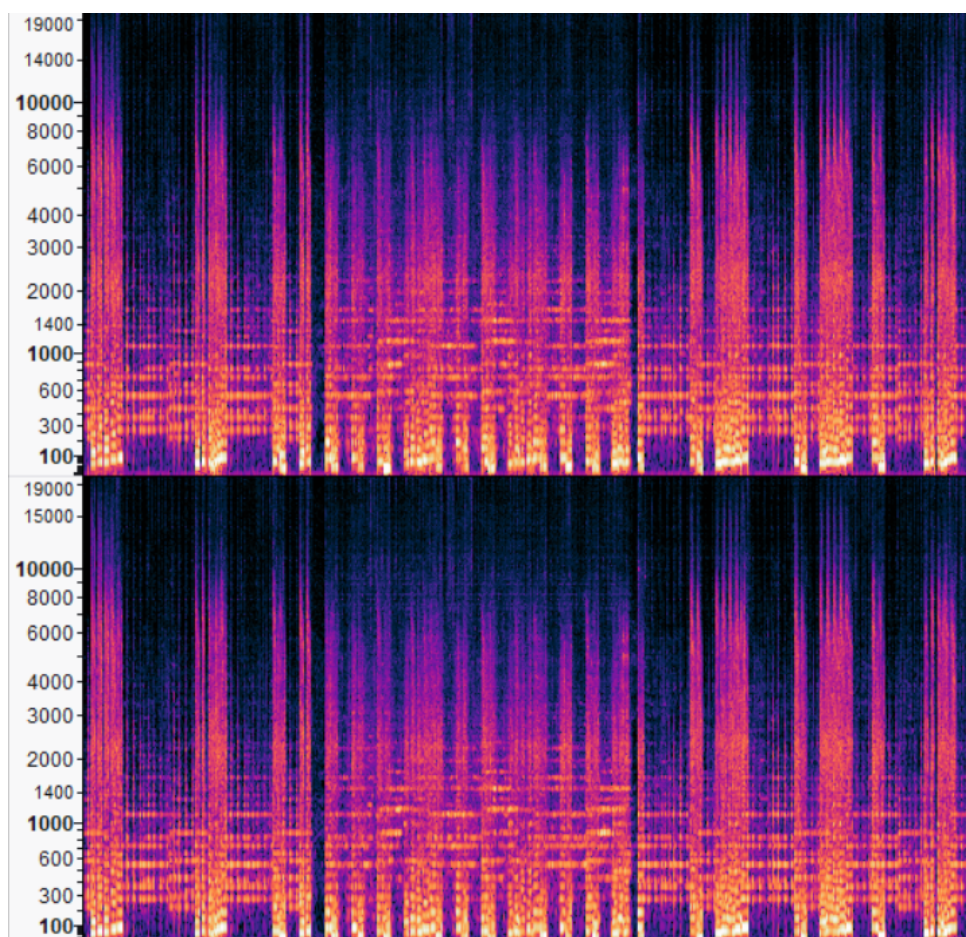


Figura 7.6: Espectrograma de la pista Other L y R sin procesar del Escenario 2.

Fuente: Elaboración propia.

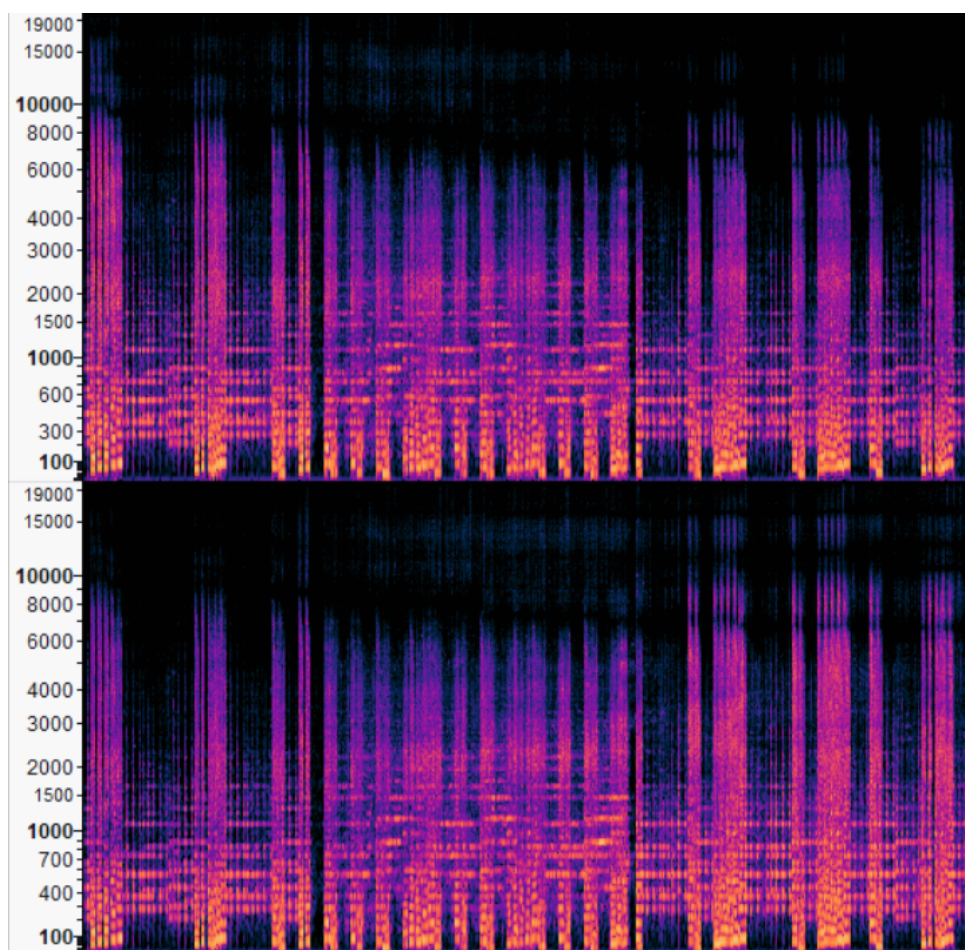


Figura 7.7: Espectrograma de la pista Other 8D L y R del Escenario 2. Eje OY - Hz
Fuente: Elaboración propia.

La comparación entre ambos espectrogramas es muy clara. Mientras que la que no ha sido procesada conserva su energía y sus frecuencias bajas son más notorias, la que ha sido procesada ve reducida su energía. La diferencia de color entre espectrogramas delata el volumen al que se percibe la pista, afirmando la existencia del factor r dentro de la subpista 8D.

Justificada la existencia de las alteraciones de la elevación en las pistas 8D dentro de las pistas y el factor r , se procederá a desarrollar un último escenario donde se pueda comparar la información completa de una canción 8D y determinar su calidad de audio cuando todas las subpistas son mezcladas y masterizadas.

7.2.3. Escenario 3: Posicionamiento mixto

El tercer escenario responde a un posicionamiento mixto de las cuatro subpistas. La justificación del escenario está orientado a reflejar si la suma de todas las subpistas 8D permite identificar correctamente sus propiedades posicionales. Para ello, se mezclan los parámetros de las dos últimas subpistas de la tabla 7.3 con las dos primeras de la tabla 7.4, resultando en la tabla 7.5:

Subpista	Radio (r)	Azimut (θ)	Elevación (δ)	Inclinación de la Órbita	Resolución Espacial
Vocals	1.0	$0^\circ \rightarrow 360^\circ$	$-45^\circ \rightarrow +45^\circ$	45°	1
Drums	1.0	$0^\circ \rightarrow 360^\circ$	$-40^\circ \rightarrow +40^\circ$	90°	30
Bass	1.0	270°	-20°	Estática	Ninguna
Other	1.0	90°	45°	Estática	Ninguna

Tabla 7.5: Parámetros de la escena y de las subpistas del Escenario 3.

La tabla 7.5 mezcla los valores de los dos escenarios anteriores a fin de medir si la pista binaural final permite identificar las características de las subpistas. Para ello, se ha utilizado la misma canción del escenario 2, a fines de poder comparar los espectrogramas que ya se conocen. Para este escenario se incluyen los espectrogramas de la canción sin procesar y la pista 8D final:

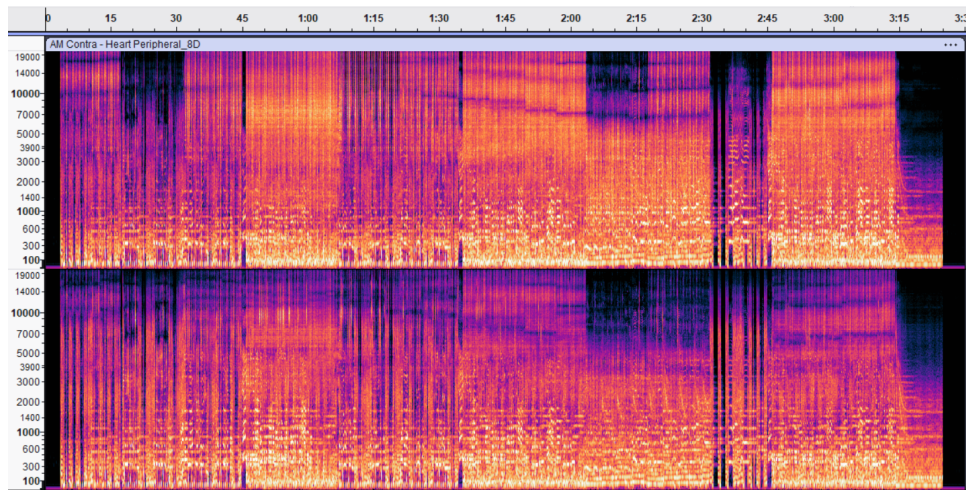


Figura 7.8: Espectrograma de la pista final 8D L y R del Escenario 3. Eje OX - Tiempo, Eje OY - Hz

Fuente: Elaboración propia.

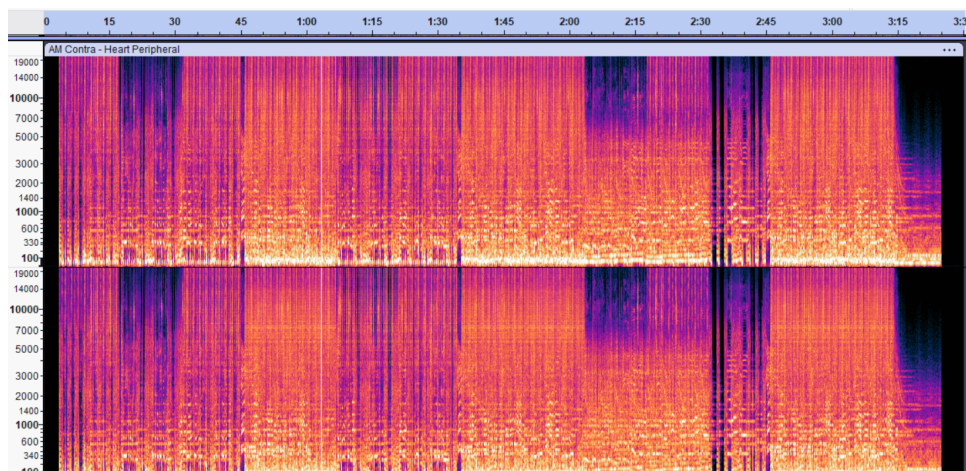


Figura 7.9: Espectrograma de la pista final sin procesar L y R del Escenario 3. Eje OX - Tiempo, Eje OY - Hz

Fuente: Elaboración propia.

La comparación entre los espectrogramas de las figuras 7.8 y 7.9 aporta la siguiente información. En primera instancia, el transcurso de las muescas espectrales es visible en el canal L de la pista 8D entre $[1,25min - 3,15min]$. En el resto de la pista 8D se pierden. Esto permite afirmar que, al haber aplicado las propiedades posicionales de la batería del segundo escenario, quedan latentes muchas de sus propiedades en la pista 8D, aunque no todas. El efecto contrario y la razón por la que no se aprecian bien las muescas espectrales durante toda la canción en el espectrograma 7.9 es la superposición de todas las pistas. Al mezclar las subpistas binaurales en una sola, al igual que en la mezcla de una canción, las frecuencias de cada instrumento irremediablemente comparten un espacio, rellendo los huecos que pudieran haber dejado las muescas espectrales. La saturación de la información aportada por cada instrumento oculta las propiedades espectrales, siendo más notorias cuando se aíslan y escuchan las pistas 8D por separado, como sucede cuando se analizan los espectrogramas por separado en el resto de escenarios.

En resumen, el conjunto de escenarios permite afirmar que las propiedades posicionales quedan latentes cuando se analizan y escuchan las subpistas 8D por separado, sin embargo, cierta información quedaría enmascarada cuando se mezclan las subpistas. Aún así, el procesamiento arroja datos que contrastan positivamente con el proceso de audición binaural del marco teórico. Tras concluir las pruebas de calidad de rendimiento y audio, se procede a obtener las conclusiones pertinentes en comparativa a la estimación realizada en la planificación y cerrar la memoria del proyecto.

Capítulo 8

Conclusiones

El siguiente capítulo tiene como objetivo presentar la comparación entre la estimación de los costes temporales y económicos junto con la discusión sobre la consecución de los objetivos, RF y RNF relacionados con los objetivos. Además, se aportará la justificación correspondiente a la resolución real de los riesgos descritos en el apartado 3.7. Al final del capítulo se redactan las conclusiones personales y trabajo futuro.

8.1. Contraste empírico del proyecto temporal y económico

La cantidad total de días **estimada** en el DG de la figura 3.1 y la planificación, es de **80.24 días** laborables. Tras completar el proyecto, el DG **real** descrito en la figura 8.1 asciende a una duración de **91 días** laborables. Once días más de lo previsto. Esto sitúa el final del proyecto previsto del **22/05/2026** → **08/06/2026**.

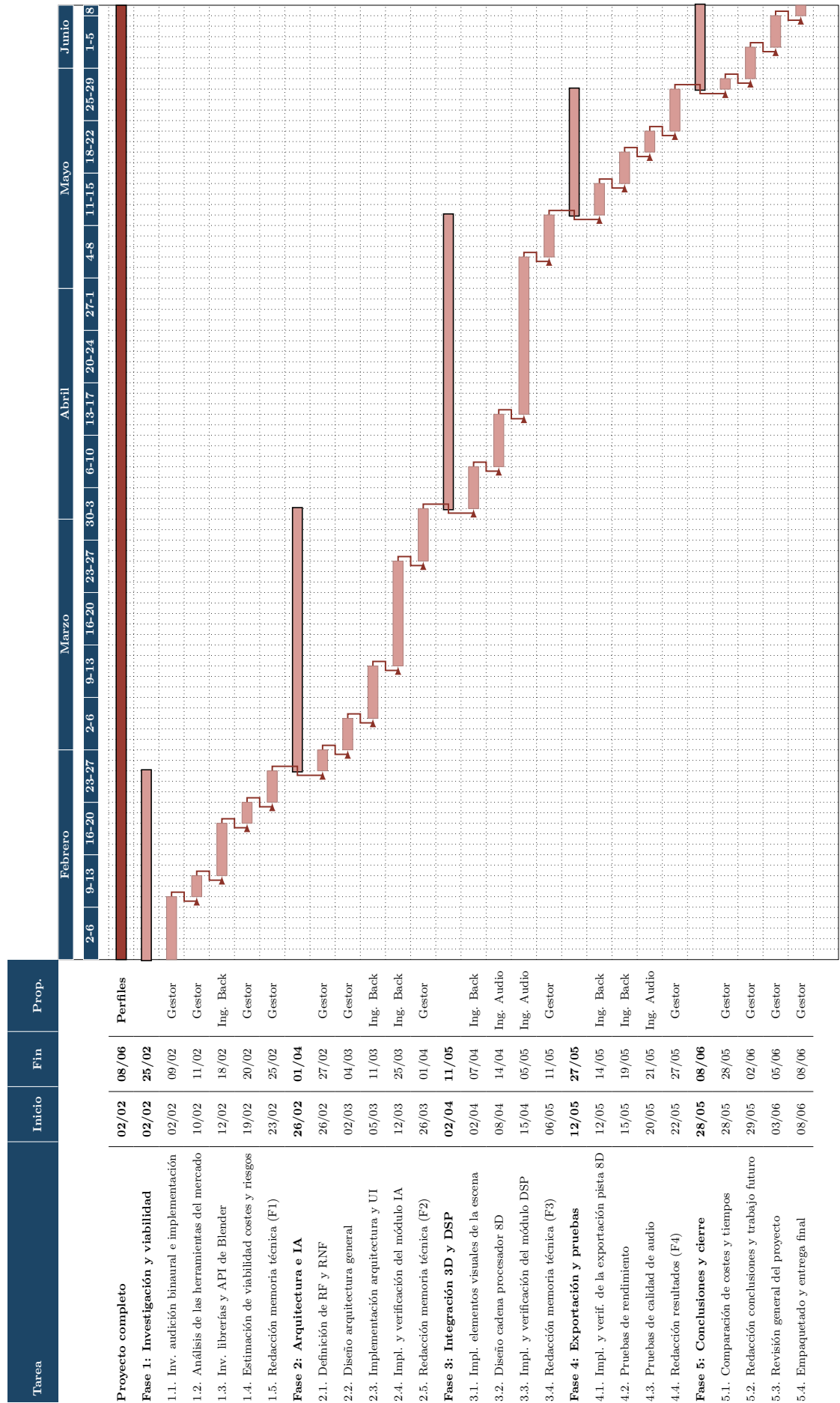


Figura 8.1: Diagrama de Gantt real del proyecto.

La principal causa del aumento de la duración del proyecto ha sido la curva de aprendizaje en la investigación de la primera fase y la implementación de los dos primeros incrementos, los más demandantes a nivel técnico, más un pequeño retraso en la última fase. El resto de tareas ha tenido la duración estimada. Las diferencias temporales de cada fase son las recogidas en la tabla 8.1:

Nombre de la fase	Tareas que causaron el aumento	Retraso de la fase	Perfiles profesionales causantes
Fase 1: Investigación y viabilidad	1.1, 1.3	+ 3 días	Gestor de proyectos, Ingeniero Backend
Fase 2: Arquitectura e IA	2.3, 2.4	+ 4 días	Ingeniero Backend
Fase 3: Integración 3D y DSP	3.1, 3.2, 3.3	+ 3 días	Ingeniero Backend, Ingeniero de Audio
Fase 4: Exportación y pruebas	Ninguna	0 días	Fase desplazada, sin aumentos en la duración
Fase 5: Conclusiones y cierre	5.2	+ 1 días	Gestor de proyectos

Tabla 8.1: Desvío de la planificación temporal frente a la real.

El retraso de la primera fase ha surgido por la curva de aprendizaje de los contenidos teóricos, al tener que demostrar y explicar el proceso de audición binaural, el cual tiene una gran cantidad de factores a tener en cuenta. Además, al tener una gran cantidad de versiones de las librerías *scipy*, *tracemalloc* y *demucs* que a su vez dependen de otras librerías, tuvieron que realizarse varias iteraciones conforme al proceso de instalación del add-on.

La segunda fase tuvo un retraso debido a la discusión sobre qué elementos serían los mejores para optimizar la visibilidad de la UI a la vez que se aseguraba que contenía todos los elementos necesarios para funcionar. Después, el primer incremento real supuso la elección del modelo de AI, mitigando el **R1**. Al final, el kernel del módulo de AI elegido fue **mdxExtraQ** al ser un modelo que trabaja exclusivamente con música y no otro tipo de archivos de audio ajenos, siendo esta la versión reducida de **mdxExtra**. También afectó la consecución de ofrecer un modelo que pudiera servir de base para cualquier sistema que ejecute el reproductor.

La segunda fase tuvo un retraso al intentar mitigar los riesgos **R2-R5** de la tabla de riesgos 3.10 de los dos procesos principales de la integración del procesador 8D. El primer proceso de este incremento es el más complicado puesto que los ingenieros tienen que optimizar el acceso a la base de datos de KEMAR, un proceso con varias formas de implementación. Al contrario, la cadena DSP tiene procesos comunes para las dos vertientes de trayectorias asignadas. La quinta y última fase tuvo un día de demora en la redacción de conclusiones.

En resumen, los costes reales asumiendo el cambio de planificación son los presentados en las tablas 8.2, 8.3 y 8.4:

Profesional	Horas Totales	Sueldo Neto	Seguridad Social	Sueldo Bruto
Gestor de proyectos	312	7800.00 €	2418.00 €	10218.00 €
Ingeniero de audio	176	5035.56 €	1561.02 €	6596.58 €
Ingeniero backend	240	6866.66 €	2128.66 €	8995.32 €
Total				25809.90 €

Tabla 8.2: Desglose de los costes personales reales post-desarrollo.

Software	Días de uso	Meses con suscripción	Precio Total
Visual Studio Code	45	∞	0.00 €
Blender	7	∞	0.00 €
Visual Paradigm 16.1 (Standard)	11	4	66.20 €
GitHub	45	∞	0.00 €
Audacity	5	∞	0.00 €
Total de Costes de Software			66.20 €

Tabla 8.3: Costes indirectos de software y licencias reales.

Profesional	Coste Bruto Estimado	Coste Bruto Real	Desvío Económico	Desvío Temporal
Gestor de proyectos	9956.00 €	10218.00 €	+ 262.00 €	+ 8 horas
Ingeniero de audio	5397.20 €	6596.58 €	+ 1199.38 €	+ 32 horas
Ingeniero backend	7196.26 €	8995.32 €	+ 1799.06 €	+ 48 horas
Total	22549.46 €	25809.90 €	+ 3260.44 €	+ 88 horas

Tabla 8.4: Comparativa de costes personales brutos estimados frente a al real y su desvío equivalente.

Finalmente, sumando los costes brutos reales y el total de coste del software, el monto total del proyecto asciende a **25876.10 €**. Con los costes temporales y económicos reales, se afirma que la desviación del proyecto en ambos aspectos no ha sido negativa, sino que ha respondido a la consecución real de querer resolver el proyecto con la mejor calidad posible, mitigando los riesgos que hubiera. A continuación, se evaluarán la consecución de los RF, RNF y riesgos.

8.2. Evaluación de objetivos, requisitos y riesgos

Tras la finalización del desarrollo y la ejecución de las pruebas, se puede afirmar que el **objetivo principal** del proyecto se ha **cumplido con éxito**, logrando un sistema funcional capaz de reproducir audio 8D con los módulos de separación de AI y el de convolución DSP en un entorno interactivo.

Para determinar el grado de cumplimiento de los subobjetivos se evalúan los RF y los RNF desde la perspectiva de los mismos. Esta evaluación permite identificar cuáles de

ellos se han alcanzado en su totalidad y cuáles presentan limitaciones que darán paso en la sección 8.3 a las líneas de trabajo futuro:

El **O1-(Desarrollar una interfaz gráfica tridimensional interactiva)** y el **O2-(Implementar un módulo de separación mediante AI)** se han alcanzado satisfactoriamente. Esto queda demostrado empíricamente al haberse validado con éxito la carga, separación y modificación de pistas descrita en los requisitos RF-01 al RF-04, operando de forma exclusiva en Blender, resolviendo el RNF-03.

El **O3-(Implementar un motor algorítmico de convolución de audio)** se ha cumplido parcialmente. Aunque la integración de las respuestas HRIR con las subpistas funcionan correctamente, validando el RF05, 06 y RNF-01, existen limitaciones arquitectónicas en el aislamiento de procesos frente a fallos RNF-05 y picos de consumo de memoria RAM que exceden lo planificado, anulando el RNF-02.

El **O4-(Garantizar la exportación y validación del audio generado)** también se ha cumplido parcialmente. Si bien la pista final se exporta libre de artefactos acústicos superando las pruebas de calidad, el sistema carece actualmente de un entorno de previsualización de audio en tiempo real, anulando el RF-07, generando archivos de audio temporales en el directorio del plugin del procesador al exportar, anulando también el RF-08.

Por tanto, las limitaciones y cumplimientos parciales detectados en los objetivos O3 y O4 son los que justifican y motivan directamente las propuestas de mejora detalladas en la sección de trabajo futuro. A continuación, se detalla en profundidad la justificación técnica de la consecución para cada uno de los requisitos y la mitigación de los riesgos en orden de definición.

Los RF-01 a RF-04 se han logrado exitosamente. Esta consecución recae en la confirmación de la verificación de uso de los módulos registrada previamente en la planificación. Los dos módulos responden correctamente a la ejecución de su acción por parte del usuario, otorgando el conjunto de pistas separadas con AI al igual que un conjunto de opciones de personalización de trayectorias que concuerda con la descripción de los requisitos RF-02, 03 y 04. Respecto a los requisitos que responden a la generación de la pista binaural, el RF-05 y 06 se implementan correctamente, desarrollando el procesamiento DSP en diferido a la escena de Blender. No obstante, el sistema no responde en su totalidad al RF-07. Por esta razón se cataloga como incompleto. Continuando, el sistema sincroniza los elementos visuales después de procesar la pista 8D, sí, pero no tiene ninguna opción para poder previsualizar las modificaciones que se realizan antes de procesar la pista con respecto al audio. Este factor causa que el usuario pueda perder percepción a la hora de querer construir la pista 8D deseada, determinando al usuario iterar varias veces en el proceso de generación. Después, el RF-08 sí guarda exitosamente la pista 8D final, a causa de generar un residuo en memoria en el directorio donde reside el plugin. Por esta razón se cataloga como incompleto. En resumen, el reproductor realiza correctamente las tareas para las que fue diseñado, pero no termina de encajar algunos matices que enriquecerían enormemente el trabajo.

Los RNF-01 y 02, ya discutidos en el apartado de pruebas de rendimiento 7.1, el segundo se cataloga como incompleto. Continuando, el resto de RNF-03 y 04 se consiguen con éxito. El sistema opera totalmente a través de Blender, ayudándose de su API conformando el esqueleto del trabajo, filtrando los archivos no deseados con las clases auxiliares que

ofrece el motor. Por último, el aislamiento de procesos responde correctamente gracias a las estructuras `try...except` implementadas en código de forma individual para cada proceso, parando el flujo cuando surge un error, evitando extenderlo al resto de elementos del sistema. No obstante, la dependencia de los operadores a realizar múltiples operaciones que no sean estrictamente la de un controlador, no aísla en totalidad los procesos. Por esta razón se cataloga el RNF-05 como incompleto.

Por último, consultando la tabla 3.10 se pueden realizar las siguientes afirmaciones. El **R1** ha sido mitigado gracias a la propia verificación exitosa de operabilidad del módulo. El factor limitante de la memoria RAM es totalmente dependiente al sistema informático que ejecuta el reproductor, dependiendo en gran parte de este y no del sistema diseñado. **R2** y **R3** son mitigados al aplicar los conceptos teóricos necesarios y la propia encapsulación del procesador frente al resto de procesos. El **R4** es eliminado gracias al último proceso de masterización de la cadena del procesador 8D. El **R5** no ha sido mitigado completamente. Pese a proporcionar una calidad de audio aceptable, siempre pueden existir factores externos como la calidad del módulo de separación empleado. En el caso del modelo implementado no existe riesgo, pero sí podría para otros modelos. Este es un factor futuro a evitar. Por último, el **R6** es eliminado al iterar el proceso de instalación, el cual compatibiliza las librerías necesarias al ejecutar los procesos en un entorno de Python dedicado a la implementación del sistema, proceso ajeno al Python interno de Blender.

Finalmente, se recogen estas especificaciones en la tabla 8.5:

Categoría	Totales	Conseguidos / Mitigados	Incompletos / Parciales	Tasa de éxito
Requisitos Funcionales	8	6	2	75.00 %
Requisitos No Funcionales	5	3	2	60.00 %
Riesgos	6	5	1	83.33 %
Balance Global del Sistema	19	14	5	73.68 %

Tabla 8.5: Resumen porcentual del grado de consecución de los objetivos, requisitos y la mitigación de los riesgos del proyecto.

A partir del porcentaje del balance global del sistema de la tabla 8.5 el desarrollo del presente TFG verifica una calidad de rendimiento y audio exitosos. El reproductor integra a través de las herramientas elegidas un espacio inmersivo donde seleccionar, escuchar y facilitar el acceso a tecnologías y aspectos de la música 8D. Estos aspectos resaltan los objetivos propuestos por la motivación explicada al principio de la memoria. La implementación del sistema de reproducción que propone el TFG ha llevado a cabo construir un espacio que sea fácil de entender tanto para personas que no han interactuado con este tipo de tecnologías hasta servir como una herramienta de trabajo para ingenieros de audio. La personalización que integra el sistema responde a los estándares de la industria al haber utilizado tecnologías ya consolidadas como Blender. Por último y con fines de cerrar la memoria, se proponen una serie de líneas de trabajo a futuro para continuar y mejorar el proyecto presentado.

8.3. Líneas de trabajo futuro

Las líneas de trabajo propuestas responden a la mejora del porcentaje del balance global del sistema. Resulta prioritario desarrollar un sistema de previsualización en tiempo real que permita al usuario auditar la sincronización entre las trayectorias espaciales y el audio antes de ejecutar el renderizado definitivo, mejorando así la usabilidad. También, la resolución y mayor optimización del uso de memoria RAM es otra línea de trabajo a futuro, mejorando el sistema de interpolación base proporcionado por la librería *scipy* y por ende mejorando la calidad de audio.

A fecha de depósito de este TFG, el reproductor únicamente implementa el modelo esférico simétrico, por tanto, es de interés integrar otros modelos que respondan a distintos tipos de anatomía del oído, los cuales modifiquen la forma del oído y así hacer responder al usuario de forma distinta, adaptando el reproductor a la anatomía del oyente.

Por último, la arquitectura microkernel del reproductor deja abierta la integración de distintos módulos. Esto responde a cualquier módulo que tenga que ver con audio espacial o audio estéreo como el de realizar una mezcla de las pistas en tiempo real. Por último, el módulo de separación con AI podría modificarse para utilizarse en sistemas con una GPU, abriendo la posibilidad de intercambiar, mezclar o aislar la separación de las diferentes subpistas, las cuales en número podrían aumentar como trabajo futuro.

Apéndice A

Resolución de diagramas, tablas y declaración de uso de IA

A continuación se incluyen los elementos que, por razones de maquetación y estilo, no han podido ser comprendidos dentro del desarrollo habitual de la lectura del documento. Aparte, se incluye la declaración de uso de inteligencia artificial.

A.1. Descripción de atributos del DCC

Clase	Atributo	Definición del atributo
Pista estéreo	<code>archivoAudio</code>	Archivo de audio <i>.wav</i> original cargado por el usuario a través de la interfaz.
	<code>frecuenciaMuestreo</code>	Tasa de muestreo de la señal original. <i>Ej. 44.1 kHz.</i>
	<code>duracionSegundos</code>	Cantidad de segundos que dura la pista.
	<code>cantidadFrames</code>	Cantidad en frames que dura la pista.
Subpista	<code>tipoFuente</code>	Clasificación de la pista (voz, batería, bajo u otros) extraída por el motor de AI.
	<code>datosAudio</code>	Vector con las muestras de audio de la subpista.
Escena 3D	<code>fps</code>	Fotogramas por segundo de la escena.
	<code>escala</code>	Factor de escalado del conjunto de objetos que conforman la escena.
	<code>elevacionBase</code>	Distancia sobre el plano $(0, 0, 0)$ sobre el que están suspendidos los elementos de la escena.
Trayectoria	<code>coordenadasEspaciales</code>	Vector con la posición en coordenadas (r, θ, δ) en cada instante temporal.
	<code>limitesEspaciales</code>	Restricciones sobre las libertades de movimiento en una trayectoria.
	<code>tipoTrayectoria</code>	Categoría de la trayectoria.
	<code>inclinacionOrbita</code>	Valor de inclinación sobre el eje Y que sufre la trayectoria.
	<code>resolucionFrames</code>	Cantidad de puntos que determinan cada cuánto se obtiene una métrica binaural.
BD Kemar	<code>directorioRaiz</code>	Ruta de la carpeta local que contiene los datos acústicos HRIR medidos por el MIT.
HRIR	<code>azimut</code>	Ángulo horizontal (θ) correspondiente a la medición.
	<code>elevacion</code>	Ángulo vertical (δ) correspondiente a la medición.
	<code>coeficientesHRIR</code>	Vectores con la respuesta al impulso del oído izquierdo y derecho ante el estímulo en (θ, δ) .
ProcesadorDSP	<code>bufferProcesamiento</code>	Memoria temporal donde se aplica la cadena DSP.
	<code>estadoProcesamiento</code>	Indicador del éxito o fallo de la cadena DSP.
PistaBinaural	<code>audio8D</code>	Archivo de audio estéreo con el efecto de espacialización aplicado.
	<code>directorioExportacion</code>	Ruta donde se almacenará la pista binaural final.

Tabla A.1: Identificación y descripción de los atributos del DCC.

A.2. DA del CU-02

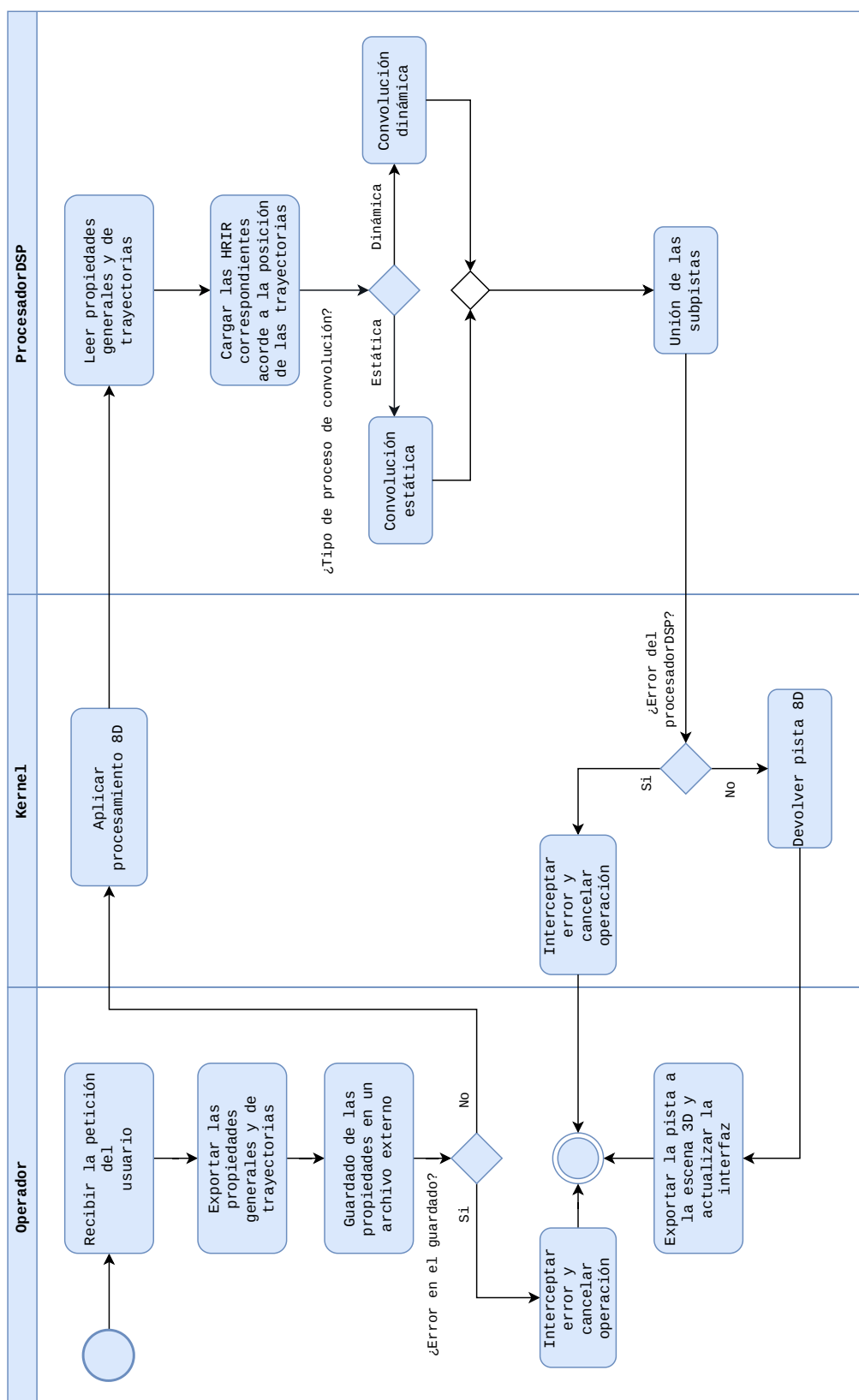


Figura A.1: Diagrama de actividad del CU-02

A.3. Bloque de código de la inicialización de paneles

```
#PANEL DE SEPARACION DE PISTAS POR AI
class AUDIO8D_PT_panel_separacion_ia(bpy.types.Panel):
    bl_idname = "AUDIO8D_PT_panel_separacion_ia"
    bl_label = "1. Separación IA"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = "Audio 8D"

    def draw(self, context):
        #Contenidos del panel...

#PANEL DE MODIFICACIÓN DE LAS TRAYECTORIAS DE LAS SUBPISTAS
class AUDIO8D_PT_panel_control(bpy.types.Panel):
    bl_label = "2. Modificación de trayectorias"
    bl_idname = "AUDIO8D_PT_panel_control"
    #Lo mismo para este panel...

#PANEL DE EXPORTACIÓN DE LA PISTA 8D
class AUDIO8D_PT_panel_exportacion(bpy.types.Panel):
    """Vista (Panel UI) para la exportación de la pista 8D."""

    bl_idname = "AUDIO8D_PT_panel_exportacion"
    bl_label = "3. Exportar pista binaural"
    #Lo mismo para este panel...

#INIT.PY DE PANELES
from . import panel_exportacion, panel_modificacion, panel_separacion_ia
#Inclusión de todas las clases UI
classes = (
    panel_separacion_ia.AUDIO8D_PT_panel_separacion_ia,
    panel_modificacion.AUDIO8D_PT_panel_control,
    panel_exportacion.AUDIO8D_PT_panel_exportacion,
)
#Registro/desregistro
def register():
    for cls in classes:
        bpy.utils.register_class(cls)

def unregister():
    for cls in classes:
        bpy.utils.unregister_class(cls)
```

Código A.1: Inicialización de los paneles del reproductor

Nota. Elaboración propia.

A.4. Bloque de código de la inicialización de operadores

```
#Imports...
from bpy_extras.io_utils import ImportHelper, ExportHelper
#INCLUSIÓN DE UN OPERADOR EN UN PANEL
class AUDIO8D_PT_panel_control(bpy.types.Panel):
    #Inclusión del operador "procesar_dsp" en una fila del panel "panel_control"
    row_boton.operator("audio8d.procesar_dsp",
                      text="Procesar Audio 8D",
                      icon='PLAY_SOUND')

    #...
#OPERADOR SEPARADOR DE PISTAS
class AUDIO8D_OT_separar_audio(bpy.types.Operator, ImportHelper):
    bl_idname = "audio8d.separar_audio"
    bl_label = "Seleccionar pista"
    bl_description = "Operador que abre el explorador, carga y separa la canción
                     seleccionada"

    def execute(self, context):
        #Flujo de ejecución del operador...

#OPERADOR GENERADOR DE PISTAS 8D
class AUDIO8D_OT_procesar_dsp(bpy.types.Operator):
    bl_idname = "audio8d.procesar_dsp"
    bl_label = "Procesar audio 8D"
    bl_description = "Operador que recoge las propiedades de las subpistas y
                     aplica el motor DSP"

    #...

#OPERADOR EXPORTADOR DE PISTAS 8D
class AUDIO8D_OT_exportar_pista(bpy.types.Operator, ExportHelper):

    bl_idname = "audio8d.exportar_pista"
    bl_label = "Exportar Pista Binaural"
    bl_description = "Abre el explorador para guardar la mezcla final y limpia
                     la caché"

    def invoke(self, context, event):
        #Condiciones previas al flujo de ejecución del botón...

    def execute(self, context):
        #Flujo de ejecución del operador...

#INIT.PY DE LOS OPERADORES
operators = (
    audio_ops.AUDIO8D_OT_separar_audio, audio_ops.AUDIO8D_OT_procesar_dsp,
    audio_ops.AUDIO8D_OT_exportar_pista
)
#Registro/desregistro...
```

Código A.2: Inicialización de los operadores del reproductor.

Nota. Elaboración propia.

A.5. Bloque de código de una trayectoria dinámica

```
#TRAYECTORIAS DINÁMICAS
#...
# 2, 3 y 4. Datos del modelo para realizar los cálculos y número de puntos
inclinacion_rad = math.radians(subpista.inclinacion_orbita)
resolucion = subpista.resolucion_espacial
tipo = subpista.tipo_trayectoria
salto_frames = 61 - resolucion
num_puntos = total_frames // salto_frames
coordenadas_blender = []
radio_xy = radio * math.cos(elevacion_rad)
z_offset = radio * math.sin(elevacion_rad)
matriz_inclinacion = mathutils.Euler((0, inclinacion_rad, 0), 'XYZ').to_matrix()
#5. Computo de vértices cartesianos
for i in range(num_puntos + 1):
    progreso = i / num_puntos
    angulo_actual = (progreso * 2 * math.pi) + azimut_rad
    if tipo == 'CIRCULAR':
        x_base = radio_xy * math.sin(angulo_actual)
        y_base = radio_xy * math.cos(angulo_actual)
    elif tipo == 'CUADRADA':
        #Cálculos para la trayectoria cuadrada...
        #Matriz de inclinación y vector con el vértice cartesiano
        z_base = z_offset
        vector_rotado = matriz_inclinacion @ mathutils.Vector((x_base, y_base,
            z_base))
        coordenadas_blender.append(vector_rotado)
#Asignación de los vértices para construir la curva
spline = curve_data.splines.new('POLY')
spline.points.add(len(coordenadas_blender) - 1)
for i, coord in enumerate(coordenadas_blender):
    spline.points[i].co = (coord.x, coord.y, coord.z, 1.0)
#Asignación de la malla con los puntos que forman la curva
malla_instancer = bpy.data.meshes.new(f"MeshInstancer_{nombre_pista}")
malla_instancer.from_pydata([c.to_tuple() for c in coordenadas_blender], [], [])
#Todos los objetos que conforman la pista dinámica están asignados y heredados
    según la explicación realizada en la fase de implementación correspondiente
#...
```

Código A.3: Sistema de generación de trayectorias dinámicas.

Nota. Elaboración propia.

A.6. Bloque de código de la convolución dinámica

```
#PROCESADOR DSP -> CONVOLUCIÓN DINÁMICA
global hrir_globales
#1. Variables globales
ruta_stem = datos_pista["ruta_stem"]
coordenadas = datos_pista.get("coordenadas", [])
audio_stem, fs_stem = sf.read(ruta_stem)
#2. Conversión a mono para aplicar el impulso (Espacialización por
    lateralización)
audio_stem = np.mean(audio_stem, axis=1)
#3. Obtención del tamaño del bloque a procesar
resolucion = datos_pista.get("resolucion_espacial", 30)
salto_frames = 61 - resolucion
muestras_por_bloque = int((frecuencia_muestreo / fps_escena) * salto_frames)
total_muestras = len(audio_stem)
n_bloques = int(np.ceil(total_muestras / muestras_por_bloque)) #Equivalente
    1:1 al número de vértices con resolución 60!
num_coordenadas = len(coordenadas)
#4. Longitud de un filtro HRIR para saber cuanto tiene que medir el master
    final y aplicación de la inversa del cuadrado conforme la amplitud
r = coordenadas[0]["r"]
az_0 = coordenadas[0]["azimut"]
el_0 = coordenadas[0]["elevacion"]
ganancia = 1.0 / r
longitud_hrir = len(hrir_globales[(az_0, el_0)[: , 0])
#5. Matriz con la salida de audio conjunta de todas las pistas
audio_salida = np.zeros((total_muestras + longitud_hrir - 1, 2))
#Convolución por bloques (Overlap-Add)
for i in range(n_bloques):
    #6. Obtenemos el bloque de audio correspondiente al índice actual
    inicio = i * muestras_por_bloque
    fin = min(inicio + muestras_por_bloque, total_muestras)
    bloque_audio = audio_stem[inicio:fin]
    #7. Obtenemos el radio y el par de coordenadas (azimut, delta) a aplicar
        correspondiente al índice actual
    i_coord = i % num_coordenadas
    az = coordenadas[i_coord]["azimut"]
    el = coordenadas[i_coord]["elevacion"]
    #Mismo procedimiento que una convolución estática...
    #10. Únimos ambas señales en una matriz estéreo y sumamos el bloque al
        audio de salida final
    fin_buffer = inicio + len(bloque_conv_L)
    audio_salida[inicio:fin_buffer] += audio_estereo_bloque
#Aplicación de la ganancia de la pista dinámica...
```

Código A.4: Proceso de convolución dinámica.

Nota. Elaboración propia.

A.7. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Descripción general

En la elaboración de este TFG se ha utilizado inteligencia artificial generativa como herramienta de asistencia técnica y soporte en la redacción. Su uso se ha centrado exclusivamente en la resolución de dudas sobre la maquetación del documento, la generación de código Latex para la estructuración de tablas y diagramas, y la revisión ortotipográfica y de estilo de ciertos fragmentos del texto.

Herramientas específicas

Para las labores de asistencia en la redacción y maquetación se ha empleado el modelo de lenguaje Gemini.. Cabe destacar que, de forma independiente a esta IA conversacional, el proyecto técnico incorpora un modelo de IA entrenado para la separación de fuentes musicales, cuyo uso está debidamente documentado en los capítulos correspondientes de la memoria al ser parte central del desarrollo del software.

Propósito y alcance

El propósito exclusivo del uso de IA generativa ha sido agilizar la presentación formal del documento y optimizar su legibilidad. El alcance se ha limitado a tareas de asistencia en la edición de texto y formato, sin delegar en ningún momento la búsqueda de información teórica ni la toma de decisiones técnicas.

Impacto en el contenido

La inteligencia artificial no ha influido en el desarrollo de la arquitectura del sistema, el análisis de los resultados obtenidos en las pruebas empíricas, ni en las conclusiones finales del proyecto. Su impacto se reduce estrictamente a la mejora estructural y gramatical de la memoria escrita.

Mantenimiento de la integridad académica

Para garantizar la originalidad y la integridad académica del trabajo, todas las sugerencias de formato y estilo proporcionadas por la IA fueron revisadas y validadas de forma crítica personalmente por el autor. El diseño del software, la programación del procesador DSP, la ejecución de las pruebas y la autoría intelectual del contenido general es responsabilidad exclusiva del autor.

Bibliografía

- Agrawal, S., & Reiss, J. D. (2015). *Spatial audio through head-related transfer functions*. ResearchGate. https://www.researchgate.net/figure/nteraural-Time-Difference-ITD-and-Interaural-Level-Difference-ILD_fig1_276473132
- Algazi, V. R., Duda, R. O., Thompson, D. M., & Avendano, C. (2001). The CIPIC HRTF database. *Proceedings of the 2001 IEEE workshop on the applications of signal processing to audio and acoustics*, 99-102.
- Audioalter. (2026). 3D audio: Create an immersive 3D experience. <https://audioalter.com/3d-audio>
- Begault, D. R. (2000). *3-D sound for virtual reality and multimedia* (inf. téc. N.º NASA/TM-2000-209606). National Aeronautics and Space Administration.
- Blauert, J. (1997). *Spatial hearing: The psychophysics of human sound localization*. MIT Press.
- Blender Foundation. (2026). *Blender Python API documentation*. <https://docs.blender.org/api/current/index.html>
- Défossez, A., Usunier, N., Bottou, L., & Bach, F. (2019). Music source separation in the waveform domain. *arXiv*. <https://arxiv.org/abs/1911.13254>
- eMe - Escuela Universitaria de Música. (2020). *Psicoacústica: Conos de confusión*. <https://www.eumus.edu.uy/eme/ensenanza/acustica/presentaciones/psicoac/localizacion/12.html>
- Gardner, B., & Martin, K. (1994). *HRTF measurements of a KEMAR dummy-head microphone* (inf. téc. N.º 280). MIT Media Lab Perceptual Computing.
- Gumm, A. J. (2012). Psychoacoustics in music. <https://alanjgumm.wordpress.com/psychoacoustics-in-music/>
- Hays. (2026). *Calculadora salarial: Descubre tu salario ideal*. <https://guiasalarial.hays.es/profesional/calculadora-salarial>
- Langendijk, E. H. A., & Bronkhorst, A. W. (2002). Contribution of spectral cues to human sound localization. *The Journal of the Acoustical Society of America*, 112(4), 1583-1596.
- Ministerio de la Presidencia, Justicia y Relaciones con las Cortes. (2026). Orden PJC/297/2026, de 30 de marzo, por la que se desarrollan las normas legales de cotización a la Seguridad Social, desempleo, protección por cese de actividad, Fondo de Garantía Salarial y formación profesional para el ejercicio 2026. *Boletín Oficial del Estado*, (79), 47467. https://www.boe.es/diario_boe/txt.php?id=BOE-A-2026-7296
- Monash University. (2024). Electric fields. <https://www.monash.edu/student-academic-success/physics/fields-and-interactions/electric-fields>
- Moore, B. C. J. (2013). *An introduction to the psychology of hearing* (6.ª ed.). Brill.
- Park, S., & Choi, Y. H. (2025). Three-dimensional visualization in biological and medical research: A comparative analysis of Blender, Unity and Unreal Engine. *Journal of*

- Young Investigators*, 28(12), 1-10. <https://www.jyi.org/2025-december/2025/12/13/three-dimensional-visualization-in-biological-and-medical-research-a-comparative-analysis-of-blender-unity-and-unreal-engine>
- Raffi, Z., Liutkus, A., Stöter, F.-R., Mimitakis, S. I., & Bittner, R. (2019, diciembre). MUSDB18-HQ - an uncompressed version of MUSDB18. <https://doi.org/10.5281/zenodo.3338373>
- Rayleigh, L. (1907). XII. On our perception of sound direction. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 13(74), 214-232.
- Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.
- Sennheiser. (2024). *AMBEO spatial audio: Herramientas de audio inmersivo*. <https://www.sennheiser.com/es-mx/immersive/ambeo-spatial-audio>
- Serway, R. A., Jewett, J. W., & Peroomian, V. (2000). *Physics for scientists and engineers* (Vol. 2). Saunders College Publishing.
- Tipler, P. A., & Mosca, G. (2007). *Physics for scientists and engineers: With modern physics* (6.^a ed.). W. H. Freeman and Company.
- VanderPlas, J. (2016). *Python data science handbook*. O'Reilly Media.
- Verfaillie, V., Arfib, D., Keiler, F., von dem Knesebeck, A., & Zölzer, U. (2011). Adaptive digital audio effects. En *DAFX: Digital audio effects* (pp. 321-391). Wiley.
- Warusfel, O. (2002). *LISTEN HRTF database*. <http://recherche.ircam.fr/equipes/salles/listen/>
- Xie, B. (2013). *Head-related transfer function and virtual auditory display*. J. Ross Publishing.